



Eötvös Loránd Tudományegyetem

Informatikai Kar

Informatikatudományi Intézet

Programozáselmélet és Szoftvertechnológiai Tanszék

Jelzőtáblák megtervezése és algoritmikus elhelyezése közúti csomópontokra

Szerző:

Csuport Ákos

Programtervező informatikus BSc.

Témavezető:

Kitlei Róbert László

Egyetemi tanársegéd

Budapest, 2026

EÖTVÖS LORÁND TUDOMÁNYEGYETEM
INFORMATIKAI KAR

SAKDOLGOZAT TÉMABEJELENTŐ

Hallgató adatai:

Név: Csuport Ákos

Neptun kód: CKCBS0

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Kitlei Róbert László

munkahelyének neve, tanszéke: ELTE IK, Programozási nyelvek és Fordítóprogramok Tanszék

munkahelyének címe: 1117, Budapest, Pazmány Péter sétány 1/C.

besorolás és iskolai végzettsége: egyetemi tanársegéd

A szakdolgozat címe: Jelzőtáblák megtervezése és algoritmikus elhelyezése közötti csomópontokra

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

A szakdolgozat témája egy asztali alkalmazás fejlesztése, melyben a felhasználó kezdésképp egy térképen választhat ki egy csomópontot. Ehhez a program megtervezi az Utügyi Műszaki Előírások szabályrendszere alapján a közötti jelzőtáblákat és jelzőlámpákat. Ezeket megjeleníti a csomópont nagyrátható és mozgatható vázlatán, valamint egyesével menthetővé teszi SVG formátumban.

A fent említett szabályok között van például az egyirányú utcák megfelelő végén a „Behajtani tilos” és „Egyirányú utca” jelzőtáblák, valamint a többi irányból a korlátozást jelző „Kötelező haladási irány” táblák megfelelő elhelyezése a vázlaton. A program olyan feltételekhez kötött szabályokat is figyelembe vesz, mint a kereszteződésre figyelmeztető „Útkereszteződés alárendelt úttal” tábla elhelyezése a védett irányokban, ha a csomópont lakott területen kívül található és nem jelzőlámpával irányított.

Az egyes irányokból, ha a feltételek ezt megkövetelik, a sávbesorolás rendjét jelző táblákat is megtervezi az alkalmazás a sávok irányát és tulajdonságait feltüntetve. A program szintén képes az útbiztonsági táblákat megtervezni, amikre a feltüntetendő úticélok az út típusa, a települések távolsága, előírásban megjelölt kategóriája és a tartalmi elvek alapján szabja meg. A tábla tartalmának elrendezését és az egyes grafikai elemek hozzáadását szintén a szabályrendszer határozza meg.

A felhasználó továbbá egy szerkesztővel testre szabhatja a vázlaton elhelyezett útbiztonsági táblákat, módosíthatja a feltüntetett információkat, valamint ezzel a szerkesztővel milliról is tervezhet táblákat.

Budapest, 2025. 10. 14.

Tartalomjegyzék

1.	Bevezetés	1
1.1.	Általános fogalmak	4
2.	Felhasználói dokumentáció	7
2.1.	Rendszerkövetelmények	7
2.1.1.	Szerver	7
2.1.2.	Asztali alkalmazás	7
2.2.	A szoftver indítása	7
2.2.1.	Szerver	7
2.2.2.	Alkalmazás	8
2.3.	Az alkalmazás használata	8
2.3.1.	Menü	9
2.3.2.	Térkép	9
2.3.3.	Tervrajz	12
2.3.4.	Táblaszerkesztő	17
3.	Fejlesztői dokumentáció	22
3.1.	Áttekintés	22
3.2.	Szerver	24
3.2.1.	Definíciók	24
3.2.2.	A szerver áttekintése	25
3.2.3.	Végpontok	27
3.2.4.	Csomagstruktúra	33
3.2.5.	A tervrajzkomponens dokumentációja	38
3.2.6.	Prezentációs komponens	87
3.3.	Alkalmazás	98
3.3.1.	Áttekintés	98

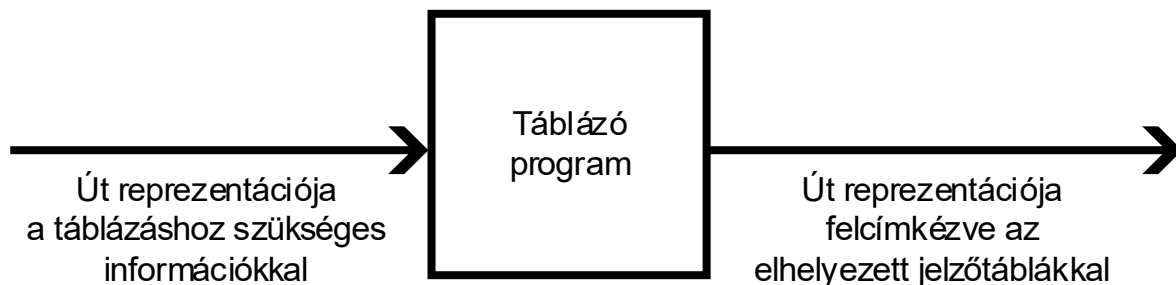
3.3.2.	Az alkalmazás általános stílusa.....	100
3.3.3.	Az alkalmazás típusdefiníciói.....	101
3.3.4.	API-hívások	101
3.3.5.	Tervrajz tárolása az alkalmazás futása során	102
3.3.6.	Oldalak és nézetkomponensek	103
3.4.	Tesztelés	114
3.4.1.	A tervrajz táblázási pontosságának és pontatlanságainak tesztelése	114
3.4.2.	Az alkalmazás tesztelése felhasználói történetekkel	127
3.5.	Mesterséges intelligencia használata a fejlesztés során.....	141
4.	Összefoglalás és további fejlesztési lehetőségek	141
5.	Irodalomjegyzék	143

1. Bevezetés

A közutak mentén található közúti jelzőtáblák (a továbbiakban „táblák” vagy „jelzőtáblák”) elhelyezése értelemszerűen szabályok alapján történik. Az általános szabályokat a 83/2004. (VI. 4.) GKM rendelet melléklete, *„A közúti jelzőtáblák megtervezésének, alkalmazásának és elhelyezésének követelményei” műszaki szabályzata* [2] (a továbbiakban JETSZ) határozza meg.

A JETSZ-et vizsgálva felmerült az az elképzelés, hogy a jelzőtáblák elhelyezésének szabályai vajon alkalmazhatók-e (legalább részben) automatikusan, ha egy erre szolgáló program megkapja bemenetként az útnak egy olyan kódolását, amiben rendelkezésre áll a táblák alkalmazásához szükséges információ, mint a távolság fogalma vagy az elsőbbségadási kötelezettségek helyszíne.

A bevezető további részében az alábbi lesz a táblázó program (1. ábra), melynek feladata felcímkézni az utat a megfelelő jelzőtáblák egy kódolásával (innenről az út táblákkal való felcímkézésével ekvivalens kifejezés a jelzőtáblák elhelyezése vagy az út táblázása).



1. ábra: A program működésének egyszerű diagramja

Vegyük példának a kijelölt gyalogos-átkelőhelyek (átkelők) táblázását. A JETSZ 18.9. a) pontja szerint a „Kijelölt gyalogos-átkelőhely” táblát (E-038, 2. ábra) az átkelőhely kezdeténél, de tőle legfeljebb 2 méterre kell elhelyezni. Továbbá egy kijelölt gyalogos-átkelőhely előjelezhető a „Gyalogátkelés” veszélyt jelző táblával (A-020, 3. ábra).



2. ábra: „Gyalogátkelés” (A-020)



3. ábra: „Kijelölt gyalogos-átkelőhely” (E-038)

A JETSZ 8.1.1. pontja ebben az esetben, mint ahogy a többi veszélyt jelző tábla esetében, a tábla elhelyezését lakott területen belül a veszélyforrástól (ebben az esetben az átkelőtől) 50-100, lakott területen kívül 150-250 méterre írja elő. Feltéve, ha az út reprezentációja tartalmazza az átkelőket, egy átkelőnél a táblázó program el tudja az útra helyezni annak közvetlen közelében a „Kijelölt gyalogos-átkelőhely” táblát, majd attól függően, hogy az út lakott területen belül vagy kívül van (feltéve, hogy a reprezentáció tartalmazza ezt az adatot), egy megfelelő távolságra a „Gyalogátkelés” táblát.

A dolgozat fő célja így egy olyan modell létrehozása, mely tartalmazza egy, a jelzőtáblák elhelyezéséhez szükséges információkat magában foglaló reprezentációját egy úthálózatnak vagy útszakasznak és egy olyan algoritmus létrehozása, mely ezt a reprezentációt bejárja és elhelyezi rajta a jelzőtáblákat, valamint annak vizsgálata, hogy egy ilyen reprezentáció mennyire engedi meg nekünk a szabályok pontos alkalmazását.

Emellett szintén cél ennek a modellnek valamilyen hasznosítása: egy asztali alkalmazás létrehozása, mely megengedi a felhasználónak egy ilyen útreprezentáció létrehozását egy térképen történő kijelöléssel, majd miután a térképrészleten szereplő utak útreprezentációvá alakított adataira a táblázó program elhelyezte a táblákat, annak megtekintését, szerkesztését, mentését és betöltését. Ezzel könnyen lehet bemenetet készíteni a táblázó programnak, vizualizálni annak kimenetét, és szerkeszteni a jelzőtáblákat a már felcímkézett úton.

Az út mentén lévő jelzőtáblák megjelenítéséhez azonban szükség van a táblák elkódolásának képpé alakításához. Bizonyos jelzőtáblák esetében (pl. „Főútvonala”) ez annyiból áll, hogy a tábla azonosítója alapján ki kell választani a megfelelő jelzést, azonban más táblák (pl. „Besorolási rend”) jelképét egyedileg meg kell tervezni. A felhasználónak a tábla neve is fontos információ lehet, így kell egy olyan program, ami a tábla elkódolását bemenetként használva képes annak nevét és képét kiadni. Ez lesz a prezentáló program (a program a tábla elkódolását névvel és képpel képes a felhasználónak prezentálni).

Az előzőket összefoglalva a cél olyan szoftver fejlesztése, melyben

- az alkalmazáson belül egy térképen ki lehet választani egy részletet, ami utána egy automatikus átalakítás után a táblázó program bemenete lesz,
- miután ezt a bemenetet feldolgozta a táblázó program, az alkalmazás megkapja a kimenetet, és a felhasználónak ezt megjeleníti, a prezentáló program segítségével.
- A megjelenített, táblázott úton a táblázást a felhasználó az alkalmazásban képes szerkeszteni, menteni és betölteni.

A szoftver nem fog nyújtani optimális táblázást minden forgalmi helyzetre, azonban közútkezelők segítségével lehet akár közutakon a már meglévő táblázás helyességének ellenőrzésére, vagy új forgalmi rend kialakítása esetén a szabályos „kezdőállapot” elrendezésére, melyből az alkalmazás szerkesztő funkciójával ki lehet alakítani egy egyedi táblázást. Szintén hasznos lehet a táblázó és a prezentáló program utak digitális modelljének kitáblázására, például interaktív oktatóanyagok vagy videojátékok esetén, ahol egy elkészített vagy akár procedurálisan generált úton kell elhelyezni jelzőtáblákat.

1.1. Általános fogalmak

A szoftverben és ebben a dokumentumban bevezetett vagy specifikus jelentéssel felruházott szavak és kifejezések itt találhatók. A szerver működésének leírásában jelentős, de az alkalmazás felhasználóinak és az alkalmazás általános működéséhez nem releváns definíciók a fejlesztői dokumentáció arra vonatkozó részében vannak leírva (3.2.1). Az egyes meghatározásoknál a kifejezés mellett zárójelben található a dokumentumban használt alternatívák, amik megfelelnek az eredeti kifejezésnek.

- **Befogó téglalap:** egy terület a Földön, melyet egy déli és északi szélességi fok, valamint egy nyugati és keleti hosszúsági fok határoz meg.
- **Felhasználói alkalmazás (alkalmazás):** az a futtatható, grafikus felülettel rendelkező alkalmazás, mely a szerverrel közösen lehetővé teszi a felhasználó számára a tervrajzok elkészítését térképes kijelölés alapján, majd a tervrajz szerkesztését, mentését, betöltését, illetve jelzőtáblák szerkesztését és jelképek exportálását.
- **Jelzőlámpák csoportja (fényjelző készülékek, jelzőlámpa, lámpacsoport):** a szoftverben a jelzőtábla egy speciális esete, olyan tábla, melynek kódja „JL” és mely az úton egy adott pontra vonatkozó úttest felett és úttest mellett elhelyezett jelzőlámpákat reprezentálja; a táblázó program vonatkozó sávonként balról jobbra reprezentálja a jelzőlámpákat, az út felett és mellett megismételt, de egy közúti sávra vonatkozó jelzőlámpákat nem számítva (3. ábra). (A sávot itt felfoghatjuk közúti sávnak, egyébként szoftverbeli definíciója a 3.2.5.5.1. fejezetben található.)



3. ábra: Egy jelzőlámpa-elrendezés két sávra egy kereszteződésben és a szoftveres reprezentációjának jelképe [2]

- **Jelzőtábla** (tábla): a közúti jelzőtáblák reprezentációja a szoftverben, mely egy kódból és testre szabható jelzőtábla esetén a jelzőtábla adataiból áll. A jelzőlámpák csoportja a jelzőtábla egy speciális esetének számít.
- **Jelzőtábla adatai** (tábla adatai, adatok): a testre szabható jelzőtáblák reprezentációjához magán a tábla kódján felül szükséges információ, például „Sebességkorlátozás” jelzőtábla esetén a sebességhatár felirata.
- **Jelzőtábla jelképe** (tábla jelképe, jelkép): a jelzőtábla megjelenése, melynek megvalósítása a szoftverben SVG formátumú vektorgrafikákkal történik.
- **Jelzőtábla kódja** (tábla kódja, táblakód): a jelzőtábla típusát meghatározó kód, mely alapján egyértelmű a jelzőtábla neve, és egyszerű jelzőtáblák esetén a jelképe, de testre szabható jelzőtáblák esetén is alapvetően meghatározza a jelképet. Ha a tábla megfeleltethető egy műszaki előírásban szereplő bejegyzett táblának, annak „A-123” formátumú kódját veszi át. Ha nem, akkor a szoftveren belül meghatározott, betűkből, számokból és kötőjelből álló kódot kap a jelzőtábla.
- **Jelzőtábla-kompozíció** (táblakompozíció, kompozíció): egy tervrajzon jelzőtáblák egységként működő listája, melynek sorrendjét tervrajz elkészítésekor a szerver kezeli, majd a felhasználó az alkalmazásban szabadon szerkesztheti. A táblakompozícióhoz tartozik a pozíciója és az útszakasz menti iránya (előre vagy hátra). A jelzőtábla-kompozíció megfeleltethető az utakon egy oszlopra elhelyezett közúti jelzőtábláknak (4. ábra).



4. ábra: egy oszlopra elhelyezett jelzőtáblák és a vele ekvivalens táblakompozíció megjelenése az alkalmazásban.

- **Kijelölt terület:** az alkalmazásban a Térkép lap rajzoló módjában (2.3.2.3. fejezet) kijelölt terület.
- **Kiválasztott terület:** a kijelölt terület befogó téglalapja, melyet az alkalmazás elküld a szervernek, hogy tervrajzot készítsen a terület OSM térképes adataiból.
- **OpenStreetMap (OSM):** az OpenStreetMap Foundation által fenntartott földrajzi adattár, melyen alapszik az alkalmazás Térkép lapjának (2.3.2. fejezet) megjelenítése, és aminek az adatait az Overpass API-on keresztül lekérdezi a szerver a tervrajz elkészítéséhez térkép alapján.
- **Overpass API:** az az API, melyet a szerver használ a kiválasztott terület OpenStreetMapen található adatainak lekérdezésére.
- **Szerver:** háttérben futó, grafikus felülettel nem rendelkező állapot nélküli program, mely felelős a tervrajzok elkészítéséért, és a jelzőtáblák prezentálásáért, tehát a jelképük elkészítéséért és a megnevezésükért.
- **Szoftver:** a szerver és az alkalmazás együttese.
- **Táblázó program:** a szerver azon része, amely egy tervrajz modelljére elhelyezi egy szabályrendszer szerint a jelzőtáblákat.
- **Tervrajz:** egy, a szerver által készített gráfként ábrázolt úthálózat, mely utakból és pontokból áll, amire a szerveren alkalmazható egy szabályhalmaz jelzőtáblák elhelyezéséhez. A szabályhalmaz definíciója és a tervrajz részleteinek definíciói a fejlesztőknek lehetnek hasznosak, a fejlesztői dokumentáció szerverről szóló részében található (3.2.5.5. fejezet).
- **Út:** a tervrajz pontjait összekötő, tulajdonságokkal feldíszített élek, melyekre a szerver elhelyezi a táblakompozíciókat, és amelyek a felhasználónak a táblakompozíciók mellett részét képezik a tervrajz nézetének. Az út modelljének csak fejlesztői szemszögből hasznos meghatározása a fejlesztői dokumentáció szerverről szóló részében található (3.2.5.5.1. fejezet).

2. Felhasználói dokumentáció

2.1. Rendszerkövetelmények

2.1.1. Szerver

- Java Runtime Environment 21 (vagy újabb)
- Operációs rendszer: Windows 11, Windows Server 2016 vagy újabb, Oracle Linux 7 vagy újabb, Ubuntu 22.04 LTS vagy újabb, RHEL 7 vagy újabb¹
- x64 architektúrájú processzor¹
- 4 GB RAM²
- 50 MB szabad tárhely³

2.1.2. Asztali alkalmazás

- Windows 10 vagy újabb, MacOS Monterey vagy újabb, Ubuntu 18.04 vagy újabb, Fedora 32 vagy újabb, Debian 10 vagy újabb⁴
- x64 vagy arm64 architektúrájú processzor
- 4 GB RAM
- 400 MB szabad tárhely⁵

2.2. A szoftver indítása

2.2.1. Szerver

A szerver indítása parancssorból ajánlott. A szerver JAR-fájljának mappájából kell futtatni az alábbi parancsot:

```
java -jar road-template-and-signage-server-1.0-runner
```

A szerver ekkor elindul, és a parancssorban megjelenik a napló.

¹ Az Oracle JDK 21 ajánlott konfigurációi alapján [3]

² A támogatott operációs rendszerek (Ubuntu, Windows 11) irányadó minimális RAM-igénye. Az alkalmazás intenzívebb igénybevétel során sem használt 300 MB-nál több RAM-ot, illetve átlagos kérések feldolgozása során 100-150 MB memóriahasználat jellemző rá.

³ A futtatható „uber-jar” fájl mérete 31 MB.

⁴ Az alkalmazást működtető ElectronJS keretrendszer követelménye [4]

⁵ A lefordított, becsomagolt alkalmazás mérete 397 MB.

2.2.2. Alkalmazás

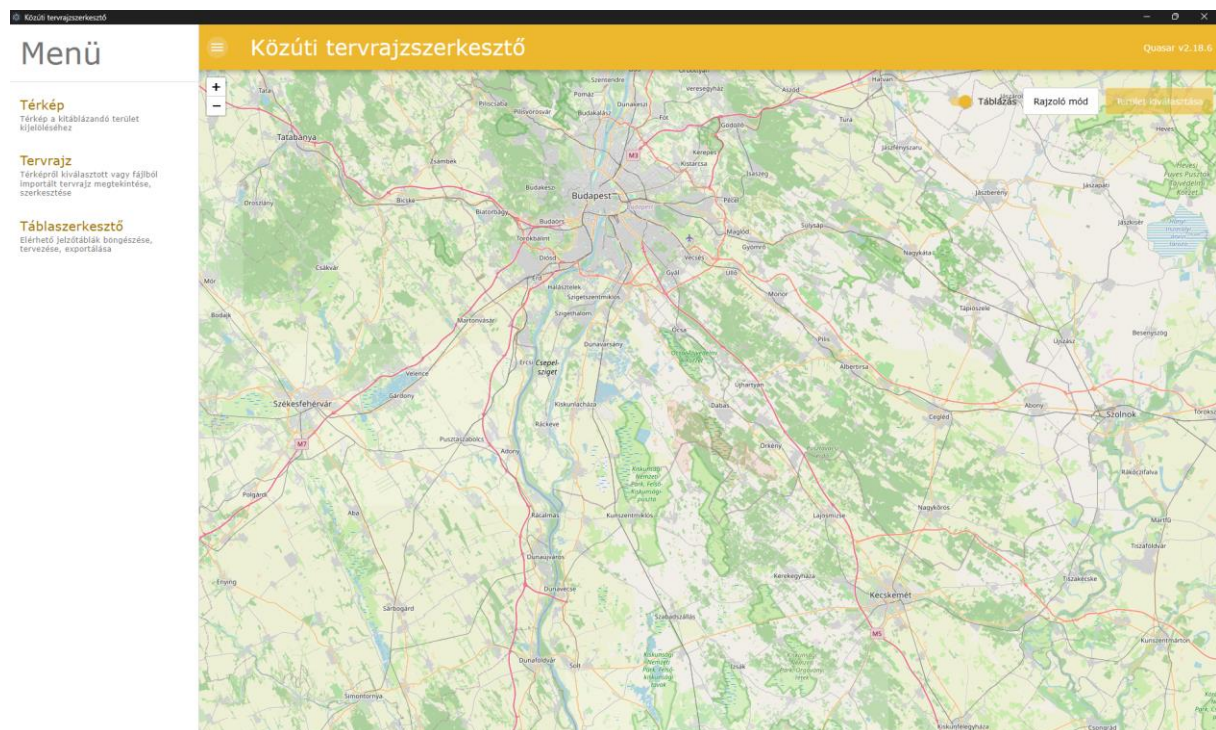
Az alkalmazás működésének feltétele, hogy fusson a szerver. Ha a szerver az alkalmazást futtató gépen megy, nincs előzetes teendő (az alkalmazás a szervert a `localhost:8080` címen éri el). Amennyiben a szerver távoli gépen fut, annak URI-ját meg kell adnunk az alkalmazás gyökérkönyvtárában található `resources` mappa `app-config.json` fájljában így:

```
{  
  
  "apiBaseUrl": "<elérési cím>"  
  
}
```

Az alkalmazás futtatásához a mappájában a „Közúti tervrajzszerkesztő” nevű alkalmazást kell elindítani.

2.3. Az alkalmazás használata

Az alkalmazás kezelőfelülete egy fejlécből, egy oldalsó menüből és három különböző lapból áll. Indításkor a felhasználó a Térkép (2.3.2. fejezet) lapon kezd (5. ábra).



5. ábra: Az alkalmazás kezdőképernyője

2.3.1. Menü

A menü az alkalmazásban az ablak bal oldalán nyitható meg. Alapesetben látszik, a fejléc bal szélén található, három vonalat ábrázoló gombbal lehet elrejteni, valamint újra megjeleníteni.

A menü megengedi a felhasználónak, hogy navigáljon az alkalmazás három lapja között:

- Térkép
- Tervrajz
- Táblaszerkesztő

2.3.2. Térkép

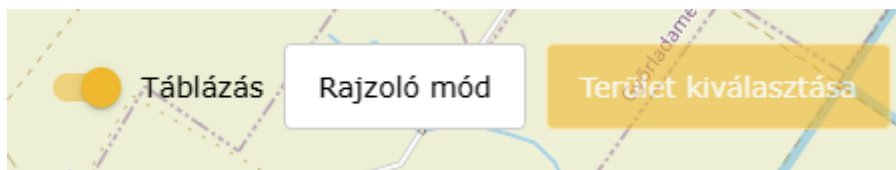
A Térkép lapon (5. ábra) egy interaktív térképet tud használni a felhasználó a tervrajzzá alakítandó terület kijelöléséhez.

2.3.2.1. Navigáció a térképen

A térkép területén böngésző módban a bal egérgomb lenyomása közben az egér mozgásával képes a felhasználó mozgatni a beágyazott térképet. Ez csak böngésző módban lehetséges. A nagyítás és kicsinyítés a görgővel, vagy a lap bal felső sarkában található „+” és „-” jelű gombokra kattintva lehetséges, ez mind böngésző, mind rajzoló módban működik.

Amennyiben a térképet alkotó négyzetek hiányosak (fehér foltok vannak a térképen), vagy nagyítás után a térkép egyes részletei pixelesen jelennek meg, ajánlott az internetkapcsolat ellenőrzése.

2.3.2.2. Vezérlőpanel



6. ábra: A vezérlőpanel

A lap jobb felső sarkában található három vezérlő (6. ábra):

- **Táblázás:** ez a kapcsoló állítja be, hogy a terület kiválasztását és tervrajzzá alakítását követően a szerveren a táblázó program elhelyezze-e a jelzőtáblákat. Ha ki van kapcsolva, a kiválasztott terület tervrajza táblák nélkül készül el, és a felhasználó a Tervrajz lapon a nulláról táblázhatja ki a tervrajzot.

- **Rajzoló mód/Böngésző mód** kapcsológomb: a fehér gomb megnyomásával a felhasználó átkapcsol a gomb címkéjén olvasható módba. A kiválasztott mód a bal egérgomb lenyomásának a parancsát állítja, böngésző módban a térkép mozgását (2.3.2.1. fejezet), rajzoló módban a kiválasztott terület téglalapjának megrajzolását (2.3.2.3. fejezet) teszi lehetővé.
- **Terület kiválasztása:** a gomb csak akkor aktív, ha a térképen van kiválasztott terület. A gomb lenyomására az alkalmazás elkészítteti a szerverrel a kijelölt terület tervrajzát, majd megjeleníti azt a Tervrajz lapon. További részletek a Terület kijelölése és kiválasztása (2.3.2.3) részben olvashatók.

2.3.2.3. Terület kijelölése és kiválasztása

Miután a felhasználó a kapcsológommbal rajzoló módba kapcsolt, a bal egérgomb lenyomásával létrehozhat a térképen egy téglalapot. A téglalapban található terület a kijelölt terület. Ha van kijelölt terület, kattintható lesz a „Terület kiválasztása” gomb a vezérlőpanelen. A gombra kattintva az „Töltés...” feliratot mutat és inaktív lesz a mód kapcsolójával együtt, miközben alkalmazás elküldi a szervernek a térképen kijelölt terület befoglaló téglalapját, ami alapján a szerver

1. lekérdezi a területen található földrajzi adatokat az internetről,
2. átalakítja az adatokat az alkalmazás által is olvasható tervrajzzá,
3. ha a táblázás be volt kapcsolva a vezérlőpanelen, a táblázó programmal elhelyezi a jelzőtáblákat,
4. végül elküldi a tervrajzot az alkalmazásnak.

Amint az alkalmazás kommunikációja a szerverrel véget ért, a vezérlőpanel újra teljesen használható lesz.

Amennyiben a művelet sikeres, az alkalmazás átirányítja a felhasználót a Tervrajz lapra (2.3.3. fejezet).



7. ábra: a felugró ablak sikertelen betöltés esetén

Amennyiben sikertelen, egy felugró ablak jelenik (7. ábra) meg az alábbi hibaüzenetek egyikével:

- *„Az Overpass API-ról nem sikerült a lekérdezés 4 próbálkozás után. Ha többször is előfordul a probléma, próbáld meg pár perc múlva”*
 - Az Overpass API nem küldte el a kiválasztott terület földrajzi adatait. A szolgáltatás gyakran túlterhelt, ilyenkor időtúllépés miatt nem tud a szerver lekérdezni adatokat. A hiba előfordulásakor érdemes egyszer-kétszer újra megpróbálni a terület kiválasztását, majd, ha továbbra is megjelenik a hibaüzenet, várni pár percet.
- *„Nem található feldolgozható útvonal az adott területen.”*
 - Nincs út a kiválasztott területen a tervrajz elkészítéséhez. A felhasználónak olyan területet kell kiválasztania, amin van útszakasz, amit a szerver képes tervrajzzá alakítani.
- *„A szerver nem ismerte fel az Overpass API állomásnevét. Lehetséges, hogy a szervernek nincs internetkapcsolata.”*
 - Ebben az esetben a legvalószínűbb hiba az, ami az üzenetben is szerepel lehetőséggént: a szervernek nincs internetkapcsolata. A felhasználó próbálja újra a terület kiválasztását a kapcsolat helyreállta után.

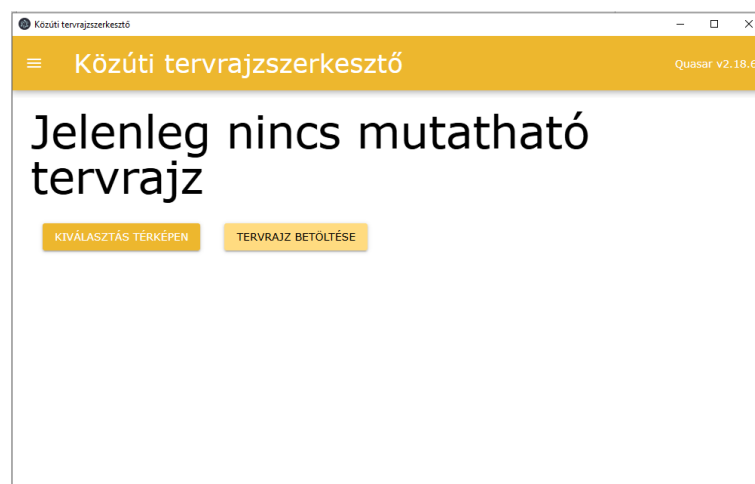
- *„Nem sikerült választ szerezni a szerver felől”*
 - Ebben az esetben az alkalmazás nem éri el a szervert. A szerver nem fut, vagy nem azon a címen található, amin az alkalmazás keresi.
- *„Hiba történt a feldolgozás során”*
 - Ebben az esetben a szerverben belső hiba történt, miközben készítette a kiválasztott terület alapján a tervrajzt. Ha ismételt kiválasztáskor is megjelenik az üzenet, a szerver a területet nem képes feldolgozni valamilyen belső hibát előidéző tényező miatt. Ekkor érdemes szűkíteni a kijelölt területen, hátha kikerül a hibát okozó pont.

2.3.3. Tervrajz

A Tervrajz lap az alkalmazás jelenleg tárolt tervrajzát jeleníti meg. Az alkalmazás egyszerre egy tervrajzot tud tárolni és két forrása van:

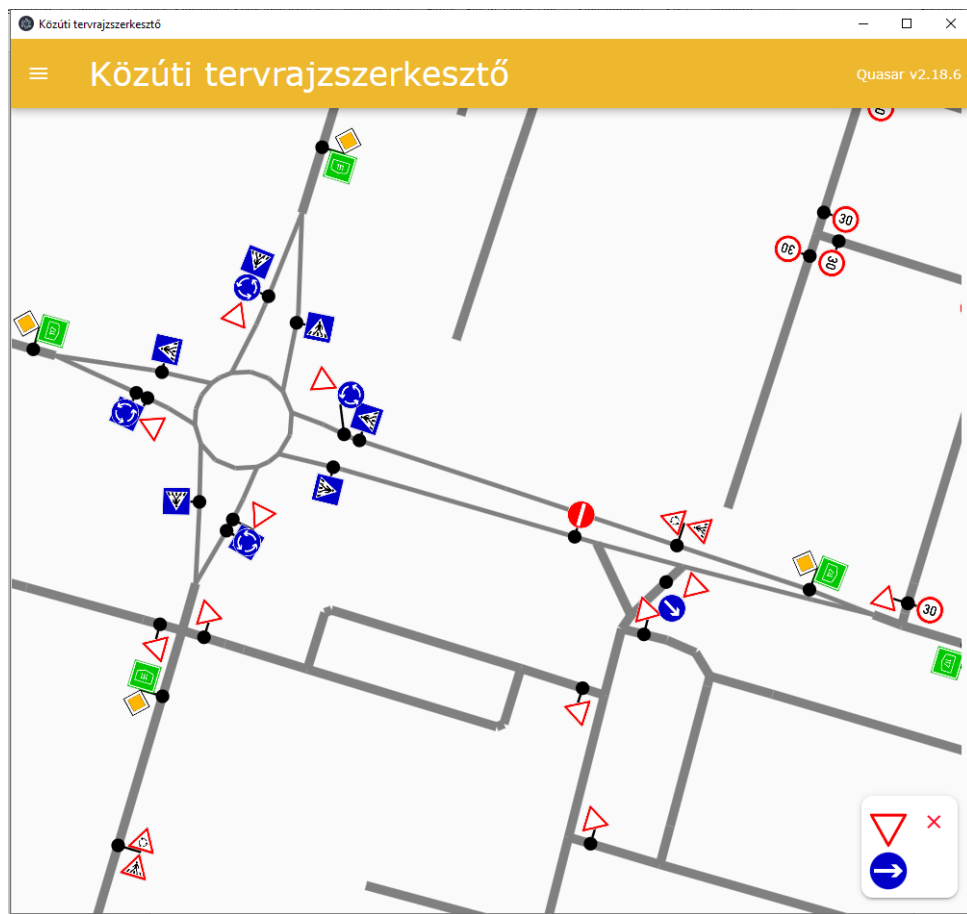
- Kiválasztás a térképen (2.3.2.3. fejezet)
- Mentett tervrajz betöltése

Amennyiben nincs tárolt tervrajz, a Tervrajz lap tájékoztatja erről a felhasználót, és két gombot jelenít meg (8. ábra), amivel át lehet menni a Térkép lapra egy terület kiválasztásához, vagy be lehet tölteni egy elmentett tervrajzot.



8. ábra: a Tervrajz lap megjelenése, amikor nincs az alkalmazásban tárolt tervrajz

Ha a felhasználó kiválasztott egy területet, amit a szerver sikeresen tervrajzzá alakított, vagy betöltött egy tervrajzt, a lap frissül, és megjelenik a tervrajz.



9. ábra: a Tervrajz lap megjelenít egy tervrajzt

2.3.3.1. A tervrajz megjelenítése

Ha az alkalmazás tárol egy tervrajzt, az megjelenik a Tervrajz lapon (9. ábra). Ha térképen kiválasztott terület alapján történik a tervrajz elkészítése, egy valódi térképhez hasonlóan a tervrajzon felfelé található észak. Az utakat változó vastagságú szürke vonalak jelölik, vastagságuk a sávok számával arányosan nő. A jelzőtáblák és jelzőlámpák helyét egy, az útra rögzített pont jelöli, amik össze vannak kötve az adott helyen található jelzőtábla-kompozíciót alkotó jelzőtáblák és/vagy jelzőlámpák jelképeivel, fentről lefelé (a jelzőlámpák reprezentációját az 1.1. fejezetben olvasható definíciója részletezi).

2.3.3.2. Navigáció a tervrajzon

A tervrajz nézete a bal egérgomb nyomva tartása közben az egér mozgásával húzható odébb. A görgővel lehet nagyítani és kicsinyíteni. A tervrajzon megjelenő táblakompozíciók

ábrái a bal egérgomb lenyomásával húzhatók. Ez csupán vizuális, a tervrajz szerkezetét nem változtatja, hiszen az úton lévő rögzített fekete pont jelzi a kompozíció tényleges helyét a tervrajzon.

2.3.3.3. Jelzőtábla-kompozíciók és jelzőtáblák megtekintése és szerkesztése

Bármelyik táblakompozícióra duplán kattintva megjelenik a táblakompozíció menüje (10. ábra), ahol bal oldalt nagy méretben láthatjuk a kompozíciót. A jelenleg kiválasztott táblát a jelképe körül lévő sötét keret jelzi; a kiválasztott táblát a jobb oldali vezérlőpanellel tudja a felhasználó kezelni, illetve itt látja a nevét. A 10. ábra menüjének kiválasztott táblája a legfelső „Sebességkorlátozás” tábla.



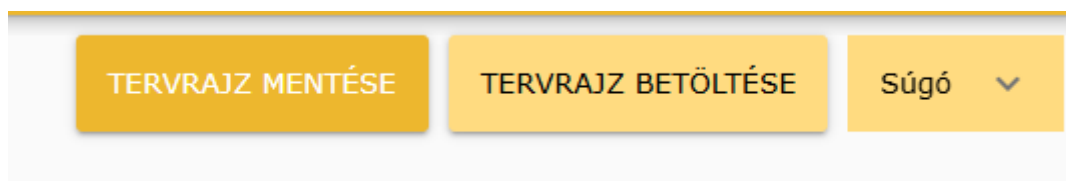
10. ábra: Táblakompozíció menüje

A felhasználó a jobb fenti gombsorból törölni tudja a kijelölt táblát, illetve hozzá tud adni a jelenleg kiválasztott tábla fölé vagy alá egy új darabot. Alatta található a kijelölt tábla neve, egy gomb a tábla exportálására, valamint a szerkesztő, ahol a tábla típusát, illetve testre szabható tábla esetén annak beállításait lehet kalibrálni. A beállítások az „Alkalmaz” gombra

kattintva lesznek érvényesek. Ez a rész a Táblaszerkesztő vezérlőpanelje a táblakompozíció-menübe ágyazva, további részletek a Táblaszerkesztő lap vezérlőpaneljéről szóló 2.3.4. fejezetben vannak.

A táblakompozíció menüje a bal alsó sarokban található „OK” gombbal vagy a felugró ablakon kívülre kattintással zárható. Ha a menü bezártakor a táblakompozíciónak egy tagja sincs, az törlődik a tervrajzról.

2.3.3.4. Mentés és betöltés



11. ábra: a Tervrajz lap jobb felső vezérlői

A Tervrajz lapon, amennyiben az alkalmazás épp egy tervrajzot mutat, a jobb felső sarokban található a gombok a jelenlegi tervrajz mentésére és betöltésére (11. ábra). A felhasználó választja ki a fájlböngészőben a mentés helyét és a fájl nevét, a tervrajz mentése JSON formátumban történik. Hasonló módon a betöltéshez a felhasználónak ki kell választania a fájlböngészőben a mentett, JSON formátumú, érvényes tervrajzot tartalmazó fájlt. Betöltést követően, ha sikeres, a Tervrajz lapon a betöltött tervrajz jelenik meg. Ha a betöltés sikertelen, megjelenik egy felugró ablak, ami tájékoztatja a felhasználót, hogy nem egy érvényes tervrajzot tartalmazó JSON-fájlt választott (12. ábra).



12. ábra: a tájékoztató felugró ablak érvénytelen formátumú JSON-fájl betöltésekor

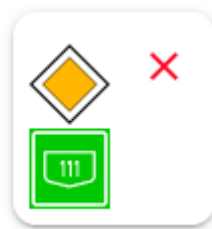
2.3.3.5. Súgó

Szintén a 11. ábra vezérlői közt található az egy kattintással megjeleníthető súgó, ami a tervrajz módosítását érintő parancsokat írja le röviden.

2.3.3.6. Új kompozíció hozzáadása és a vágólap

Az út bármelyik pontjára dupla kattintással lehet új táblakompozíciót hozzáadni. Ennek tartalma attól függ, hogy van-e valami a vágólapon: ha igen, akkor az lesz (így lehet könnyen másolni egy kompozíciót), ha nem, akkor az új kompozíció egy „Elsőbbségadás kötelező” táblából áll.

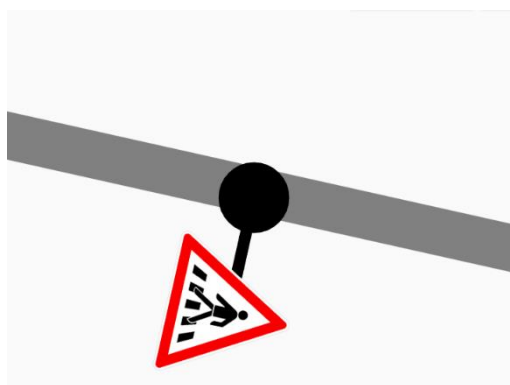
Egy kattintással valamelyik kompozíció grafikájára a felhasználó a vágólapra teszi azt. Ekkor a vágólap a lap jobb alsó sarkában megjelenik (13. ábra). Az „X” gombra kattintva a vágólap törölhető.



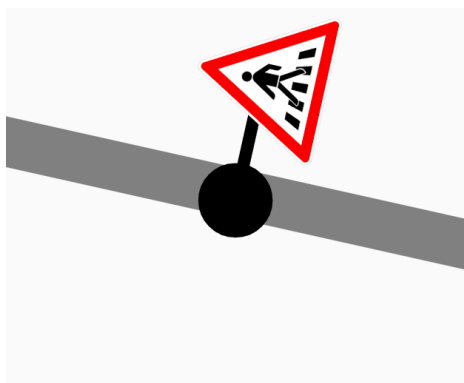
13. ábra: példa vágólapra a Tervrajz lap jobb alsó sarkában

2.3.3.7. Kompozíció megfordítása az út mentén

Főleg új kompozíció hozzáadása esetén merülhet fel, hogy a tábla iránya az út mentén háttal van annak, amit a felhasználó szeretne. Egy kompozíció megfordításához az út mentén az úton lévő pozíciót jelölő fekete pontra kell duplán kattintani. Ilyenkor a tervrajz grafikáján is az új irány szerinti jobb oldalra kerül a kompozíció (14. ábra és 15. ábra).



14. ábra: egy kompozíció megfordítás előtt



15. ábra: egy kompozíció, miután a fekete pontra dupla kattintással meg lett fordítva az út mentén

2.3.4. Táblaszerkesztő

A harmadik lap egy egyszerű táblaszerkesztő, amiben az elérhető táblák listájából tudunk válogatni, az egyedi adatokat tartalmazó táblákat testre szabni, valamint exportálni azokat SVG formátumban.

2.3.4.1. Az SVG grafikai formátum

Az SVG vektorgrafikai formátumot gyakran használják egyszerű grafikákhoz, így jelzőtáblák jelképeinek megjelenítéséhez ez egy kézenfekvő választás. A pixelekből álló raszteres képekkel szemben a vonalak bármilyen nagyítás következtében megőrzik a minőségüket, mivel a képet összetevő geometriai elemek (pl. görbék) egy koordinátarendszerben tárolódnak el, és nem látszódnak egyesével képpontok, még akkor sem, ha nagy méretben szeretnénk kinyomtatni a vektorgrafikát, például egy 60*60 centiméteres fényvisszavető fóliára egy jelzőtáblához.

2.3.4.2. Nézet



16. ábra: A Táblaszerkesztő lap

A Táblaszerkesztő lapon (16. ábra) a vezérlőpaneltől balra található az éppen szerkesztett jelzőtábla nézete. Fontos megjegyezni, hogy a nézet csak a változások alkalmazásakor változik.

2.3.4.3. Szerkesztő vezérlőpanel

A lap jobb oldalán a tábla kiválasztására, szerkesztésére és exportálására szolgáló vezérlőpanel található.

2.3.4.4. Exportálás

A tábla neve alatt jobbra található „Exportálás” címkéjű gombra kattintva tudja a felhasználó a nézeten is látható táblát elmenteni tetszőleges névvel egy tetszőleges helyre.

2.3.4.5. Szerkesztés

A „Szerkesztés” részben lehet a tábla adatait módosítani. A módosítások az „Alkalmaz” gomb lenyomásával lesznek csak láthatók a nézetben és az exportált vektorgrafikában.

A szerkesztett táblának a típusát ki lehet választani a lenyitható listából. A legtöbb típus nem szerkeszthető, azonban néhányat testre lehet szabni.

2.3.4.5.1 Sebességhatár

A „Sebességhatár” jelzőtábla sebességhatáráként megadható egy legfeljebb 3 számjegyből álló egész szám. A beírt érték megjelenik a táblán a szabványos számokkal.

2.3.4.5.2 Össztömeg-korlátozás

Az „Össztömeg-korlátozás” jelzőtábla sebességhatáráként megadható egy legfeljebb 4 karakterből álló egész vagy vesszővel elválasztott törtszám. A beírt érték megjelenik a táblán a szabványos számokkal és a tonna mértékegységet jelölő „t” betűvel. A vessző utáni szám(ok) és a „t” betű a szabvány szerint 0,7-szeres méretben, kicsinyítve jelennek meg.

2.3.4.5.3 Sávbesorolás rendjét jelző tábla

Táblaszerkesztő

Tábla: Sávbesorolás rendjét jelző tábla	
EXPORTÁLÁS	
Szerkesztés	
Táblatípus	
Sávbesorolás rendjét jelző tábla	
Megengedett irányok Balra, Egyenesen	Sáv típusa Általános
Megengedett irányok Egyenesen	Sáv típusa Kerékpár
Megengedett irányok Jobbra	Sáv típusa Általános
ÚJ SÁV	
ALKALMAZ	

17. ábra: Sávbesorolás rendjét jelző tábla nézete és beállításai a szerkesztőben

A sávbesorolás rendjét jelző táblák sávelrendezését testre lehet szabni. A szerkesztőben egy lista látható sávokkal (17. ábra). A lista elemei fentről lefelé a táblán lévő sávokat reprezentálják balról jobbra.

Az egyes sávoknak három vezérlőjük van:

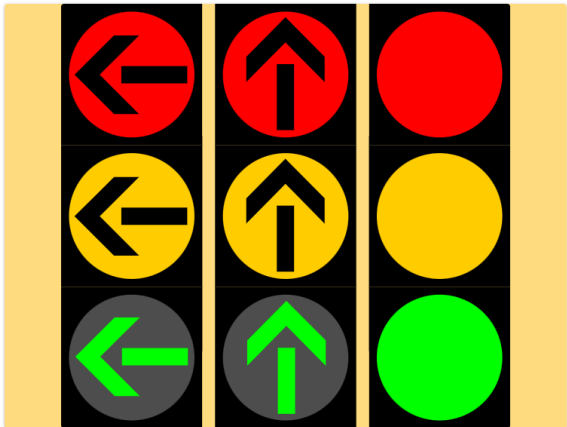
- **Megengedett irányok:** a sáv iránya vagy irányai. A felsorolásból egy vagy több irány választható, ez alapján jelennek meg a sávhoz tartozó nyílfejek.

- **Sáv típusa:** kiválasztható a sáv típusa. A beállítás alapesetben „Általános”, ilyenkor a sávon nem jelenik meg betét. A többi esetben az adott sáv típus jelölésére használt betét jelenik meg (pl. kerékpársáv esetén egy kerékpár ábrája). A táblán megjelenítendő elválasztó vonalak (kerékpársáv esetén sárga folytonos, buszsáv esetén fehér szaggatott) automatikusan elhelyezésre kerülnek a táblán a sávok típusa és a sorban lévő elhelyezkedésük alapján.
- **Törlés:** a piros kör alakú, kukával címkézett gombra kattintva a felhasználó törölheti az adott sávot.

Az alsó, „Új sáv” gombra kattintva lehet új sávot hozzáadni a lista végére.

2.3.4.5.4 Fényjelző készülék(ek)

Táblaszerkesztő



Tábla: Fényjelző készülék(ek)
EXPORTÁLÁS

Szerkesztés

Táblatípus

Fényjelző készülék(ek) ▼

Zöld jelzés típusa
Balra

Zöld jelzés típusa
Egyenes

Zöld jelzés típusa
Telezöld

❌

❌

❌

ÚJ JELZŐLÁMPA

ALKALMAZ

18. ábra: jelzőlámpák csoportja a szerkesztő beállításaival

Ez a „jelzőtábla” jelzőlámpát vagy jelzőlámpák egy csoportját reprezentálja (a definíció az 1.1. fejezetben olvasható).

A szerkesztő egy listát tartalmaz, ami a sávbesorolás rendjét jelző táblák sávjaihoz hasonlóan a jelzőlámpákat listázza a csoportban balról jobbra. Itt a zöld jelzés típusát lehet kiválasztani az egyes jelzéseknek, ami a fényjelzőkben található képet határozza meg. Ez lehet

- telezöld,
- nyíl alakú (számos különböző lehetőséggel),
- kerékpár (kerékpáros jelzőlámpa esetén).

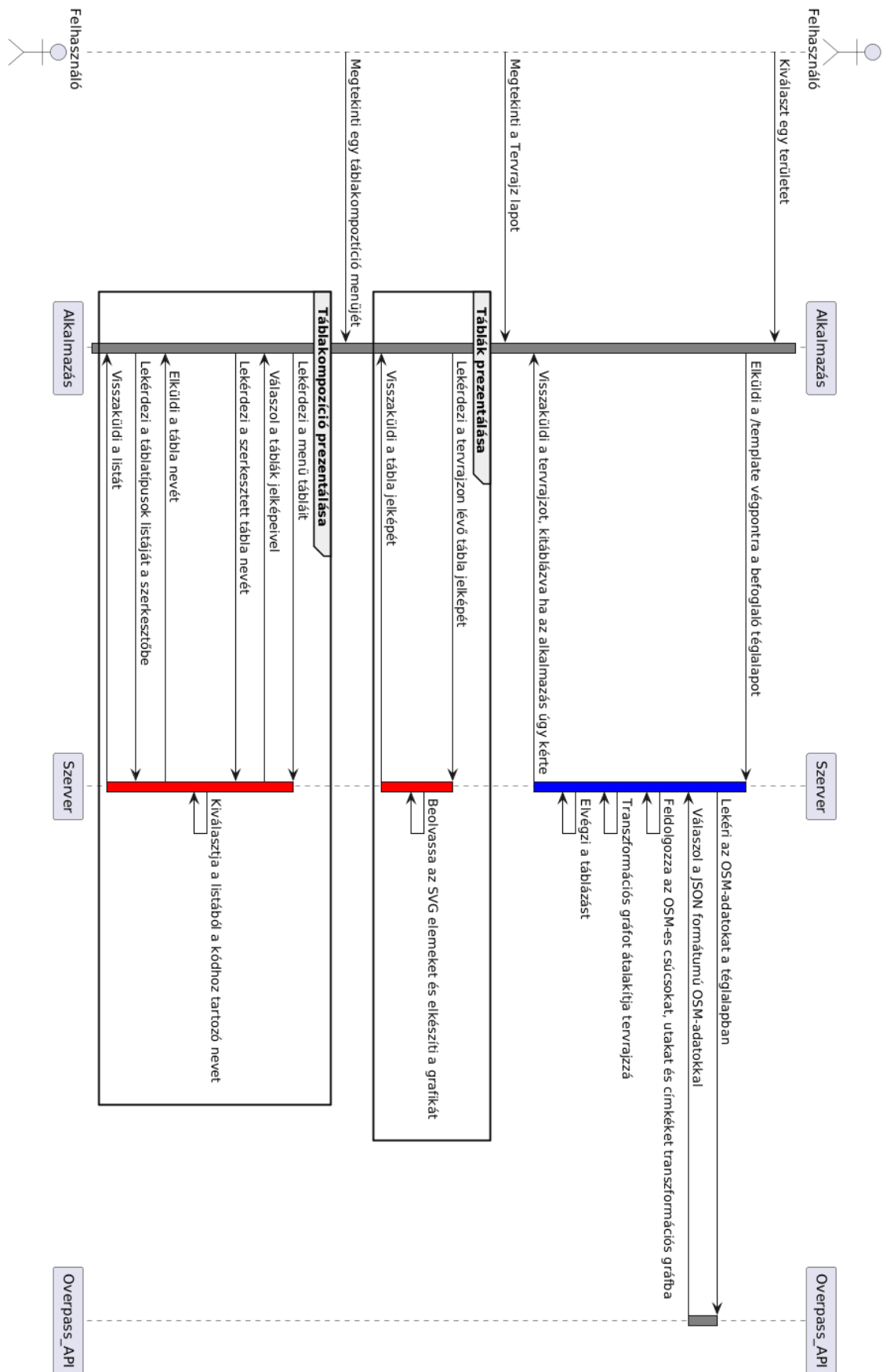
Az egyes lámpák típusai mellett a piros, kukát ábrázoló gombra kattintva a felhasználó törölheti azt, a lista alján lévő „Új jelzőlámpa” gombra kattintva pedig új lámpát adhat a lista végére (tehát a csoport jobb szélére).

3. Fejlesztői dokumentáció

3.1. Áttekintés

A szoftver két, külön futtatható komponensből áll: egy állapottal rendelkező asztali alkalmazásból és egy állapot nélküli szerverből.

A szerver egy REST API-ként működik: vannak végpontjai, amiken keresztül az alkalmazás kommunikál a szerverrel. Az alkalmazás csak lekérdezésre használja a szervert; a szerver nem tárol semmilyen adatot, és nem rendelkezik perzisztenciával, a HTTP-válasz után nem marad „nyoma” a lekérdezésnek a naplón kívül.



19. ábra: Az alkalmazás és a szerver kapcsolatának szekvenciadiagramja. A szerver színkódolásában a kék a tervrajzkomponenst, a piros a prezentációs komponenst jelöli.

Az alkalmazás a szervertől elkéri a térképen a felhasználó által kiválasztott terület tervrajzát, a hívás paramétereitől függően táblázva, és fogadja, majd eltárolja azt. A szerver az API-hívás HTTP-kéréséből a feldolgozást követően semmilyen adatot nem tárol, beleértve a válaszként küldött tervrajzot. Innentől a tervrajz az alkalmazásban tárolódik a futásának ideje alatt, annak szerkesztése és mentése mind az alkalmazásban történik. Ugyanúgy egy mentett tervrajz betöltését is egyedül az alkalmazás végzi.

Azonban az alkalmazás szintén a szervertől kéri el a jelzőtáblák prezentálásához szükséges adatokat, tehát a táblák nevét és jelképét. A táblák modelljét elküldi a megfelelő végpontra (`signs/[kód]/image` a tábla modelljével a lekérdezésben vagy `signs/[kód]/name`), majd a szerver válaszol a megfelelő névvel vagy jelképpel.

3.2. Szerver

3.2.1. Definíciók

Itt találhatók a szerver leírásának általános definíciói. A szerver specifikus részeire vonatkozó definíciók leírásai a vonatkozó részekben vannak.

- **Alkalmazás hatókörű bean:** olyan beanek, melyek a `jakarta.enterprise.context.ApplicationScoped` annotációval vannak ellátva. Bár az alkalmazás hatókörű beanekből egyszerre csak egy példány létezik, az bármikor megsemmisíthető és újra létrehozható; mivel a Quarkus a bean befecskendező objektum és a bean példánya között egy proxyt alkalmaz, a befecskendezett példány nem törődik a tényleges bean állapotával. A Quarkus általánosságban ezt a beantípust ajánlja a `@Singleton` annotációval megjelölt egyke helyett, [5] ami nem alkalmaz proxyt, így ebben a projektben is alkalmazás hatókörűek a beanek.
- **OSM-pont:** az OpenStreetMap egy `node` típusú eleme.
- **OSM-pont vagy OSM-út címkéi:** az OSM-pont vagy OSM-út JSON-objektum reprezentációjának `tags` kulcsának értéke.
- **OSM-út:** az OpenStreetMap egy `way` típusú eleme.
- **Prezentációs komponens:** a jelzőtáblák megnevezéséért és jelképeik elkészítéséért felelős része a szervernek, ami a `presentation` csomagban található.

- **Szolgáltatásoosztály:** a `service` nevű csomagokban található, „Service”-re végződő nevű osztályok.
- **Tervrajzkomponens:** a tervrajz elkészítéséért felelős része a szervernek, ami a `roadtemplate` csomagban található.

3.2.2. A szerver áttekintése

A szerver kettő fő komponensből áll: a tervrajzkomponensből és a prezentációs komponensből, melyek elkülönülnek

- az architektúra tekintetében: a projektben⁶ a forráskód gyökérkönyvtárán belül két külön csomagba kerülnek, a tervrajzkomponens a `roadtemplate`-be, a prezentációs komponens a `presentation`-be (3.2.4. fejezet),
- végpontok tekintetében: mindkét komponens rendelkezik saját végpontokkal, az érkező kérések feldolgozása nem vesz igénybe osztályokat a másik komponensből,
- és funkció tekintetében: a tervrajzkomponens tervrajzok elkészítésére és táblázására szolgál, míg a prezentációs komponens jelzőtáblák (amik lehetnek akár a tervrajzkomponens által a tervrajzra elhelyezett táblák modelljei) megnevezésére, illetve jelképének elkészítésére.

A tervrajzkomponens egyetlen végpontja fogad egy HTTP-kérést, mely egy térképes koordinátákkal megadott befogó téglalapot és egy, a táblázás szükségességét megadó igaz-hamis értéket tartalmaz, majd lekéri az Overpass API-tól az OpenStreetMap befogó téglalapban található út- és csúcsadatait, és ezeket a JSON formátumú adatokat átalakítja egy él- és csúcscímkezett transzformációs gráffá, mely végrehajt néhány, az OpenStreetMap jelöléseinek hiányosságait és következtelenségeit enyhítő transzformációt, végül ebből a gráfból létrehozza a tervrajzot, melyen a komponens részét képező táblázó program elvégzi a jelzőtáblák elhelyezését, ha a kérésben meg volt adva. A végpont visszaadja az elkészült, adatátviteli objektumok által meghatározott JSON-struktúrává alakított tervrajzot.

⁶ A szerverről szóló fejezetben (3.2) a „projekt” a szerver projektjére és az annak könyvtárában található összes fájlra utal, leggyakrabban a forrásfájlokra fogok hivatkozni így.

A prezentációs komponens végpontjain az alkalmazás képes

- az összes, szerver által ismert jelzőtábla kódját és nevét elkérni,
- kód alapján egy jelzőtábla nevét lekérni,
- kód és testre szabható táblák esetén a táblamodell adatai alapján lekérdezni a tábla jelképét.

Egyszerű táblák esetén a jelkép visszaadása egy SVG-fájl visszaadásából áll a HTTP-válaszban, testre szabható tábláknál azonban a szervernek meg kell előtte azt szerkesztenie, és a szerkesztéshez szükséges (szintén SVG típusú) vektorgrafikus elemeket importálja.

A két fő komponens részletesebb leírása lentebb található a 3.2.5 és 3.2.6 fejezetekben.

3.2.2.1. Alkalmazott technológiák

A szerver kódja Java nyelven íródott, a fordító a Java SE platform 21-es verzióját használja, így a kód is a Java 21-gyel kompatibilis.

A szerver a Quarkus keretrendszert használja. A Quarkus lehetővé teszi a projektben a függősfecskendezést ⁷bean komponensek létrehozásával a Jakarta Enterprise Edition (EE) specifikációját használva. [5] A kódban a szolgáltatásosztályok nagy része beanként működik, melyet más beanekbe lehet injektálni, továbbá így lehetséges az `application.properties` erőforrásfájlból konfigurálni értékeket (például jelzőtáblák elhelyezését meghatározó távolságok, vektorgrafikai elemek helye), melyet be lehet fecskendezni induláskor a megfelelő osztályokba.

A Spring Boot a Quarkushoz hasonlóan szintén rendelkezik függősfecskendezéssel, és szintén egy gyakran használt keretrendszer Java-projektekhez, én mégis az utóbbit választottam. Noha igaz, hogy a Spring Boot (és a Spring keretrendszer, aminek a részét képi) régebbi, így több forrás van rá az interneten és nagyon sok könyvtár található rá, [6] a Quarkus 2019-es indulása óta nagyon népszerű lett, így bőven találni internetes segédanyagokat, és kompatibilis könyvtárak is szép számmal vannak hozzá. [7] A fejlesztést is sok módon

⁷ A kifejezést „bean”-nek hagytam magyarul is bevett meghonosult kifejezés híján.

megkönnyíti a Quarkus: a szerver fejlesztői módban elindítva elérhető egy Dev UI nevű felhasználói interfész, ahol számos módon lehet interakcióba lépni az alkalmazással, például az OpenAPI kiegészítő telepítése után a Swagger UI-jal teszteltem a REST végpontokat. A Quarkus képes live codingra is: az alkalmazás forráskódjának megváltoztatása után szinte rögtön újra lefordul a program, így könnyen lehet fejlesztés közben ellenőrizni a működését. [8]

Naplózáshoz a projekt az SLF4J-t használja. Ez egy interfész, ami miatt nem függ a kódban a naplózás az azt megvalósító keretrendszertől. [9]

A jelzőtáblák ábrázolására a szoftver SVG-dokumentumokat használ. Az SVG-dokumentumokban található formák és egyéb elemek vagy elemcsoportok könnyen mozgathatók és egymásba ágyazhatók, így könnyen lehet elkészíteni a jelzőtáblák egyszerű formákból álló jelképeit. Ráadásul mivel a kép elemei vektorokból állnak, és nem képpontokból (mint a raszteres képek esetén, pl. PNG vagy JPG formátum), az elkészült grafika bármennyire felnagyítható anélkül, hogy szemmel tisztán láthatók legyenek az egyes képpontok. Az SVG-dokumentumok kezeléséhez az Apache Batik Quarkusra adaptált változatát használtam. [10]

3.2.3. Végpontok

A szerver egy REST-elveket követni törekvő API-ként működik, így végpontokon keresztül teszi lehetővé az interakciókat. Ezen végpontok valamelyikére érkeznek az alkalmazás egyes hívásai is, de felhasználhatók további alkalmazások fejlesztésére is, melyek a szerver szolgáltatásait használják.

3.2.3.1. */template*

Metódus: GET

Lekérdezés paraméterei:

- `hasSignage` (true/false): a visszaadott tervrajz táblázva legyen-e
- `latStart` (valós szám, [-90;90]): a befogó téglalap déli szélességi foka
- `lonStart` (valós szám, [-90;90]): a befogó téglalap nyugati hosszúsági foka
- `latEnd` (valós szám, [-90;90]): a befogó téglalap északi szélességi foka
- `lonEnd` (valós szám, [-90;90]): a befogó téglalap keleti hosszúsági foka

Válasz formátuma: application/json

Ezen a végponton lehet elkészíttetni a szerverrel egy földrajzi befogó téglalap OSM-adatai alapján a terület tervrajzát, amin a `hasSignage` paraméter értékétől függően a táblázó program el is helyezheti a jelzőtáblákat.

Válaszok:

- Sikeres elkészítés esetén 200-as státuszkód mellett a HTTP-válasz törzsében a szerver elküldi a tervrajz JSON-modelljét.
- Ha a befogó téglalap paraméterei nem helyesek (nem a [-90;90] intervallumban vannak, vagy negatív a területe), a szerver 400-as státuszkóddal válaszol, és JSON-modell helyett a törzsben a hibás lekérdezés okát küldi el:
 - „*Latitude and longitude values must be between -90 and 90*”: a téglalap szélességi és hosszúsági fokai nincsenek az értelmezhető tartományban
 - „*Start latitude/longitude must not be greater than end*”: a téglalap déli szélessége magasabban van az északínál, vagy a nyugati hosszúsága keletebbre a keletinél, így a befogó téglalap nem értelmezhető.
- Ha a lekérdezési paraméterek helyesek, és már a feldolgozása közben történik probléma, a szerver 500-as kóddal válaszol, a törzsben pedig elküldi a hibaüzenetet. Az 500-as hiba oka gyakran az Overpass API szerverének elérhetetlensége, ami ilyenkor 504-es státuszkóddal válaszol magának a szervernek (a szerver azonban nem továbbítja ugyanezt a kódot a válaszban, mert a HTTP-kérés küldője és a szerver közt rendben van a kapcsolat, ez belső hibának tekinthető). A hibát okozó kivétel minden alkalommal naplózásra kerül a szerveren. A lehetséges üzenetek:
 - „*Could not resolve hostname of Overpass API*”: a szerver nem tudja feloldani az `application.properties` fájlban megadott állomásnevet. Okozhatja a problémát az, hogy a szervernek nincs internetkapcsolata, de az is, hogy helytelenül van megadva a `quarkus.rest-client.overpass-api.url` értéke (az `application.properties` fájlban).
 - „*An exception occurred while querying data via Overpass API*”: az Overpass API hívásakor kivétel dobódott. Ez leggyakrabban azért lehet, mert az API 504-es

státuszkóddal válaszol túlterheltség miatt (ami egy `jakarta.ws.rs.WebApplicationException` típusú kivételt vált ki).

- „*The Overpass query result contains no ways*”: Az Overpass API válaszában lévő adathalmazban nem található út, azaz nincs a válaszban olyan JSON-objektum, melynek `type` tagjának értéke `way`.
- „*An internal error occurred while creating template*”: A feldolgozás közben dobódott egy kivétel, ami a szerver kódjából származik.

3.2.3.2. `/signs/[kód]/name`

Metódus: GET

Útvonal paramétere:

- `[kód]`: a jelzőtábla kódja (tábla JSON-objektumának `code` tagja)

Válasz formátuma: `application/json`

A végpontra érkező kérés válasza egy JSON-objektumot tartalmaz, melynek egyetlen tagjának (`name`) értéke az útvonalban szereplő táblakódhoz tartozó név:

```
{name : [tábla neve]}
```

Válaszok:

- Ha a szerver sikeresen megtalálja a nevet, 200-as státuszkóddal válaszol, a törzsben a nevet tartalmazó objektummal.
- Ha az adott kódhoz nem talál nevet, 404-es státuszkód lesz a válaszban.
- Ha valami egyéb hiba történik, 500-as státuszkóddal tér vissza a szerver. A hibát okozó kivétel minden alkalommal naplózásra kerül a szerveren.

3.2.3.3. `/signs`

Metódus: GET

Válasz formátuma: `application/json`

A végpontra érkező kérés válasza egy lista JSON formátumú stringlista a táblakódokkal, melyek szerepelnek a táblakódok és -nevek listájában a `resources` mappa `sign_names.csv` fájljában.

Válaszok:

- Ha a szerver megtalálja és beolvassa a listából a kódokat, 200-as státuszkóddal tér vissza, a válasz törzsében a kódok listájával.
- Ha problémába ütközik, 500-as a válasz státuszkódja.

3.2.3.4. */signs/[kód]/image (GET)*

Metódus: GET

Válasz formátuma: image/svg+xml

Útvonal paramétere:

- [kód]: a jelzőtábla kódja (tábla JSON-objektumának code tagja)

A végpontra érkező kérés válasza a kódhoz tartozó egyszerű jelzőtábla jelképe. Ha nem egyszerű, hanem testre szabható jelzőtábla kódja van az útvonalban, akkor a jelképet nem küldi vissza, hiszen az elkészítéséhez további adatokra van szükség (pl. címke, vagy sávelrendezés sávbesorolás rendjét jelző tábla esetén). Testre szabható táblák esetén ugyanerre az útvonalra POST metódussal kell kérést küldeni, a törzsben a táblához tartozó adatokkal, ugyanis konvenció alapján GET metódusú kérés nem tartalmazhat törzset. [11]

Az alábbi kódú jelzőtáblák esetén tér vissza 400-as státuszkóddal a szerver:

- C-023 (Össztömeg-korlátozás)
- C-033 (Sebességkorlátozás)
- G-152 (Számozott út azonosító táblája)
- E-SAVBESOR (Sávbesorolás rendjét jelző tábla)
- JL (Fényjelző készülék(ek))

Válaszok:

- Ha sikerül a táblához tartozó jelkép megszerzése, a szerver 200-as státuszkód mellett a válasz törzsében küldi az SVG formátumú vektorgrafikát.
- Ha a szerver nem találja a jelképet, bár 200-as státuszkód mellett visszaad egy SVG-t, az egy szürke kört ábrázoló helytartó grafika. Ez azért van, hogy ha a szerver nem számol az elérhető táblák kódjainak listájával, a tábla SVG-fájlját eltávolítják a szerver

erőforrásai közül, vagy nagy mennyiségű kérés elenyésző része tartalmaz rossz táblakódot (pl. nagy tervrajz tábláinak megjelenítésekor), akkor ne kelljen a kérés küldőjének a speciális esetekkel foglalkoznia.

- Ha a kérés teljesítése közben kivétel dobódik, ami nem a tábla fájljának nem meglétéből ered, 500-as státuszkód lesz a válaszban.

3.2.3.5. */signs/[kód]/image (POST)*

Metódus: POST

Válasz formátuma: `image/svg+xml`

Útvonal paramétere:

- [kód]: a jelzőtábla kódja (tábla JSON-objektumának `code` tagja)

A végpontra érkező kérés válasza a kódhoz tartozó jelzőtábla jelképe. A metódus törzsének tartalmaznia kell a jelzőtábla adatait. Egyszerű tábla esetén a törzs lehet üres vagy egy üres JSON-objektum (`{}`). A testre szabható táblák esetén az alábbi formátumot várja el a szerver:

- C-033 (Sebességkorlátozás):
 - Formátum: `{ label: string }`
 - A címke (`label`) string formátumú, de csak egész számot tartalmazhat, különben a szerver nem tudja elkészíteni a jelképet. A számjegyek számára nincs korlátozás (alsó sem), de ahhoz, hogy a címke a jelzőtábla piros keretén belül maradjon, ajánlott legfeljebb hármat használni.
- C-023 (Össztömeg-korlátozás):
 - Formátum: `{ label: string }`
 - A címke (`label`) string formátumú, de csak számokat, vesszőt, illetve „t” betűt tartalmazhat, különben a szerver nem tudja elkészíteni a jelképet. A karakterek számára nincs korlátozás (alsó sem), de ahhoz, hogy a címke a jelzőtábla piros keretén belül maradjon, ajánlott legfeljebb négyet használni egész számú korlátozás esetén, vagy ötöt, ha törtszám a korlátozás. A „t” mértékegység-jelölés nem feltétel, de ajánlott a címke végére tábla jelentése miatt.
- G-152 (Számozott út azonosító táblája)
 - `{ label: string }`

- A címke (`label`) string formátumú, de csak egész számot, illetve „M” betűt tartalmazhat, különben a szerver nem tudja elkészíteni a jelképet. A karakterek számára nincs korlátozás (alsó sem), de ahhoz, hogy a címke a számpajzs keretén belül maradjon, ajánlott legfeljebb hármat használni.
- E-SAVBESOR (Sávbesorolás rendjét jelző tábla)
 - Formátum:

```
{
  lanes: [
    directions: [
      <'FORWARD' | 'LEFT' | 'RIGHT' | 'REVERSE'>
    ];
    type: <'GENERAL' | 'BICYCLE' | 'BUS' | 'BUS_BICYCLE'>;
  ];
}
```

- A `<'A' | 'B' | ...>` minta felsorolástípus lehetséges értékeit jelöli, melyek JSON-formátumban stringként vannak tárolva. A `directions` értéke az adott sáv irányainak listája, a `type`-é pedig a sáv jellege (általános sáv, kerékpársáv, buszsáv, vagy buszsáv megengedett kerékpáros közlekedéssel)
- JL (Fényjelző készülék(ek))
 - Formátum:

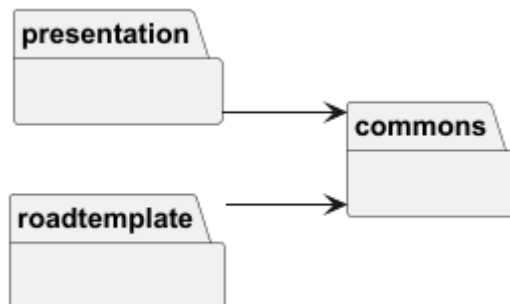
```
{
  signals : [< 'FULL', 'STRAIGHT', 'LEFT', 'RIGHT',
'LEFT_RIGHT',
'STRAIGHT_LEFT', 'STRAIGHT_RIGHT', 'REVERSE',
'BICYCLE'>]
}
```

- A `<'A' | 'B' | ...>` minta felsorolástípus lehetséges értékeit jelöli, melyek JSON-formátumban stringként vannak tárolva. A `signals` értéke a jelzőlámpa fajtáját tartalmazza, amely lehet telizöld (FULL), a fényjelzőben szereplő nyilak iránya (STRAIGHT, LEFT_RIGHT, stb.), vagy kerékpáros jelző (BICYCLE).

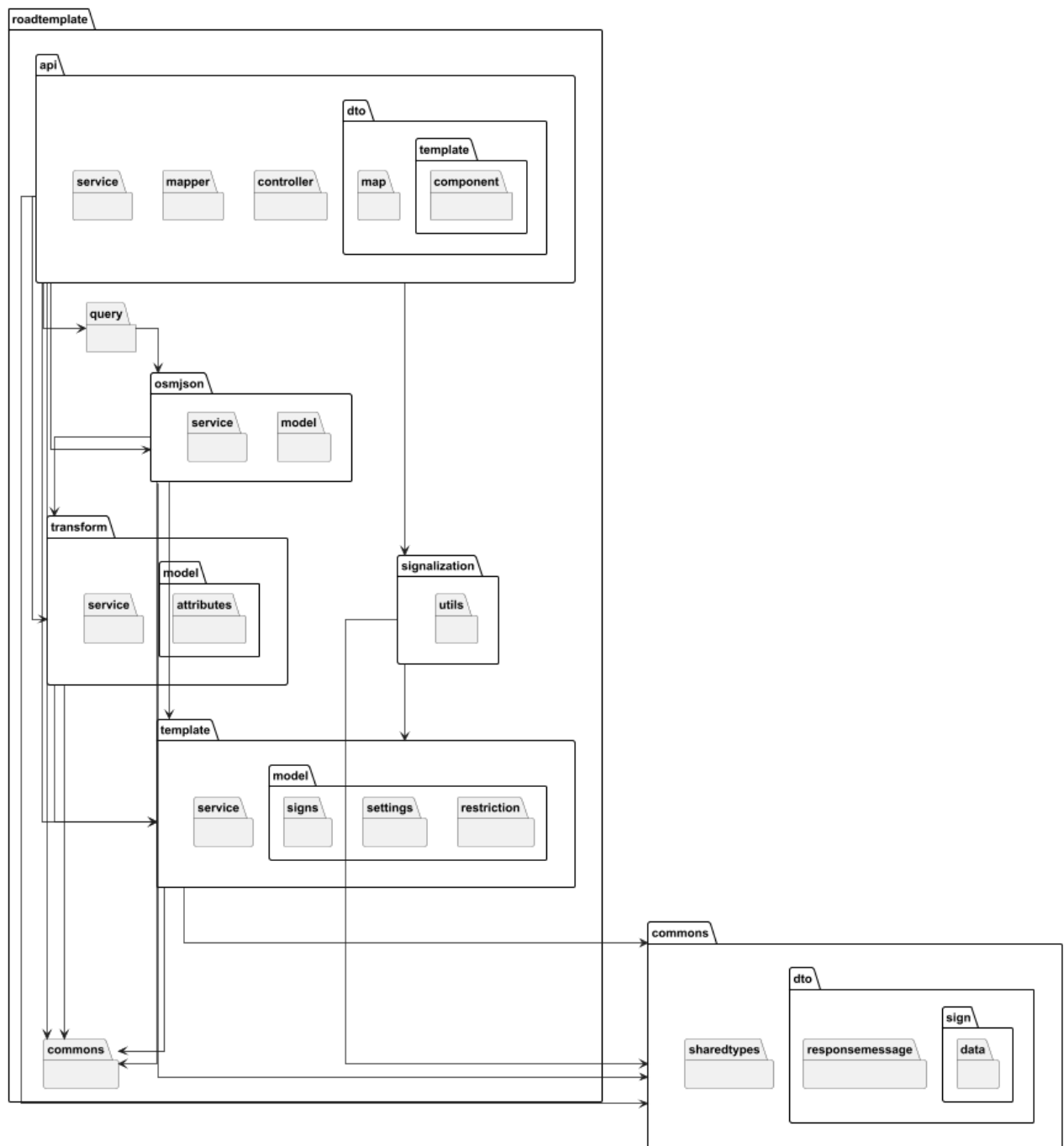
3.2.4. Csomagstruktúra

A csomagstruktúra kialakítása során törekedtem arra, hogy a szerver különböző funkciói minél jobban külön legyenek választva. A szerver logikája számos, egymástól független vagy lazán függő funkcióért felelős: a térkép tervrajzzá alakításának nem igazán van köze a jelzőtáblák jelképeinek elkészítéséhez.

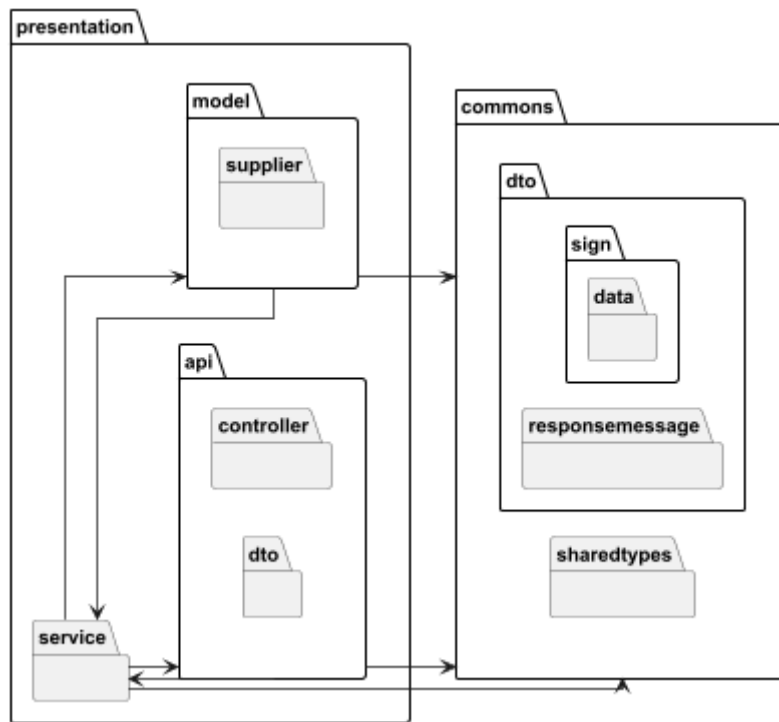
Éppen ezért lett a program „függgőlegesen felszeletelve”, tehát a csomagok főként az alapján vannak létrehozva, hogy a szerver üzleti logikájában milyen szerepet vállalnak a benne lévő osztályok. Így például a modellosztályok, amikből nagyjából 30 darab van a projektben, nincsenek egy csomagba osztva, hanem a szerepük szerint (jelzőtáblák grafikájának reprezentációja, tervrajz modellje stb.) el vannak választva, a tervrajz modellje így a tervrajzért felelős csomagban található, és ugyanígy vannak elrendezve a további kategóriák. Ezért, ha hozzá szeretnénk adni valamilyen funkcionalitást az üzleti logika egy részéhez, akkor kevesebb csomag lehet érintett, mivel leggyakrabban a funkcionalitás kiegészítése az üzleti logika egyik adott szerepéhez köthető, ezért csak azt a „függgőleges szeletet” kell kiegészíteni. [12]



20. ábra: A szerver magas szintű csomagdiagramja



21. ábra: a tervrajzkomponens csomagdiagramja, a függőségek a `roadtemplate` alatti legfelső szinten vannak jelölve



22. ábra: a prezentációs komponens csomagdiagramja, a függőségek a *presentation* csomag alatti közvetlen szinten vannak jelölve

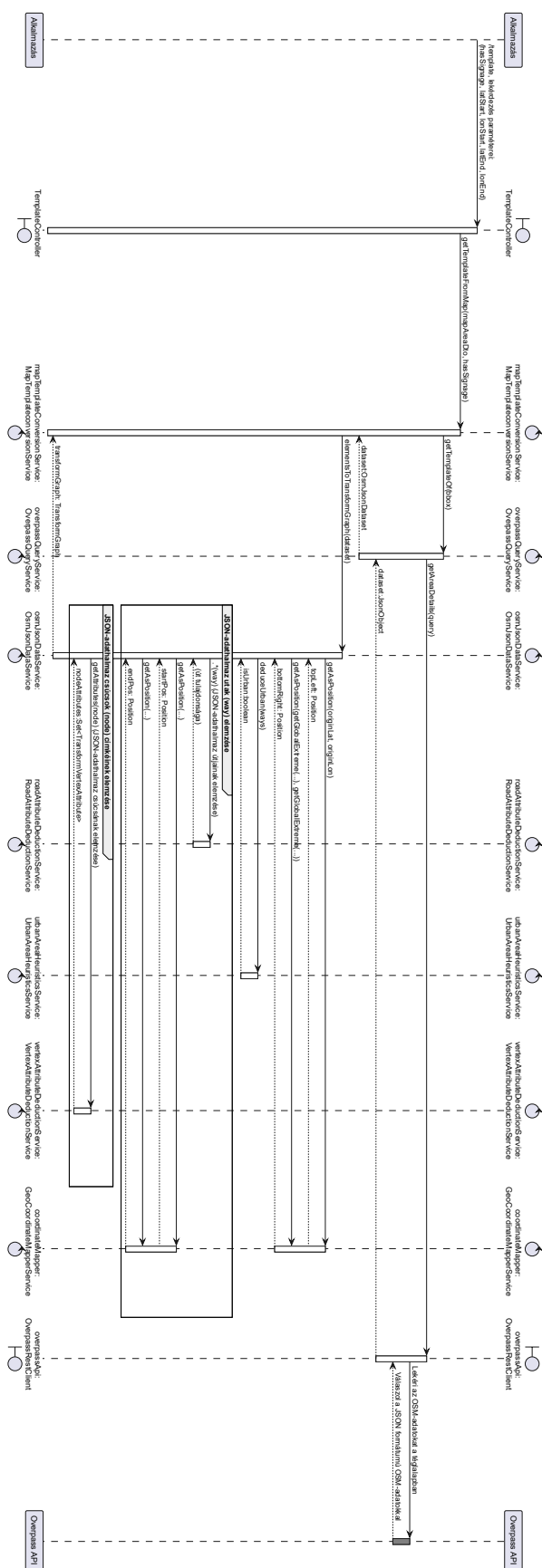
A szerver csomagstruktúrája többretegű, és legfelső szintje (20. ábra) két csomagból áll, melynek minden közös osztálya egy harmadik csomagban (**commons**) található:

- **roadtemplate**: ebben a csomagban található minden, ami valamilyen szinten a tervrajzhoz köthető. Csomagszerkezetét a 21. ábra mutatja. Az alábbi alcsomagokkal rendelkezik:
 - **api**: itt található egy további alcsomagban a térkép befoglaló téglalapjának struktúrája, illetve a tervrajz JSON-formátumát megalkotó adatátviteli objektumok (**dto**), egy külön csomagban a térkép-tervrajz konverziót indító API-végpont (**controller**), melybe egyéb végpontok kerülhetnek későbbi fejlesztés során az osztályaikkal, a modell objektumait adatátviteli objektumokká alakító leképző osztályok csomagja (**mapper**), és a **service** csomagban az a szolgáltatás (**MapTemplateConversionService**), amely a **roadtemplate** többi szolgáltatásának befecskendezésével elvégzi a **controller** csomagban található végponton beérkező hívás feldolgozását az Overpass API meghívásától a JSON-adatok elemzésén és a transzformáción át egészen a tervrajz létrehozásáig, kitáblázásáig (ha a hívásban kérve van) és konvertálásáig adatátviteli objektummá.

- **commons**: itt van a `roadtemplate`-en belül általánosan használt osztályok helye, amely ebben az esetben csak a koordinátarendszerbeli pozíciót reprezentáló `Position` osztály.
- **osmjson**: ebben a csomagban van a **model** alcsomagban az OSM-ből lekérdezéssel kapott út- (`way`) és csúcsadatok (`node`) nyers OSM-adathalmaza (`OsmJsonDataset` osztály), illetve a **service** alcsomagban az OSM-adathalmazt és a térképes koordinátákat feldolgozó, és transzformációs gráffá alakító szolgáltatások.
- **query**: a `query` csomag nem rendelkezik alcsomagokkal, az Overpass API eléréséhez szükséges REST-klienst tartalmazza, és a hívást elvégző osztályt (`OverpassQueryService`), ami a REST-kliens felől érkező adatokat az `osmjson` csomag által feldolgozható struktúrává alakítja.
- **signalization**: ez a csomag tartalmazza a táblázó program osztályait, amelyekkel a tervrajzon képes elhelyezni a jelzőtáblákat a csomagban található `RuleSet` interfész valamely implementációja (a csomagban ez a `RuleSetImpl`). A **utils** alcsomag osztályaiban a táblázáshoz használt szabályrendszerhez használható triggerek (a triggerek fogalma a 3.2.5.6.1. fejezetben található) és pozíciókeresők (3.2.5.6.2. fejezet) található.
- **template**: itt található a tervrajz elemeinek modelljei a **models** alcsomagban (melyben néhány elem további csomagba van ágyazva: a jelzőtáblák modelljei a **sign**, a tervrajz újtaira tehető korlátozások a **restrictions**, a modellek inicializálásához szükséges adatokat egyesítő rekordok pedig a **settings** csomagban kaptak helyet), a modellhez tartozó szolgáltatások helye pedig a **service** (ahol egy szolgáltatásosztály található, a `SignCompositionConfigService`).
- **transform**: ebben a csomagban vannak a térképadatok és a tervrajz közötti átmeneti transzformációs gráf osztályai, a **model** alcsomagban a gráf modellosztályai vannak, ennek az **attributes** alcsomag a gráf elemeinek címkéit gyűjtő osztályokat tárolja; a **service** csomagban található a `TransformGraphProcessorService` osztály, ami feldolgozza a transzformációs gráfot.

- **presentation:** ebben a csomagban található azok a funkciók, amelyek jelzőtáblák tervrajzon található kódolásának valamilyen prezentálásához köthetők: a grafika létrehozásához vagy egy tábla kódjának a nevével történő összekötéséhez. Bár ez a két funkció nincs szoros függőségben egymással, mivel a táblák megnevezését megvalósító rész egy szolgáltatásosztályból és egy végpontból áll (ami egy metódus a `SignController` osztályban), nem tartottam célszerűnek a funkció szerinti szétválasztást. Csomagszerkezetét a 22. ábra mutatja.
 - **api:** ebben a csomagban található a **controller** alcsoomagban a jelzőtáblákkal kapcsolatos végpontokat megvalósító **SignController** osztály, valamint a **dto** alcsoomag, ami az adatátviteli objektumok helye. A prezentációs komponens egyetlen exkluzív DTO-ja (ami nem a **commons** főcsomagban van) a táblák nevének átviteléhez szükséges `SignNameDto`.
 - **model:** itt található azok a jelképek grafikáinak modelljei. A **supplier** alcsoomagban található azok a szolgáltató osztályok, melyek a modellosztály egyes példányait képesek előállítani a szerver fájlrendszerének segítségével (maguk a modellosztályok nem olvasnak be fájlokat).
 - **service:** ebben a csomagban található a prezentációs komponenshez tartozó összes szerepet betöltő szolgáltatásosztály, beleértve a végponton át beérkezett adatátviteli objektumok elemzését, a táblanevek gyűjtését és párosítását táblák kódjához, valamint a jelképek SVG reprezentációinak elkészítését.
- **commons:** ebben a csomagban vannak elhelyezve a mindkét másik csomagban közösen használt osztályok, ami összesen néhány jelzőtáblákkal kapcsolatos adatátviteli objektum (a **dto** csomagban), melyet egyszerre felhasznál a **roadtemplate** csomag a tervrajzon elhelyezett táblák reprezentálására, amikor az API-on keresztül lekéri a tervrajzot az alkalmazás, majd ugyanezeket a táblareprezentációkat küldi vissza a **presentation** csomag végpontjaira, ami a közös struktúrát felhasználva visszaadja a táblák nevét vagy jelképét. Továbbá a mindkét másik projektben táblák jellemzéséhez közösen használt felsorolások is itt vannak a **sharedtypes** alcsoomagban.

3.2.5. A terveyskomponens dokumentációja



23. ábra: egy tervrajz készítésének szekvenciadiagramja, 1. rész (HTTP-kéréstől a transzformációs gráf létrehozásáig)

A tervrajzkomponens (aminek a működését a 23. ábra és a 24. ábra vázolja fel) felelős a tervrajz elkészítéséhez szükséges összes lépésért az erre irányuló HTTP-kérések fogadásától a térképes adatok megszerzésén és a tervrajz összeállításán át a jelzőtáblák elhelyezéséig és a tervrajznak adatátviteli objektumokká alakításáig annak elemeivel együtt.

3.2.5.1. API alcsomag

3.2.5.1.1 `TemplateController`

Az `api` csomag `controller` alcsomagjában található a `TemplateController` osztály, ami megvalósítja a `/template` végpontot. A végpont annotációi a Jakarta EE specifikációinak felelnek meg, a `jakarta.ws.rs.Path` annotációt használja az osztály a végpont definiálásához. Egyetlen metódusa, a `convertToMap`, egy GET kérést fogad (a `jakarta.ws.rs.GET` annotációval megadva) az osztály `Path`-annotációjában megadott útvonalra, és egy `Response` objektumot ad vissza, mely a builder tervezési mintával példányosul a metódus visszatérésekor. Paramétereit a lekérdezés paramétereiből kapja (ezért a `@QueryParam` annotáció felelős). A `TemplateController` befecskendez egy `MapTemplateConversionService` típusú beant (`mapTemplateConversionService` néven), mely a lekérdezésben a befogó téglalap szélességi és hosszúsági fokait leíró paraméterekből készített `MapAreaDto` példányt és a táblázás szükségességét eldöntő igaz-hamis értéket (`hasSignage`) megkapva végigmegy a tervrajz készítésének minden lépésén az egyes lépéseket naplózva, majd visszatér egy `TemplateDto` objektummal, ami a kész tervrajz reprezentációja adatátviteli objektumként, melyet a visszaadott válasz törzsében JSON formátumban küld el a végpont.

3.2.5.1.2 `MapTemplateConversionService`

A `MapTemplateConversionService` szolgáltatásosztály egy alkalmazás hatókörű beanként működik. Végrehajtja egyetlen nyilvános metódusával, a `getTemplateFromMap` a tervrajz készítésének lépéseit, a tervrajz elkészítéséhez szükséges befecskendezett szolgáltatásosztályokkal. Az osztály a tervrajz készítése közben naplózza is az egyes lépéseket, és azoknak időtartamát, így a végén láthatjuk, mennyi időt vett igénybe a tervrajz elkészítése és táblázása. Alább olvashatók a `getTemplateFromMap` metódus lépései.

1. Kiszámolja és naplózza a befogó téglalap (**bbox** paraméter) területét. Ha a befogó téglalap területe nagyobb, mint 10 négyzetkilométer (a kiszámított terület értéke több mint 10 000 000), a metódus **IllegalArgumentException** típusú kivételt dob.
2. Az **OverpassQueryService** szolgáltatásosztály befecskendezett példányával lekérdezi az Overpass API meghívásával az OpenStreetMap út- és pontadatait (**way**, **node**), amit megkeper a JSON-objektumokat tartalmazó, feldolgozásra kész OSM-adathalmazként. A lépést naplózza, a végén kiírja a végrehajtási idejét.
3. Az **OsmJsonDataService** szolgáltatásosztály befecskendezett példányával elemezteti az **overpassQueryService**-től szerzett **OsmJsonDataset**-példányt. Az **OsmJsonDataService** osztály **elementsToTransformGraph** metódusa az elemzést követően visszaadja a transzformációs gráfot, melyből tervrajzot lehet készíteni. A lépést naplózza, a végén kiírja a végrehajtási idejét.
4. A transzformációs gráfot átküldi az azt feldolgozó **TransformGraphProcessorService** szolgáltatásosztály befecskendezett példányának. A szolgáltatás elvégzetteti a transzformációs gráffal a címkék hozzáadását és átalakítását, hogy az OpenStreetMaptől szerzett információt feljavítsa a készülő tervrajz számára, majd elkészíti a transzformációs gráf modelljét használva a tervrajzot (aminek típusa **RoadTemplate**), és visszaadja azt. A lépést naplózza, a végén kiírja a végrehajtási idejét.
5. Naplózza az elkészült tervrajz útjainak (**Road**) és pontjainak (**Node**) számát, valamint azt, hogy az elkészült tervrajz a befogó téglalapból kilógó feldolgozott útszakaszokkal együtt mekkora területű téglalapot feshít ki, és ez mekkora a kiválasztott területhez képest.
6. Ha a metódus **hasSignage** paraméterének értéke igaz, a befecskendezett **RuleSetProvider** bean példányától megszerzi a szabályhalmazt, majd a szabályhalmaz **applyToTemplate** metódusának meghívásával alkalmazza a szabályokat a tervrajzra. A lépést naplózza, a végén kiírja a végrehajtási idejét.
7. Naplózza a tervrajz elkészítésének teljes idejét az Overpass API hívásával együtt és nélküle.
8. A **RoadTemplateDtoMapper** statikus **toDto** metódusával adatátviteli objektummá alakítja a tervrajzot, és visszatér vele.

```

2026-04-18 20:29:28,516 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Selected area: 214673,29 m2
2026-04-18 20:29:28,517 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) querying Overpass API...
2026-04-18 20:29:28,517 INFO [roadtemplate.query.OverpassQueryService] (executor-thread-1) ----
2026-04-18 20:29:28,517 INFO [roadtemplate.query.OverpassQueryService] (executor-thread-1) Fetching template data from Overpass API with the following query:
    [out:json][timeout:90];
    (
      node(47.48991996055614, 18.579339981079105, 47.49260027116379, 18.58869552612305);
      way["highway"](47.48991996055614, 18.579339981079105, 47.49260027116379, 18.58869552612305);
    );
    out geom;

2026-04-18 20:29:29,169 INFO [roadtemplate.query.OverpassQueryService] (executor-thread-1) Fetch successful
2026-04-18 20:29:29,170 INFO [roadtemplate.query.OverpassQueryService] (executor-thread-1) Processed 20 ways as JSON objects
2026-04-18 20:29:29,170 INFO [roadtemplate.query.OverpassQueryService] (executor-thread-1) Processed 20 nodes as JSON objects
2026-04-18 20:29:29,170 INFO [roadtemplate.query.OverpassQueryService] (executor-thread-1) ----
2026-04-18 20:29:29,171 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Queried Overpass API in 653 ms
2026-04-18 20:29:29,171 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Passing 65 nodes and 20 ways to JsonDataService
2026-04-18 20:29:29,176 INFO [roadtemplate.osmjson.service.OsmjsonDataService] (executor-thread-1) Minimum latitude of dataset: 47.4890094, minimum longitude of dataset: 18.5756133
2026-04-18 20:29:29,177 INFO [roadtemplate.osmjson.service.OsmjsonDataService] (executor-thread-1) Template bounds:
    top left: Position[x=0.0, y=-0.0]
    bottom right: Position[x=2583.6008131479457, y=-697.89528363080828]

2026-04-18 20:29:29,188 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) OSM JSON data processed as TransformGraph in 16 ms
2026-04-18 20:29:29,188 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Passing graph of 43 vics and 46 edges to template processor
2026-04-18 20:29:29,189 INFO [roadtemplate.transform.model.TransformGraph] (executor-thread-1) Transform graph: deducing unmarked priority rules
2026-04-18 20:29:29,191 INFO [roadtemplate.transform.model.TransformGraph] (executor-thread-1) Added 4 probable unmarked give-way obligations
2026-04-18 20:29:29,191 INFO [roadtemplate.transform.model.TransformGraph] (executor-thread-1) Transform graph: merging duplicate crossings...
2026-04-18 20:29:29,191 INFO [roadtemplate.transform.model.TransformGraph] (executor-thread-1) Merged duplicate crossings at 0 location
2026-04-18 20:29:29,191 INFO [roadtemplate.transform.model.TransformGraph] (executor-thread-1) Transform graph: merging probable signalled crossings marked as distinct...
2026-04-18 20:29:29,191 INFO [roadtemplate.transform.model.TransformGraph] (executor-thread-1) Merged 0 signal crossings

2026-04-18 20:29:29,197 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Transform graph turned into template in 8 ms
2026-04-18 20:29:29,198 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Template has 46 roads and 43 nodes
2026-04-18 20:29:29,198 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Template size is 375304,34 m2 (174,83% of size selected in query)
2026-04-18 20:29:29,198 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Applying signage...
2026-04-18 20:29:29,233 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Signage applied to template in 34 ms
2026-04-18 20:29:29,233 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Template created and signalised in 716 ms (62 ms without Overpass API query)
2026-04-18 20:29:29,234 INFO [roadtemplate.api.service.MapTemplateConversionService] (executor-thread-1) Returning template as DTG...

```

25. ábra: egy tervrajz elkészítésének naplózása. A *MapTemplateConversionService* bejegyzései a naplóban pirossal vannak kiemelve.

3.2.5.1.3 Adatátviteli objektumok

A **dto** csomagban vannak az adatátviteli objektumok (DTO-k). Ezek mind rekordok, és csak adatokat tárolnak, önálló logikával nem rendelkeznek, a tervrajzkomponensben csak a **TemplateController** és a **MapTemplateConversionService** használják, így a komponens „belső” részeiben nem vesz részt. Sem a modell, sem a többi szolgáltatásosztály nem használja az adatátviteli objektumokat.

A csomagban a **MapAreaDto** összegyűjti egy struktúrába a `/template` végpont lekérdezéséből a befogó téglalap adatait. A többi adatátviteli objektum a tervrajz kiadott JSON formátumának meghatározására van; a **RoadTemplateDto** közvetlen adattagként vagy valamelyik adattagjának adattagjaként tartalmaz minden adatátviteli objektumot a **template** alcsomagban. Sok helyen az adatstruktúrák egyeznek a modell adattagjaival, de bizonyos adattagokat nem vesz át (pl. táblakompozíciók tagjainak maximális száma, ami a táblázás folyamatában játszik csak szerepet), máshol pedig hozzáad értékeket (pl. a táblakompozíciók abszolút pozíciója, hogy könnyebben lehessen őket elhelyezni egy kétdimenziós vásznon). Az adatátviteli objektumok felsorolási típusokon kívül nem tartalmaznak a modellhez tartozó típusokat. Jelzőtáblákkal kapcsolatok bizonyos adatátviteli objektumok nem a tervrajzkomponens `roadtemplate` csomagjában, hanem a `commons`-ban találhatók, mert a prezentációs komponens is használja őket.

3.2.5.1.4 Mapper osztályok

A **mapper** osztályban található a „Mapper” végződésű osztályok, melyek statikus metódusokkal rendelkeznek az osztály nevében szereplő modellosztály egy példányának adatátviteli objektummá való átalakítására. Egy tervrajz átalakításához elég a **RoadTemplateDtoMapper** `toDto` metódusát meghívni paraméterként a tervrajz **RoadTemplate**-példányát átadva, a visszaadott objektumban minden átalakul a megfelelő adatátviteli objektummá.

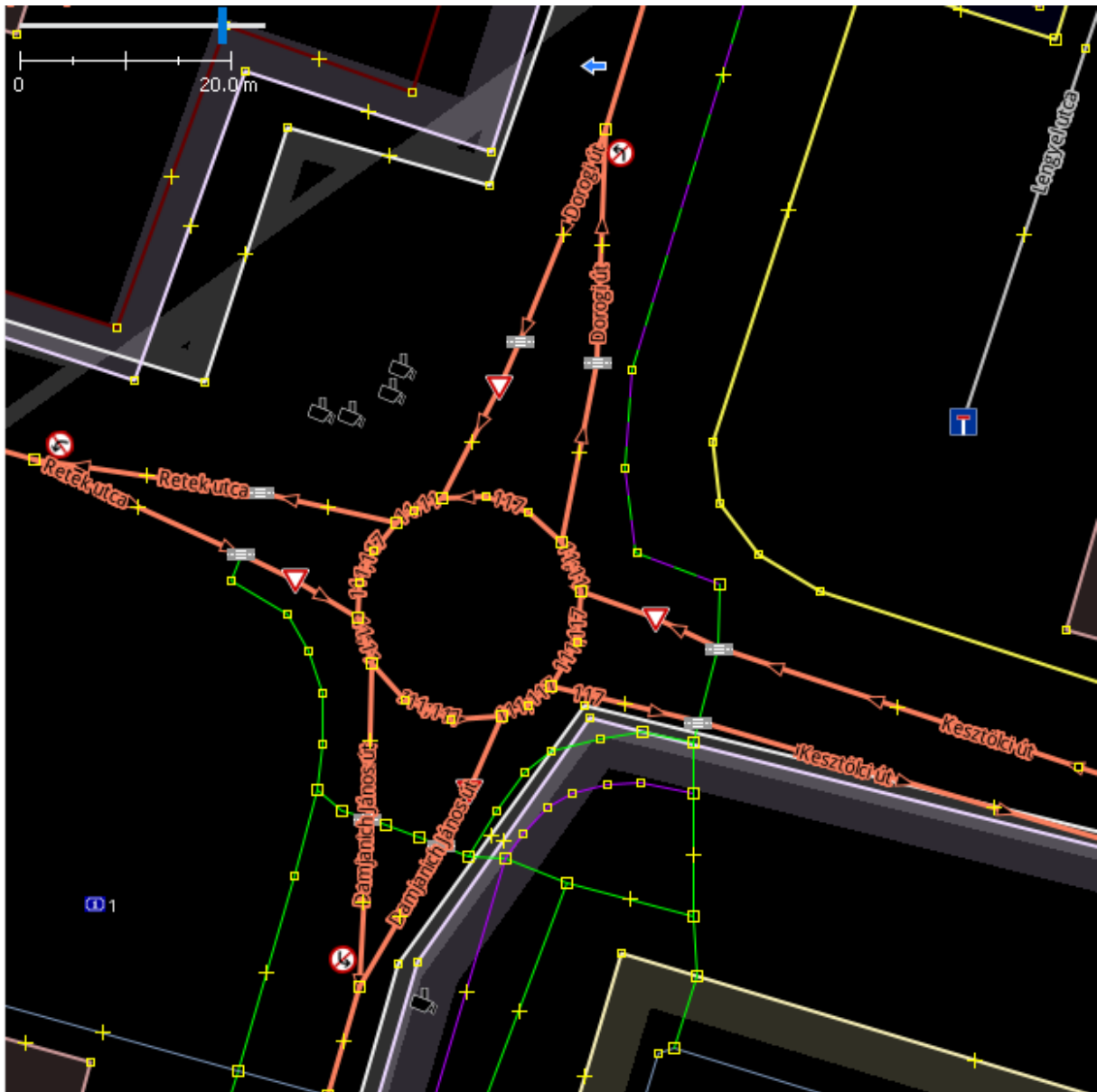
3.2.5.2. Overpass API lekérdezésének osztályai és menete

A **query** csomagban található **OverpassQueryService** befecskendezi az **OverpassRestClient** típusú `overpassApi` adattagot, ami képes hívást végezni az Overpass API felé. Az Overpass API elérési helyét az **OverpassRestClient** számára az

`application.properties` fájlban szereplő `quarkus.rest-client.overpass-api.url` attribútum értéke adja meg.

Az `OverpassQueryService` Overpass-lekérdezésének stringje a `queryTemplate` adattagban van, melyben a befogó téglalap hosszúsági és szélességi fokait lecseréli a paraméterben kapottakra a `getTemplateOf` metódus, majd az `overpassApi`-től lekéri az OSM-es adatokat. A lekérdezés az Overpass saját lekérdezésnyelvét, az Overpass Query Language-et használja, [13] és az API-tól a befogó téglalapban található OSM-utakat és OSM-pontokat szerzi meg.

Az OpenStreetMap adatai JSON formátumban érkeznek. Három fő objektumtípus van, `way` (OSM-út), `node` (OSM-pont) és `rel` (OSM-es kapcsolatok), ebből a lekérdezés csak az első kettőt kéri el az Overpass API-től a befogó téglalap területén, hiszen ezekből az adatokból fog elkészülni a tervrajz. Egy OSM-út nem feltétlen közút, sokkal általánosabb: járdákat, hegyvonulatokat és közigazgatási határokat is ezzel jelölnek, csak épp `way` a neve. Az OSM-utaknak van azonosítójuk és számos címkéjük, amikben benne van a kategóriájuk és számos egyéb tulajdonságaik (pl. ha közútról van szó, a sebességhatár vagy a sávok száma egyes irányokban). Az OSM-pontok szintén fel vannak címkézve, és számos szerepet vehetnek fel: fa, gyalogos-átkelőhely, jelzőlámpa, stb.



26. ábra: OSM-utak és OSM-pontok megjelenítve a JOSM alkalmazásban. [14] OSM-út például az épületek határa, járdák (zöld) vagy a közutak (narancs), az OSM-pontok lehetnek címkézetlenek (sárga x-ek), de akár gyalogos-átkelőhelyek (zebra ikon) vagy elsőbbségi kötelezettségek is. Ezek a címkék tárolódnak az OSM JSON-adathalmazban, majd elemzés után a modellben.

Az OSM-pontokat OSM-utak köthetik össze, azonban egy út nem feltétlen csak két pontot köt össze, érinthet többet is. A transzformációs gráf és a kész tervrajz gráfja megfeleltethető az út típusú OSM-utak és OSM-pontok elrendezésének, de a tervrajz gráfjában az utak csak kettő pontot kötnek össze, ezt a feldolgozás következő szakasza (az `OsmJsonDataService` osztály) intézi el a transzformációs gráf létrehozásakor.

A JSON formátumban kapott válaszból azután kiszűri két listába az OSM-utakat és OSM-pontokat, amik szintén JSON-objektumok, majd beleteszi őket egy `OsmJsonDataset`

objektumba (aminek az osztálya a `roadtemplate.osmjson.model` csomagban található), amivel végrehajtható a JSON-elemzés. Az `OsmJsonDataset` egy példányának tehát két listája van:

Ha a lekérdezés megghiúsul, egy **`OverpassQueryException`** dobódik. Ez ellenőrzött kivétel, ezért az `OverpassQueryService` `getTemplateOf` metódusát meghívó osztályoknak kezelniük kell.

3.2.5.3. OSM-es JSON formátumú adatok feldolgozása

3.2.5.3.1 `OsmJsonDataset`

Az **`OsmJsonDataset`** osztály csupán két listát tartalmaz:

- **`nodes`**: `List<JsonObject>` (OSM-pontok JSON-objektumként)
- **`ways`**: `List<JsonObject>` (OSM-utak JSON-objektumként)

Ilyen a lekérdezést elvégző `OverpassQueryService` visszatérési értéke, és ilyen típust vár paraméterként az **`OsmJsonDataService`**, mely ezeket az OSM-es JSON-adatokat átalakítja a címkézett transzformációs gráffá.

3.2.5.3.2 `OsmJsonDataService`

Az **`OsmJsonDataService`** szolgáltatásosztály felelős az OSM-adathalmaz (`OsmJsonDataset`) átalakításáért transzformációs gráffá.

Az osztály egyetlen nyilvános metódusa, az **`elementsToTransformGraph`** bemenetként egy `OsmJsonDataset`-et vár el.

Az adathalmaz elemeinek pozíciói szélességi és hosszúsági fokban vannak megadva, ezt a metódus minden esetben a **`GeoCoordinateMapperService`** osztály `getAsPosition` metódusával megalkotja a befogó téglalap hengeres, egyenlő távolságú vetületét () egy sík, kétdimenziós koordinátarendszerbe, középpontként a legalacsonyabb szélességi és hosszúsági fokot használva, amit a konstruktor paraméterében kap a metódus elején (mivel ez kijelölt területenként változó, a metóduson belül példányosul az osztály **`coordinateMapper`** néven, nem célszerű befecskendezni). Ezt a vetítést nagyon egyszerű implementálni, és a pontok síkra vetítése közben történő torzítás egy pár négyzetkilométeres úthálózat esetén elenyésző, még úgy is, hogy nem a minimális és maximális szélességi és

hosszúsági fokok átlagával számol, mint középpont. A `getAsPosition` vetítéskor a kivetített y koordináta -1-szeresét adja vissza, mivel a tervrajz koordinátarendszerében lefelé (tehát dél felé, mivel a tervrajzon is felfelé található észak) nő az y-érték, azonban a szélességi fokok észak felé nőnek.

Az `elementsToTransformGraph` az alábbi műveleteket végzi el:

1. Ha nincs feldolgozható út, `IllegalArgumentException` típusú kivételt dob. Feldolgozható, út típusú OSM-útnak számít az, amelynek van `highway` címkéje, és annak értéke egyike a következő közúti kategóriáknak: `primary`, `secondary`, `tertiary`, `residential`, `service`, `primary_link`, `trunk_link`, `secondary_link`, `tertiary_link`, `motorway_link`, `motorway`, `unclassified`. A módszer mindegyik kategória esetén ugyanolyan élet hoz létre a transzformációs gráfban, a `highway` címke értéke csak a precedenciaértéket befolyásolja, ami alapján a transzformációs gráf elhelyezi az elsőbbségadási kötelezettségeket (3.2.5.4.2. fejezet listájának 1. pontja). Ez a precedenciaérték számként kerül a transzformációs él címkéi közé, így azon a szinten már nincs jelentősége a kategóriáknak.
2. A `coordinateMapper` először megnézi és kivetíti a legnagyobb hosszúsági és szélességi fokú út típusú OSM-utat (`getGlobalExtreme` módszer), ez lesz a tervrajz nézetének egyik sarka (a másik pedig a 0;0). Mivel az y koordináta meg van fordítva, az x értéke a saroknak várhatóan pozitív, az y pedig negatív lesz, de a szoftver támogatja a negatív koordinátákat.
3. Utána a befecskendezett `UrbanAreaHeuristicsService` szolgáltatásosztály példányának (`urbanAreaHeuristicsService`) a `deduceUrban` módszerével különböző heurisztikák alapján (3.2.5.3.4. fejezet) megpróbálja eldönteni a program, hogy a tervrajz lakott területen belülinek tekinthető-e. A tervrajz modelljének és az egész tervrajzkomponensnek egy korlátozása az, hogy az egész tervrajz vagy lakott területen belülinek, vagy lakott területen kívülinek számít, nem lehet csak részben lakott terület.
4. Ezután az út típusú OSM-utakon végigmegy egy ciklus.
 - Egy belső ciklus az OSM-út egyes pontokat összekötő szakaszain végigmegy. Mivel egy OSM-út minden öt érintő OSM-ponttal együtt több száz méterre,

akár kilométerekre is kilóghat a befogó téglalap területéből, csak azokat a két OSM-pont közti szakaszokat dolgozza fel, melyeknek legalább egyik végén lévő pontja (az OSM-út a lekérdezésben visszakapott JSON-struktúrájában az érintett pontjairól tárolja a szélességi és hosszúsági fokukat, valamint az azonosítójukat) benne van a befogó téglalapban, így nem fed le az elkészült tervrajz sokkal nagyobb útszakaszt, mint ami a kiválasztott területen található.

- Egy szakasz feldolgozásakor a program a készülő transzformációs gráfhoz (**graph**) hozzáadja az új TransformEdge (3.2.5.4.3. fejezet) típusú transzformációs élel. Az OSM-út releváns címkeértékeit a TransformEdge processedRoadData nevű, TransformEdgeData típusú rekordjába. A címkeket a RoadAttributeDeductionService befecskendezett példány, a **roadAttributeDeductionService** végzi a 3.2.5.3.5. fejezetben leírt metódusokkal. Az élel határoló OSM-pontok azonosítóit és koordinátáit használva pedig
 - ha már létezik az azonosítóval transzformációs csúcs a készülő transzformációs gráfban (**graph**), összeköti vele egy transzformációs csúcs-foglalaton keresztül (**startNodeSlot** és **endNodeSlot** adattagok beállításával) a létrehozott élel,
 - ha még nem, akkor az OSM-pont szélességi és hosszúsági fokából **coordinateMapper**rel számolt koordinátaival és az OSM-pont azonosítójával létrehoz egy új transzformációs csúcsot a készülő transzformációs gráfban, és ezt egy transzformációs csúcs-foglalaton keresztül (**startNodeSlot** és **endNodeSlot** adattagok beállításával) összeköti az szakaszból létrehozott élel.

A transzformációs gráfról és az elemeiről, amiket a metódus a ciklusban létrehoz, a 3.2.5.4.1. fejezetben olvasható leírás.

5. A végén a metódus az **OsmJsonDataset** OSM-pontjainak a címkeit alkalmazza a transzformációs gráf csúcsaira. Mivel az utak adatstruktúrája már tartalmazta az érintett OSM-pontok azonosítóját, minden OSM-pontra csak meg kell keresni a gráfon az azonosítójának megfelelő transzformációs csúcsot, és felcímkézni a befecskendezett **vertexAttributeDeductionService** **getAttributes**

metódusának feldolgozott adataival, melynek bemenete `JsonObject` típusú JSON-objektumként az OSM-pont `tags` kulcsának értéke (ami maga is JSON-objektum), kimenete pedig egy, a transzformációs gráf csúcsának tervrajz elkészítéséhez szükséges címkéit tartalmazó `Set<TransformVertexAttribute>` típusú halmaz. Ez a felcímkézés a transzformációs gráf (`graph`) `processVertex` metódusának meghívásával történik. A `VertexAttributeDeductionService` működése a 3.2.5.3.6. fejezetben olvasható.

6. A metódus végül visszatér a `graph`-fal, ami az elkészült transzformációs gráf.

3.2.5.3.3 `GeoCoordinateMapperService`: szélességi és hosszúsági fokok vetülete a koordinátarendszerben

A `GeoCoordinateMapperService` osztály képes szélességi és hosszúsági fokokat átalakítani egy szélességi és hosszúsági fokot egy síkra. Az osztály a földrajzi koordináták egyenlő távolságú hengeres vetületét használja fel a leképzésükön a készülő tervrajz sík kétdimenziós vásznán, ahol az x és y tengely értékei a konstruktorban kapott kiindulási ponttól való távolságot reprezentálják méterben. Egy földrajzi koordináta vetületének x értéke kelet felé növekszik, y értéke azonban dél felé, tehát ha pont y értéke magasabb egy másikénál, akkor a szélességi foka alacsonyabb (mivel közelebb van az egyenlítőhöz). Ez azért van, mert sok x és y koordinátán alapuló vásznon (ilyen az alkalmazásban használt GoJS-diagram vászna is, melyről a 3.3.6.2.1. fejezetben lehet olvasni).

Ez a vetületfajta ugyan nem a legpontosabb (más típusokkal, pl. a Mercator-vetülettel szemben), de egy kisebb terület kivetítéséhez megfelel, és az implementációja egyszerű. [15] Egy legfeljebb néhány négyzetkilométeres síkon elenyésző a torzítás mértéke, a tervrajzokon sem látszik szemmel látható torzítás és a táblázási szabályok távolságai is racionálisan alkalmazkodnak a térképes adatok úthálózata alapján készült tervrajzon.

A vetület kiszámításához a konstruktor kap egy középpontnak számító szélességi és hosszúsági fokot (`originLat: double, originLon: double`), majd innentől kezdve a `getAsPosition(lat: double, lon: double)` meghívásával (ahol `lat` a szélességi, `lon` a hosszúsági foka a vetíteni kívánt koordinátának) lehet a méter egységű síkra vetíteni földrajzi koordinátákat.

A `getAsPosition` az alábbi módon alakítja át a szélességi és hosszúsági fokot x és y értékké:

- **x**: a hosszúsági fok és a kiinduló hosszúsági fok radiánjainak különbsége a Föld sugarával és a kiinduló szélességi fok koszinuszával megszorozva
- **y**: a szélességi fok és a kiinduló szélességi fok radiánjainak különbsége a Föld sugarával és (-1)-gyel megszorozva.

A Föld egyenlítő menti sugara (a konvenció alapján elnevezett **R** konstansban tárolva) méterben van megadva, mivel méter a sík egysége is.

A vetület típusának kiválasztásához és a megértéséhez, implementálás módjához segédeszközként a Codexet használtam, aminek feladatként megadtam, hogy írjon le egy megfelelő, lehetőleg egyszerű átalakítási módszert, melyre a bemenet egy földrajzi koordináta, a kimenet pedig egy megfelelően arányos, síkra történő, `Position`-példányként meghatározható pont. A Codex által felvázolt lehetőségek jó kiindulási pontot adtak a vetületek kutatására (a Mercator- és webes Mercator-vetületet ajánlotta még). A különböző vetületek implementációjának komplexitását megfontolva és John P. Snyder *Map projections: A working manual* publikációjában a vetületek leírását [15] elemezve arra jutottam, hogy a hengeres egyenlő távolságú vetület egyszerűsége mellett is alkalmas erre a feladatra, az osztályban magát a kódot már én írtam.

3.2.5.3.4 `UrbanAreaHeuristicsService`: heurisztikák lakott terület megállapítására

Az OSM-es JSON-adathalmaz feldolgozásakor meg kell állapítani, hogy a készülő tervrajz lakott területen belül vagy lakott területen kívül található (ezt a transzformációs gráf (`TransformGraph`) `isUrban` adattagja tárolja, ami később a tervrajznak is átadódik). Ez az adat a kész tervrajz esetén a táblázásnál számít.

Bár az OpenStreetMap rendelkezik külön jelöléssel, hogy egy út lakott területen belüli-e (az OSM-út objektumában a címkék közt található `urban` kulcs jelöli `yes` értékkel [16]), ezt az előzetes kutatás közben nem láttam használva ezt a jelölést az elemzett OSM-es adatokban, így egyéb feltételekhez (heurisztikákhoz) kötöttem a lakott terület megállapítását. Ezt oldja meg az `UrbanAreaHeuristicsService` nevű alkalmazás hatókörű bean.

A **deduceUrban** az egyetlen, nyilvános metódusa, paramétere OSM-utak listája, visszatérési értéke igaz-hamis érték (lakott terület vagy nem).

Minden vizsgált OSM-útnál alábbi heurisztikákat használja a lakott terület státusz eldöntéséhez:

- Autópályát vagy autóutat tartalmaz (**highway** címke értéke **motorway** vagy **expressway**): visszatér **hamis** értékkel
- Lakóövezeti utat tartalmaz (**highway** címke értéke **residential**): visszatér **igaz** értékkel
- Kerékpársávot tartalmaz (**cycleway:right** vagy **cycleway:left** címke jelen van bármilyen értékkel): visszatér **igaz** értékkel
- Az út sebességkorlátjának a forrását megjelölték és a forrás a lakott területen belüli általános sebességhatár (**source:maxspeed** címke értéke tartalmazza, hogy **urban**): visszatér **igaz** értékkel
- Az út nevében benne van, hogy „utca” (**name** címke jelen van és értéke tartalmazza az „utca” stringet): visszatér **igaz** értékkel

Amennyiben egy út vizsgálatakor sem tért még vissza, a ciklus végén **hamis** értéket ad vissza, szóval az alapeset az, hogy lakott területen kívüli a vizsgált terület.

A heurisztikák nem működnek garantáltan, vannak hamis pozitív (például nem kizárt, hogy legyen lakott területen kívül kerékpársáv, [17] de annyira ritka, hogy elfogadható a lakott terület jelének), és hamis negatív értékek is (épp nincs igaz értéket kiváltó út az adatok közt), de nagyrészt helyes értéket ad vissza a **deduceUrban** metódus, ha a kiválasztott területnek tényleg egésze lakott terület vagy nem lakott terület.

3.2.5.3.5 **RoadAttributeDeductionService**: utak címkéinek feldolgozása

Az OSM-utak különböző releváns címkéit a **RoadAttributeDeductionService** alkalmazás hatókörű bean képes feldolgozni, az **OsmJsonDataService** ezeket felhasználva építi fel a transzformációs **TranformEdgeData** típusú címkéit, amit a transzformációs gráf átalakít az út adataivá a tervrajzban (**RoadTemplate**). Minden metódus kéri az OSM-út **tags** kulcsának értékét, ami egy, a címkéket tartalmazó **JsonObject**, de van, ami más argumentumot is elvár.

Az alábbi nyilvános metódusokkal rendelkezik:

- **isPriorityRoad**
 - Argumentum: OSM-út címkéi (JsonObject)
 - Visszatérési érték: főút-e az útszakasz címke (priority_road=yes) alapján (boolean)
- **isSlipLane**
 - Argumentum: OSM-út címkéi (JsonObject)
 - Visszatérési érték: összekötő út-e az útszakasz (a highway címke értéke tartalmazza-e a link stringet) (boolean)
- **getRestrictionsForward**
 - Argumentum: OSM-út címkéi (JsonObject), lakott terület-e a tervrajz (boolean)
 - Visszatérési érték: út korlátozásainak listája címkék alapján (maxspeed, maxweight) az OSM-út irányával megegyező irányban haladók részére (List<Restriction>)
- **getRestrictionsBackward**
 - Argumentum: OSM-út címkéi (JsonObject), lakott terület-e a tervrajz (boolean)
 - Visszatérési érték: út korlátozásainak listája címkék alapján (maxspeed, maxweight) az OSM-út irányával ellenkező irányban haladók részére (List<Restriction>)
- **getLaneConfiguration**
 - Argumentum: OSM-út címkéi (JsonObject)
 - Visszatérési érték: az útszakasz sávkonfigurációja (List<Lane>) a lanes:* és oneway kulcsú címkék alapján kikövetkeztetve (a sávkonfigurációról részletesebben a 3.2.5.5.1 részben)
- **getPrecedence**
 - Argumentum: OSM-út címkéi (JsonObject)
 - Visszatérési érték: az út precedenciája (int) (3.2.5.4.2. fejezet); ha alacsonyabb, mint egy másik úté, akkor annak alá van rendelve
 - A precedenciaérték az OSM-út highway címkéjének értékétől függ:
 - motorway: 10

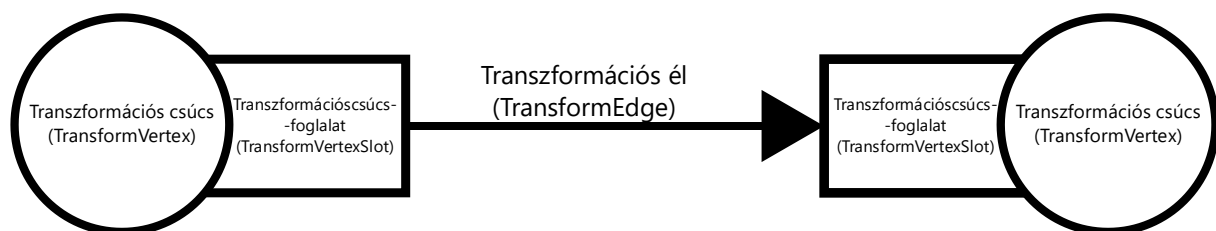
- primary: 9
 - secondary: 8
 - tertiary: 7
 - residential: 6
 - service: 5
 - egyéb: 4
- **getIfRoundabout**
 - Argumentum: OSM-út címkéi (JsonObject)
 - Visszatérési érték: az útszakasz körforgalom része-e címke alapján (junction=roundabout) (boolean)
 - **getRouteNumber**
 - Argumentum: OSM-út címkéi (JsonObject)
 - Visszatérési érték: az útszakasz száma (OSM-út ref értéke) ha van, vagy nullérték (String)
 - Az út száma String típusú, mivel az útszám értékének nincs relevanciája, ráadásul lehet betű is benne (pl. „M”, vagy akár „A”, ha a terület átlóg egy országhatáron)

3.2.5.3.6 NodeAttributeDeductionService

A RoadAttributeDeductionService-szel ellentétben a **NodeAttributeDeductionService** csak egy nyilvános metódussal rendelkezik a **getAttributes**-szal, ami egy OSM-pont címkéit kéri JsonObjectként visszaad egy TransformVertexAttribute típusú halmazt, ami tartalmazza az OSM-ponthoz tartozó releváns címkék típusát (3.2.5.5.7. fejezet) és irányát.

3.2.5.4. Transzformációs gráf

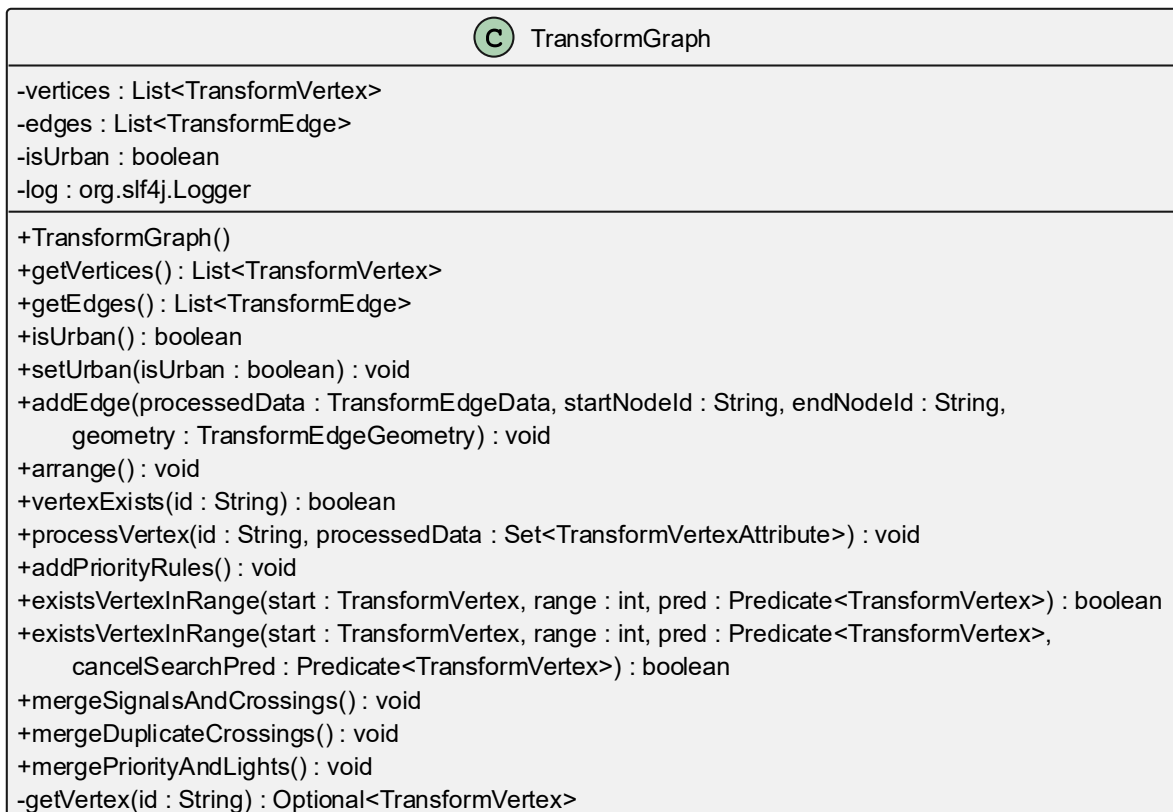
3.2.5.4.1 A transzformációs gráf modellje (TransformGraph)



27. ábra: transzformációs gráf modellje

A transzformációs gráf egy köztes állapotot tükröz az OSM-es JSON-adathalmaz és a tervrajz között. Modellje a `TransformGraph` osztályban található, egy speciális címkézett irányított gráfnak feleltethető meg, ahol a csúcsok a `TransformVertex` példányai, az élek pedig `TransformEdge`-példányok, azonban a csúcsok és élek között áll egy transzformációs csúcsgoglalat (`TransformVertexSlot`). A foglalatok azért vannak, hogy a gráf könnyen átalakítható legyen tervrajzzá, aminek a gráfstruktúrája szintén használ foglalatokat, ennek indoklása a tervrajz modelljéről szóló részben olvasható (3.2.5.5. fejezet), de itt is alkalmazva van egy-két előnye, pl. amikor az addig jelöletlen elsőbbségadási kötelezettségek elhelyezésekor megnézi a csúcsgoglaltain keresztül a metódus, hogy az él sávkonfigurációja alapján van-e forgalmi elérése az egyes élek felől a csúcshnak. A foglalat ilyenkor elvégzi az él címkéi közt található sávkonfiguráció elemzését és összeveti az él irányával (az adott csúcshból indul vagy oda érkezik), hogy a metódus eldöntse, hogy kell-e a jelölés.

A transzformációs gráf és elemei egyszerűbb osztályok, mivel nem ezeken történik a táblázás, sokkal kevesebb logikát tartalmaznak és átláthatóbbak, mint a tervrajz modellje.



28. ábra: A `TransformGraph` osztály osztálydiagramja

3.2.5.4.2 A transzformációs gráf átalakításai

A transzformációs gráf átalakításokon megy át, mielőtt tervrajz lesz belőle (innen jön a neve), az ezeket végrehajtó metódusokat a **TransformGraph** magában tartalmazza; hogy minden transzformáció egyszerre legyen végrehajtva és helyes sorrendben, a nyilvános **arrange** metódus hívja meg őket, tehát kívülről az **arrange** meghívásával ajánlott a transzformációs gráf elkészítését követően végrehajtani az átalakításokat, mielőtt egy szolgáltatásosztály átalakítja tervrajzzá (bár az egyes transzformációs metódusok publikusak, megadva a lehetőséget a „manuális” transzformációra).

Az átalakítások az OpenStreetMap jelöléseinek következetlenségeinek részleges javítására és az OpenStreetMapen nem jelölt jelölendő tulajdonságok gráfhoz való hozzáadására szolgál. További fejlesztés esetén így érdemes fejben tartani, hogy a transzformációk azt veszik alapul, hogy OpenStreetMap-adathalmazból jött létre a transzformációs gráf.

Az egyes átalakítások naplózzák a folyamat kezdetét, majd a végén azt is kiírják, hány helyen történt változtatás.

Az **arrange** metódus az alábbi metódusokat hívja meg, a lista sorrendjében:

1. **addPriorityRules**: a JSON-elemzéskor az transzformációs élek kaptak egy precedenciaértéket is, ami a **processedRoadData** rekordban található (**precedence** néven) a többi adat mellett. Azonban a precedencia nem a tervrajz útjának modelljéhez kell, hanem ennek a metódusnak a végrehajtásához: ha egy csúcsba legalább 3 él torkollik (tehát kereszteződés vagy elágazás), megvizsgálja, hogy az OpenStreetMap adataiból már el lettek-e helyezve az elsőbbségadási kötelezettségek (környékbeli foglalat rendelkezik-e **STOP** vagy **GIVE_WAY** attribútumtípussal (**attributes** adattag a **TransformVertexSlot** rekordban)); ha nem, akkor ellenőrzi, hogy van-e különbség a precedenciák között, és ha van, akkor a maximálisnál kisebb precedenciájú utak foglalatához hozzáadja az elsőbbségadási kötelezettséget jelölő attribútumot. Ez mindig **GIVE_WAY** típusú, mivel a rendelkezésre álló információ alapján nem lehet eldönteni, hogy az alárendeltség mellett megállási kötelezettség is van-e a ponton.
2. **mergeDuplicateCrossings**: néha az egymás mellett párhuzamosan futó járdavonalak keresztezése kettő átkelőként címkézett OSM-pontként van jelölve. A

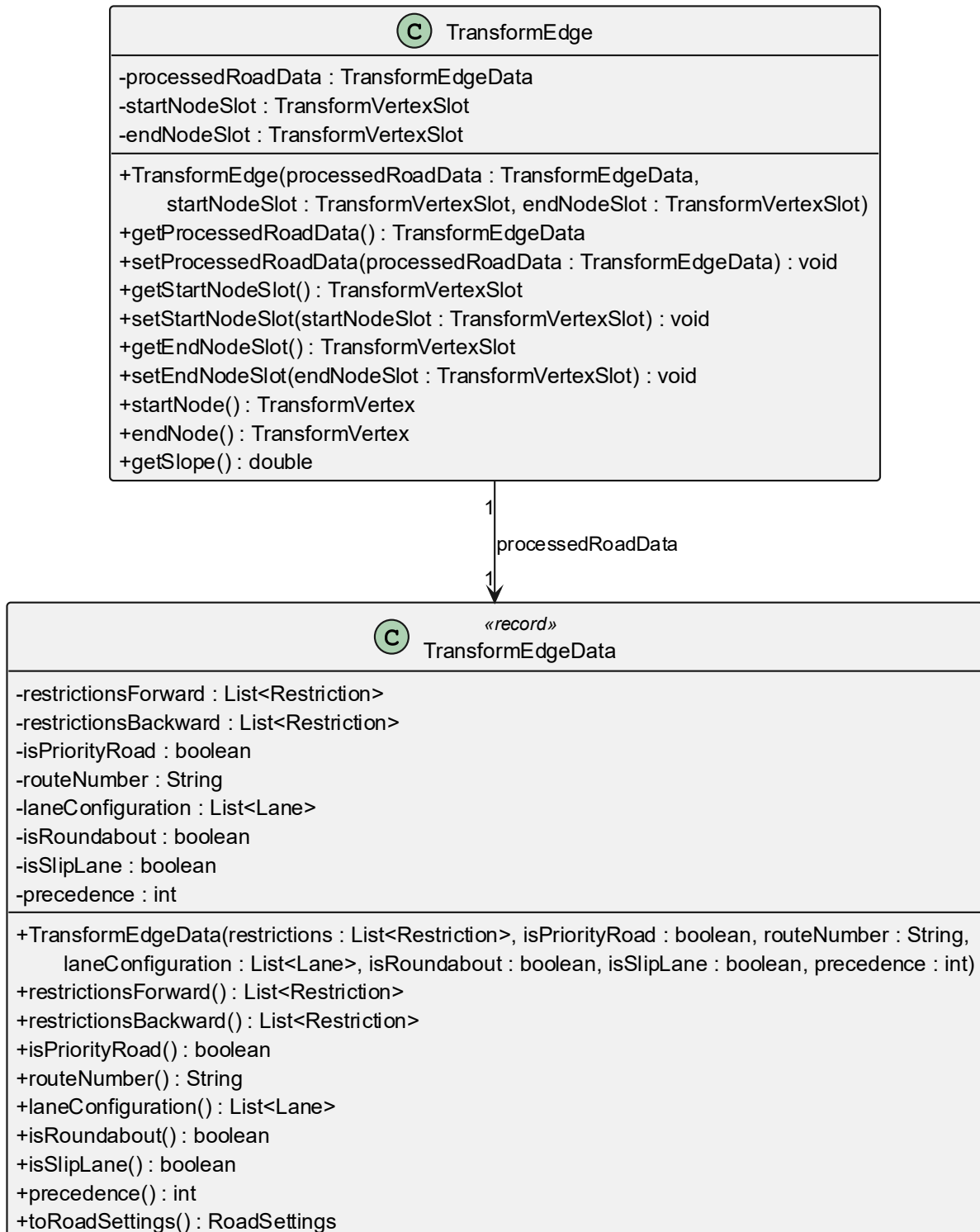
`mergeDuplicateCrossings` ellenőrzi, hogy két szomszédos csúcs foglalatai tartalmazznak-e átkelőt; ha igen, akkor az egyik átkelő attribútumot kitörli a csúcsának a halmazából.

3. **`mergeSignalsAndCrossings`**: az OpenStreetMap adataiban a lámpával irányított zebrákat úgy szokták jelezni, hogy egymás mellett helyeznek el egy `crossing` értékű `highway` címkével ellátott (átkelő) és egy `traffic_signals` értékű `highway` címkével ellátott (jelzőlámpát jelölő) OSM-pontot. Ez a metódus a gráfon a `CROSSING` címkéjű foglalatokkal szomszédos jelzőlámpás (`TRAFFIC_SIGNAL` címkéjű) foglalatok címkéjét a zebra mögé helyezi, ha a zebrás foglalat pontja és a jelzőlámpás foglalat pontja egymáshoz közel van (a távolság x és y tengelyen is 20 alatt van). (Megjegyzés: `crossing` értékű `highway` esetén lehet még az OSM-pont címkéi közt egy hozzáadott `crossing` kulcs is, melynek értéke lehet „`traffic_signals`”, ami explicit módon jelzi az átkelő lámpás irányítását, de ez egyrészt nem következetes, mert gyakran a lámpás zebránál is csak `yes` a `crossing` értéke, másrészt ekkor is szinte mindig van egy szomszédos `highway=traffic_signals` címkéjű OSM-pont, így elemzéskor az OpenStreetMapen explicit módon lámpásnak jelölt és a nem jelölt átkelők közt nem tesz az elemző különbséget.)
4. **`mergePriorityAndLights`**: az OpenStreetMapből vagy az `addPriorityRules` transzformációs lépésből származó `GIVE_WAY` vagy `STOP` (tehát elsőbbségadási kötelezettséget szabályzó) attribútumokat egyesíti az ugyanazon úton lévő ugyanarra néző jelzőlámpákkal, ha van ilyen. Ez megkönnyíti a tervrajz feldolgozásakor a lámpás keresztezések helyes táblázását (az egy pontra vonatkozó elsőbbségadási kötelezettségnek és a lámpának kézenfekvő egy helyen lenni). A lépés azért is szükséges, mert egy kereszteződéstől vagy elágazástól (tehát 3 vagy több élnek foglalatot biztosító csúcstól) 1-2 pontra is lehet elsőbbségadási kötelezettség vagy lámpa, pl. ha van lámpás átkelőhely is a csomópontban, vagy bonyolultabb a geometriája.

3.2.5.4.3 **`TransformEdge`**: transzformációs él

A transzformációs él rekordja tárolja a kezdődő csúcs hozzá tartozó foglalatát (**`startNodeSlot`**), a végződő csúcs hozzá tartozó foglalatát (**`endNodeSlot`**), valamint az

úthoz kapcsolódó címkéből kinyert feldolgozott adatokat (**processedRoadData**, egy egyszerű TransformEdgeData adatstruktúrában).



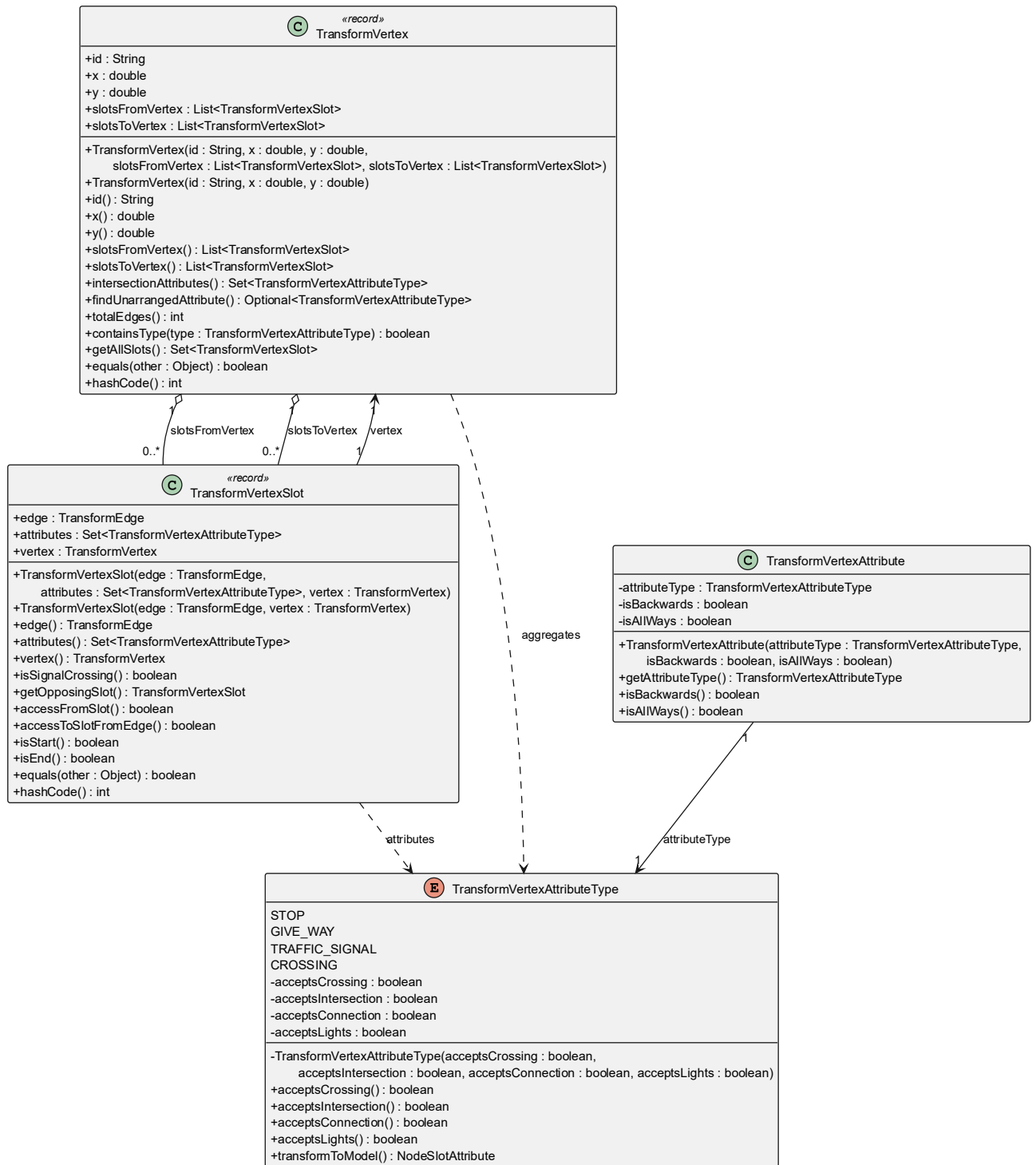
29. ábra: a TransformEdge és TransformEdgeData osztálydiagramjai

3.2.5.4.4 **TransformVertex** és **TransformVertexSlot**: transzformációs csúcsok és transzformációs csúcs-foglalatok

A transzformációs csúcsok a következő adatokat tárolják:

- **slotsFromVertex, slotsToVertex:** `List<TransformVertex>`: az éleket tartalmazó foglalatok két külön listában (attól függően, hogy kiinduló vagy beérkező élt tartalmaz a foglalat)
- **id:** `int`: egy azonosító, amely a JSON-elemzés során annak az OSM-pontnak az azonosítója, amiről készült a csúcs
- **x:** `double`, **y:** `double`: a koordináták.

Biztosítva vannak metódusok az összes csatlakozó foglalat attribútumainak lekérésére (**intersectionAttributes**), egy attribútum jelenlétének tesztelésére (**containsType**) valamint az összes (tehát kiinduló és beérkező élt összekötő) foglalat lekérésére (**getAllSlots**). A **TransformVertex** felülírja a rekordok alapvető **equals** metódusát, mivel adattagjai egymásra tartalmaznak referenciákat, így nem lehet őket rekurzívan ellenőrizni: csupán az azonosítókat hasonlítja össze, de mivel minden OSM-pontnak egy transzformációs csúcs feleltethető meg a program helyes működése mellett, ez egy helyes feltétel az egyenlőségre.



30. ábra: A TransformVertex, TransformVertexSlot és az attribútumok osztályainak diagramja

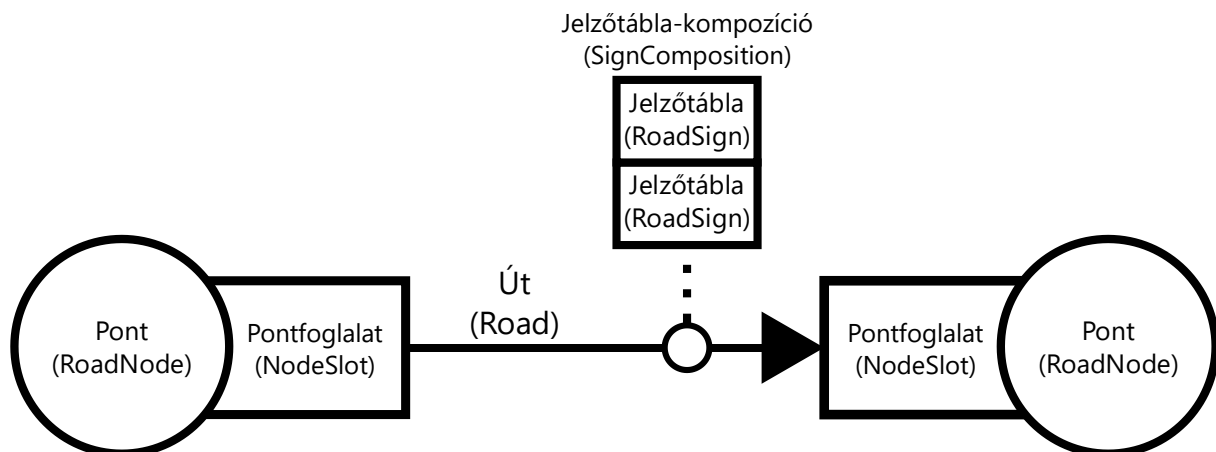
3.2.5.4.5 TransformGraphProcessorService: a transzformációs gráf feldolgozása

A **TransformGraphProcessorService** alkalmazás hatókörű bean felelős a transzformációs gráf feldolgozásáért és átalakításáért tervrajzzá (RoadTemplate).

A tervrajz modelljében a jelzőtábla-kompozíciók (3.2.5.5.2. fejezet) rendelkeznek néhány `application.properties`-ben beállítható paraméterrel. A táblakompozíció osztálya, a `SignComposition` modellosztály, nem tud bean jellegű szolgáltatásosztályokat befecskendezni, sem konfigurációs értékeket az `application.properties` fájlból. Így mivel a tervrajz modellje ebben a szolgáltatásban jön létre, ez fecskendezi be adattagként a `SignCompositionConfigService` típusú `compositionConfigService` adattagot, mely az `application.properties` konfigurációjából beolvassa a táblakompozíciók maximális számát, és hogy a jelzőlámpák beleszámítsanak-e a kompozíció darabszámába (3.2.5.5.2. fejezet).

Egyetlen nyilvános metódusa van az osztálynak, a `processTransformGraph`, mely egy transzformációs gráfot vár argumentumként. A metódus először végrehajtja a 3.2.5.4.2. fejezetben felsorolt változtatásokat a transzformációs gráf `arrange` metódusának hívásával, majd a gráf komponenseit és címkéit felhasználva létrehozza a `RoadTemplate` típusú tervrajzot. A visszaadott tervrajz már maga a modell, lehet rá táblázási szabályokat alkalmazni, vagy ha nem kell a táblázás, adatátviteli objektummá alakítani és JSON-ként elküldeni az alkalmazásnak.

3.2.5.5. Tervrajz modellje



31. ábra: tervrajz gráfjának modellje

A tervrajz modellosztálya a `RoadTemplate`. A tervrajz egy címkézett élekkel rendelkező irányított gráf (a transzformációs gráfhoz hasonlóan), struktúrája hasonlít a transzformációs gráféhoz. A csúcsok a tervrajz pontjainak (`RoadNode`) feleltethetők meg, melyekhez onnan

kiinduló és ott végződő irányított élek megfeleltethető utak (**Road**) csatlakoznak pontfoglatokon (**NodeSlot**) keresztül. Egy pontfoglatat egy utat és egy pontot köt össze.

Az élek és foglatok adattagként tárolt címkéi, ha a **TransformGraphProcessorService**-ből (3.2.5.4.5. fejezet) származik a tervrajz, a transzformációs gráf címkéiből jönnek, de azokkal ellentétben nincsenek egy külön rekordba elkülönítve.

Amellett, hogy a címkék itt nem rekordba vannak elkülönítve, hanem a modellosztály adattagjai, szintén a tervrajz sajátossága a transzformációs gráffal ellentétben, hogy a gráf elemei számos metódussal rendelkeznek, amik a címkék (adattagok) és a környezetükben található gráfelemek felhasználásával képesek további információt adni jellegükről a tervrajzban, tehát röviden a tervrajz elemei gazdag saját logikával rendelkeznek. Példaképpen egy pontfoglatat rendelkezik olyan metódussal, ami képes megmondani, hogy a pont, amihez csatlakozik kereszteződés-e, vagy a foglaltól jobbra, balra, egyenesen csatlakoznak-e a pontjához utak, és ha igen, arra be lehet-e hajtani a foglatat útjából.

A tervrajz gráfjának szintén fontos elemei az utakban tárolt jelzőtábla-kompozíciók, melyek jelzőtáblák listáit tárolják. Az utak a táblakompozíciókat egy szótárban tárolják (**signCompositions**), aminek kulcsa egy **SignLocation** típusú rekord, mely a jelzőtábla pozíciójának egy reprezentációját tárolja.

A tervrajz elemeinek saját logikája lehetővé teszi a gráf bejárását olyan szempontok alapján, mint az út követése (útszámozás vagy a ponttal geometriailag szemben lévő foglatat alapján) vagy bejárás egy adott pontból forgalmi elérhetőség szerint (olyan szomszédos pontok látogatása az érintett pontokból, ahol a kettőt összekötő út kétirányú vagy egyirányú a szomszédos pont felé).

3.2.5.5.1 **Road**: a tervrajz útjai

Az utakat a tervrajzban **Road** típusú objektumok reprezentálják, melyek a tervrajzot gráfként értelmezve irányított, címkézett éleknek feleltethetők meg, valamint a tervrajz által ábrázolt úthálózat útjainak. Adattagjai és metódusai az adott útszakasz különböző tulajdonságait tükrözik.

Bár a **Road** nem rekord, hanem osztály, ez csak azért van, hogy a tervrajz létrehozásakor az utat és a kiinduló és végződő foglalatait, amik egymás adattagjaiként vannak jelen, össze lehessen kötni két lépésben, ekkor az úthoz hozzá kell rendelni a foglalatokat, miután az előző utasításban a foglalatokhoz hozzárendeltük az utat, így a foglalatok adattagjai nem lehetnek az út példányosításakor véglegesek.

- **isRoundabout** (boolean): az út körforgalom része-e
- **isSlipLane** (boolean): az út összekötő út-e
- **isPriorityRoad** (boolean): az út főútvonal-e
- **routeNumber** (int vagy null): az út száma, ha van
- **laneConfiguration** (List<Lane>): a sávok listája

Az **isRegularRoad** metódus igazzal tér vissza, ha az út sem összekötő út, sem körforgalom része, hanem „közönséges út”, hiszen a körforgalmakra és összekötő utakra különleges táblázási szabályok vonatkoznak (pl. körforgalomban menetiránnyal szemben nem kell elhelyezni „Behajtani tilos” táblát vagy körforgalmon belül számpajzsot, még ha számozott főútvonal része is).

A **laneConfiguration** listában található a sávok listája, mindkét irányban az egyes sávok az irány szempontjából balról jobbra. A **Lane** rekord az úton lévő egy közúti sávot reprezentál, ami három adattaggal rendelkezik:

- **directions**: Set<LaneDirection>: besorolás esetén a sávban a megengedett irányok halmaza.
 - A **LaneDirection** felsorolástípus értékei: FORWARD, LEFT, RIGHT, BACKWARD, REVERSE, SINGLE (ez utóbbi arra az esetre van, amikor az útszakaszon nincs külön besorolás).
- **isForward**: boolean: a sáv iránya az út gráfbeli irányával megegyezik-e.
- **laneType**: LaneType: a sáv típusa.
 - A **LaneType** felsorolástípus értékei: GENERAL (általános sáv), BICYCLE (kerékpársáv), BUS (buszsáv), BUS_BICYCLE (buszsáv megengedett kerékpáros közlekedéssel).

Az út adattagként tárolja a kiinduló és végződő pontfoglalatot:

- **startNodeSlot** (NodeSlot): kiinduló pontfoglatat
- **endNodeSlot** (NodeSlot): végződő pontfoglatat

Mivel a foglatat tárolja magában a pontot (RoadNode), amihez csatlakozik, az út látja a pontjait, amihez csatlakozik.

A jelzőtábla-kompozíciókat a **signCompositions** szótár tartalmazza, aminek kulcsa egy **SignLocation** típusú rekord, erről az 3.2.5.5.2. fejezetben lehet olvasni.

A jelzőtábla-kompozíciók elhelyezésekor a **DESIRED_SHIFT_DISTANCE** (int) konstans meghatározza, hogy ha egy jelzőtáblát már megtelt kompozícióra szeretnének helyezni, a „túlsorduló” táblát tartalmazó második kompozíciót milyen távolságra próbálja elhelyezni az **addSign** metódust.

A **compositionSettings** adattagot az út arra használja, hogy a létrehozott táblakompozícióiba befecskendezze konstruktoron keresztül az **application.properties** fájlban megadható kompozíciós konfigurációkat, azaz

- a kompozíción szereplő jelzőtáblák maximális számát (**template.composition.maxsigns**), ami alapból 3,
- illetve, hogy a kompozíción már egyszer szereplő táblákat elvesse-e hozzáadáskor (**template.composition.discardduplicates**), ami alapból igaz (**true**).

Ezeket a beállításokat azért tárolja magában az út, mert a tervrajz kezdeti gráfját egy bean szolgáltatásosztály hozza létre (**TransformGraphProcessorService**), amibe be lehet fecskendezni a **SignCompositionService**-en keresztül az **application.properties** konfigurációját, és ez a szolgáltatás az utakat létrehozza, de a táblakompozíciók már a modellosztályként (és nem beanként) működnek és modellosztályok (táblázási szabályhalmazok) hozzák létre, amiket nem a Quarkus függőségbefecskendezési rendszere kezel, így konfigurációs tulajdonságot nem lehet **@ConfigProperty** annotációval befecskendezni, mint a beanekben. Röviden az utat (Road) bean példányosítja, de a táblakompozíciót (**SignComposition**) nem, így a Quarkus által kezelt bean-kontextus és a modell határa a gráf (és ezzel együtt az út) létrehozása a **TransformGraphProcessorService**-ben, modellen belül pedig konstruktoron

keresztül injektálhatók a beállítások, így az út `compositionSettings` adattagja adja át az `application.properties` konfigurációját a táblakompozíció modelljének.

3.2.5.5.2 Jelzőtábla-kompozíciók az úton

Az úton egy jelzőtábla-kompozíció (egy kompozíció a **SignComposition** egy példánya) helyét egy, a **SignLocation** rekorddal megvalósított, egy valós számból és igaz-hamis értékből álló rendezett pár határozza meg:

- **distance** (`double`): a kompozíció távolsága az út mentén a ponttól, ahonnan az út indul.
- **isForward** (`boolean`): a táblák az út gráfbeli irányára vonatkoznak-e, tehát ha az `isForward` érték igaz, a táblakompozíció az úton az út kiinduló foglalatának pontja (a `startNodesSlot` pontfoglalathoz tartozó pont) felé van „fordítva”, hamis esetén pedig a végződő pont és foglalat felé néz.

A táblakompozíció létrehozása a **Road** **addSign** metódusával történik, ahol az út kiinduló pontjától az út mentén a `location` **SignLocation** típusú argumentumban megadott távolságra és irányban helyezze el az új tábla modelljét (ami a **RoadSign** osztály egy példánya). Ha ezen a helyen

- még nincs táblakompozíció, a metódus hozzáad egyet az úthoz az adott helyen (tehát a **signs** hely-kompozíció szótárhoz hozzáad egy bejegyzést, melynek kulcsa a hely, értéke az új táblából álló **SignComposition** példány, amely a `maxSigns` és `discardDuplicates` értékeit az útba a beanek által az útba befecskendezett **SignSettings** típusú rekordból veszi), az egyetlen táblája a hozzáadott tábla lesz.
- már van kompozíció és van rajta szabad hely (alapesetben 3 táblát lehet elhelyezni), hozzáadja a tábla precedenciája (a **RoadSign** `precedence` adattagja) alapján megfelelő helyre a táblát a kompozícióhoz. A magasabb precedenciaértékű tábla előbb van a listában, ami azt reprezentálja, hogy feljebb van a kompozíció által képviselt oszlopon. Egyenlő precedencia esetén a sorrend nem lényeges, de a **SignComposition** osztály **addSign** metódusa ekkor a többi azonos precedenciájú tábla fölé helyezi.
- már van kompozíció, de van ilyen tábla ugyanezen kompozíción és a **discardDuplicates** adattag értéke igaz (ezt az értéket az út fecskendezi be

példányosításkor a `template.composition.discardduplicates` konfigurációs érték alapján), akkor a táblakompozíció `addSign` metódusa naplózza a tábla elvetését, és visszatér a tábla hozzáadása nélkül.

- már van táblakompozíció és tele van, akkor megpróbálja a tele lévő táblakompozíció előtt és után `DESIRED_SHIFT_DISTANCE` távolságra (vagy az út elejére/végére, ha az adott irányban már kevesebb a hely az út elejéig/végéig) elhelyezni új táblakompozícióként vagy hozzáadni egy esetleges ott lévő nem teli kompozícióhoz a táblát. Ha ez sem sikerül, az eredeti helyen található kompozícióba helyezi mégis.

3.2.5.5.3 Utak korlátozásai

Az utak rendelkeznek irányonként korlátozásokkal, amiket a szerver OSM-adatok esetén az OSM-út (way) elemzésekor hoz létre az OSM-út címkéi alapján (3.2.5.3.5. fejezet).

A korlátozások a **Restriction** interfészt valósítják meg. A korlátozások:

- Sebességkorlátozás (**SpeedLimit**)
 - **limit** (`int`): az úton megengedett legnagyobb sebesség (km/h-ban).
- Össztömeg-korlátozás (**WeightRestriction**)
 - **maxweight** (`String`): az úton megengedett legnagyobb tömeg (tonnában).
A lebegőpontos számok tárolásának időnkénti pontatlansága miatt stringként van tárolva, hogy hibamentes legyen a reprezentáció az adatátviteli objektumban és egy esetleges táblacímkén, ami ezt az értéket használja (3.2.5.5.9. fejezet).
- Járműtípus-korlátozás (**VehicleRestriction**)
 - **vehicles** (`List<VehicleType>`): a tiltott járműtípusok. A `commons.sharedtypes` csomagban található `VehicleType` felsorolástípus az alábbi értékekkel rendelkezik:
 - HGV, BICYCLE, MOTORCYCLE, CAR, SLOW_VEHICLE, HORSE_CARRIAGE

Az utak két listában tárolják a korlátozásokat:

- **restrictionsForward**: `List<Restriction>`: az út gráfbeli irányával megegyező irányra vonatkozó korlátozások listája.

- **restrictionsBackward**: `List<Restriction>`: az út gráfbeli irányával ellentétes irányra vonatkozó korlátozások listája.

3.2.5.5.4 Utak hossza és foka

Az út **getLength** metódusa visszaadja az út hosszát a kezdődő és végződő foglalathoz tartozó csúcsok pozíciói közt húzott egyenes hosszának kiszámolásával, ami hasznos, ha a tervrajzot távolság alapján járjuk be.

A **getSlope** az út irányát adja vissza fokban, ahol 0 fok azt jelenti, hogy az út gráfbeli iránya szerint jobbra (keletre) néz, és óramutató iránya szerint megy körbe 360 fokig (de 360 nem lehet az érték). Így például az egyenesen észak felé haladó utakra (gráfbeli irány szerint) a **getSlope** hívásakor a visszaadott érték `270.0`.

3.2.5.5.5 Utak gráfbeli és forgalmi iránya

Az utak gráfbeli irányára korábban már többször hivatkoztam, ez, ha a tervrajzot egy irányított gráfnak nézzük, az útnak megfeleltethető él iránya. Ezt az irányt használja a tervrajz például a sávkonfiguráció meghatározására, ahol a **laneConfiguration** `List<Lane>` típusú adattagban egyes (Lane típusú) sávok **isForward** adattagja jelöli, hogy az úton a sáv az út gráfbeli irányába megy, vagy azzal ellentétben. Épp ezért egy út gráfbeli iránya nem az úton áthaladó forgalom irányára utal: a sávkonfiguráció határozza meg a forgalom irányait, így egy úton értelemszerűen lehet a forgalom kétirányú (például kétsávós út egy-egy sávval mindkét irányba: ekkor az út sávkonfigurációja egy kételemű `Lane`-lista, az az egyik **isForward** adattagja igaz értékű, a másiké hamis). Sőt, ha egy út összes sávja az út irányával szemben halad, az úton a forgalom iránya lehet ellentétes az út gráfbeli irányával. A gráfbeli irány tulajdonképpen arra szolgál, hogy legyen mihez viszonyítani a sávok irányát, a korlátozások irányát és táblakompozíciók helyét (ami az út mentén a kiindulóponttól való távolság alapján van meghatározva a gráfbeli irány szerint).

A **getDirection** metódus egy háromelemű felsorolástípus, a **RoadDirection** valamelyik elemét adja vissza:

- **FORWARD**, ha az út forgalma egyirányú az út gráfbeli irányával megegyezően.
- **BACKWARD**, ha az út forgalma egyirányú az út gráfbeli irányával ellentétesen.
- **TWO_WAY**, ha az út forgalma kétirányú.

Az **isOneWayForward**, **isOneWayBackward** és **isAccessibleBothDirections** a **getDirection** értékeit használják fel egy igaz-hamis **boolean** típusú érték visszaadására az egyirányúságra vagy kétirányúságra.

3.2.5.5.6 Út pontjai (**RoadNode**)

A tervrajz gráfjának a csúcsai, a pontok a **RoadNode** osztállyal vannak megvalósítva. Magában a tervrajz osztályában (**RoadTemplate**) a **roadNodes** lista adattagban tárolódnak. A pontok adatstruktúrája egyszerű:

- **id: String**: ez a pont azonosítója, továbbá annak az OSM-pontnak az azonosítója, melyből készült a tervrajzi útpont.
- **nodeSlots: List<NodeSlot>**: a ponthoz csatlakozó foglalatok listája; a pontok a foglalatok **occupier** tagján keresztül így látják a foglalatokon át hozzájuk csatlakozó utakat is.
- **position: Position**: a pont pozíciója a koordinátarendszerben; az utak a kiinduló és végződő pontjaik pozíciója alapján számolják ki hosszukat és irányukat fokban.

Az adattagokat elérő és beállító metódusokon kívül ezek elérhetők:

- **isBranching()**: **boolean**: eldönti, hogy a pont elágazás-e csupán annyival, hogy megnézi, 2-nél több foglalat csatlakozik-e hozzá.
- **isMerge()**: **boolean**: eldönti, hogy a pont fonódási pont-e. Egy pont akkor fonódási pont, ha az alábbi feltételek mindegyike teljesül:
 - A pont három foglalattal rendelkezik.
 - A három foglalat útjai közül a legtöbb sávval rendelkező útnak annyi sávja van, mint a másik kettő foglalathoz csatlakozó utak sávjainak összege (tehát a két út sávjai a harmadik útba „fonódnak össze” úgy, hogy egy sáv sem szűnik meg a pontban).
 - A három út közül legalább egy egyirányú forgalmú.
- **isRoundaboutMerge()**: **boolean**: eldönti, hogy a pont körforgalom csatlakozási pontja-e attól függetlenül, hogy be- vagy lecsatlakozó ágról van szó. Egy pont akkor körforgalom becsatlakozási pontja, ha az alábbi feltételek mindegyike teljesül:
 - A pont három foglalattal rendelkezik.

- A három foglalat közül pontosan kettőnek a csatlakozó útja körforgalom része (`isRoundabout` adattagjának értéke `true`).

Azért rendelkezik a pont ennyire kevés adattaggal és metódussal, mert a tervrajz esetén hatékonyabbnak gondoltam sok információ (pl. címkék) tárolását a foglalatokban, mivel az jelöli az út és a pont közti kapcsolatot. Például a címkék, amelyeket OSM-adatokban a pontok birtokolnak, itt átkerülnek a foglalatokra. Így lehet például útszakasz közbenső, nem elágazást jelölő (tehát 2 foglalattal rendelkező) pont esetén irányt megadni egy címkének (pl. jelzőlámpa esetén a jelzés vonatkozó iránya) azzal, hogy az abban az irányban lévő foglalatra helyeződik a címke, amelyre érvényes, például a következő, 3.2.5.5.7. fejezetben a 32. ábrán látható elsőbbségadási kötelezettségek.

3.2.5.5.7 A foglalatok

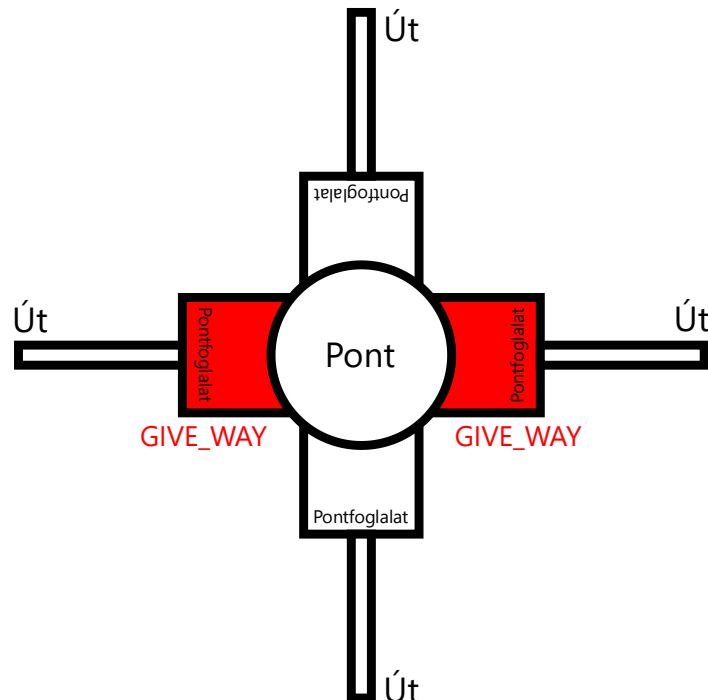
A tervrajzban a pontfoglalatok megvalósítása a **NodeSlot** osztályban található. A tervrajzot gráfként tekintve a foglalat az egyes csúcsok (a tervrajzban pontok) és élek (a tervrajzban utak) közötti kapcsolatokat reprezentálja, melynek tervrajzok esetében saját adattagjai vannak és saját logikával rendelkeznek (ráadásul többel, mint maguk a pontok). A foglalatok, bár bonyolítják a tervrajz gráfként történő megvalósítását, hasznosak tudnak lenni táblázáskor: egy út mentén a foglalat mellett például akkor kell elhelyezni „Sebességkorlátozás” táblát, ha a foglalat

- útja rendelkezik sebességkorlátozással, és
- a pont elágazásban van, nem közbenső pont két út között, vagy közbenső pontja két útnak, de az előző úton a pont felé nem volt még sebességkorlátozás, vagy másik volt érvényben.

A foglalat modellezi az út és pont közti kapcsolatot, és ahogy látható, a „Sebességkorlátozás” tábla elhelyezésére van egy feltétel az útra és a pontra vonatkozóan is, amiknek együtt kell teljesülniük.

De gondolhatunk egy egyszerű, védett és alárendelt utakkal rendelkező kereszteződésre is (elágazás, ami nem egy 3.2.5.5.6. fejezetben definiált fonódási pont), amit egy ponttal és a foglalataival lehet ábrázolni. Az elsőbbségadási kötelezettséget jelölő címkék el vannak helyezve az alárendelt utakat és a pontokat összekötő foglalatokon, így könnyen látható a

keresztvezetés formája a modellben, és egyszerűen az elsőbbségadási kötelezettséggel címkézett foglalat útján, a foglalatnak háttal kell elhelyezni az „Elsőbbségadás kötelező” vagy „ÁLLJ! Elsőbbségadás kötelező” jelzőtáblákat.



32. ábra: egy pontból álló négyágú csomópont sémája, két alárendelt ággal (a pirossal jelölt foglalatok rendelkeznek GIVE_WAY címkével)

A pontfoglalatoknak nincs hosszuk vagy területük a koordinátarendszerben, a pontok és a foglalatok is pontnak számítanak, melynek pozíciója a pont pozíciója, melyhez csatlakoznak.

A `NodeSlot` osztály három adattaggal rendelkezik:

- **occupier**: Road (a foglathoz csatlakozó út)
- **attributes**: Set<NodeSlotAttribute> (a foglalat címkéi)
- **node**: RoadNode (a pont, amihez a foglalat csatlakozik)

A `NodeSlotAttribute` felsorolástípus tartalmazza a pontfoglalatok lehetséges címkéit:

- **TRAFFIC_SIGNALS** (jelzőlámpák a foglalat útja felé)
- **CROSSING** (kijelölt gyalogos-átkelőhely)
- **GIVE_WAY** (elsőbbségadási kötelezettség az út felől érkezőknek)
- **STOP** („ÁLLJ! Elsőbbségadás kötelező” táblával szabályozandó elsőbbségadási kötelezettség az út felől érkezőknek)

A `NodeSlot` bár osztály, rekordként működik az adattagok elérését illetően (az adattaggal megegyező metódus meghívásával szerezzük meg az adattag értékét), azzal a különbséggel, hogy az `occupier` adattagot, a csatlakozó utat, be lehet utólag állítani a `setOccupier` metódussal, hogy beállítható legyen az út és foglalat közti kapcsolat, ahol mindkettő tartalmazza a másikat a tervrajz létrehozásakor.

Az alábbi nyilvános metódusok elérhetők az osztályban, melyeket a tervrajz feldolgozásakor használni lehet:

- **`isStart(): boolean`**
 - Igaz, ha a foglalat pontja az út kiinduló pontja.
- **`isEnd(): boolean`**
 - Igaz, ha a foglalat pontja az út végződő pontja.
- **`isOnewayOpposite(): boolean`**
 - Igaz, ha a foglalat útja egyirányú forgalmú, a foglalat felől nem mehet rá forgalom.
- **`isOnewayCorresponding(): boolean`**
 - Igaz, ha a foglalat útja egyirányú forgalmú, csak a foglalat felől mehet rá forgalom.
- **`isOneway(): boolean`**
 - Igaz, ha a foglalat útja egyirányú valamerre.
- **`getDistanceFromNode(distance: double): double`**
 - Egy, az út mentén lévő pontot lehet jellemezni azzal, hogy milyen távolságra van a kiinduló foglalattól/ponttól. a `getDistanceFromNode` visszaadja azt az értéket, amivel lehet jellemezni az út mentén a foglalattól `distance` méterre lévő távolságot. Ha a foglalat pontja az út kiindulópontja, akkor a visszatérési érték maga a `distance`, ha a végződő pontja, akkor az út hossza mínusz `distance`, tehát `distance` távolság az út végétől számítva. Ha az argumentum nagyobb, mint az út hossza, akkor `IllegalArgumentException`-t dob a metódus, így a metódus meghívása előtt célszerű tesztelni, hogy a paraméterként átadni szánt szám kisebb-e, mint a foglalat útjának hossza (például `slot` nevű pontfoglalat esetén egy

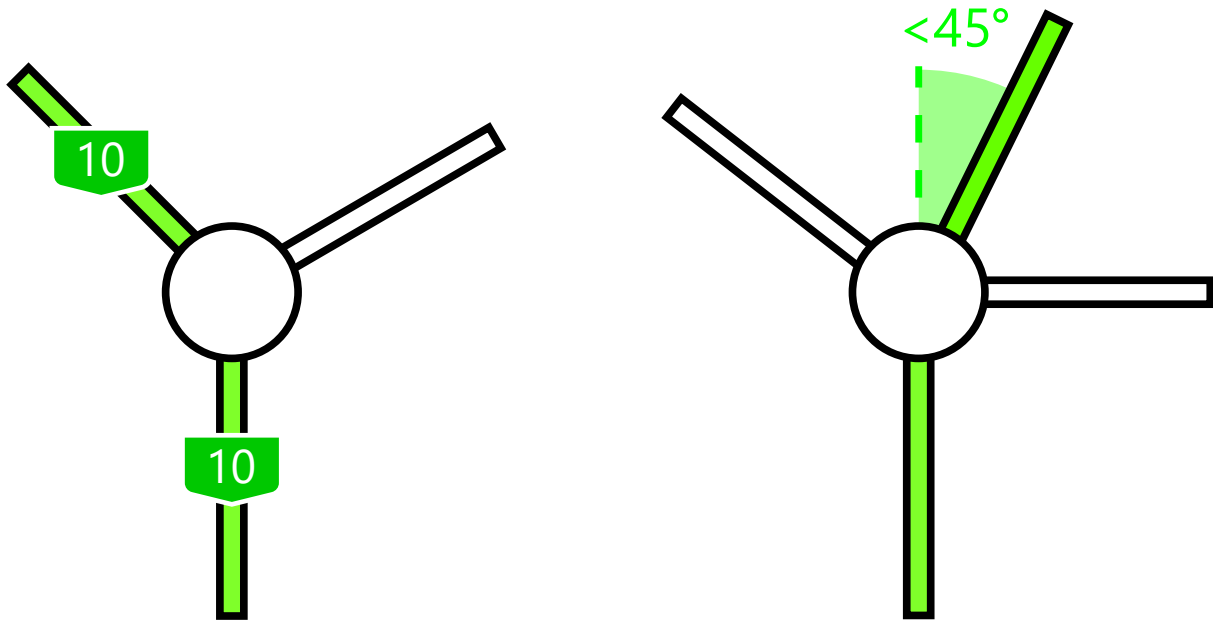
`slot.occupier().getLength() >= distance` feltétel mögé tenni a metódushívást).

- **`getContinuation(): Optional<NodeSlot>`**

- Visszaadja egy `java.util.Optional` értékbe csomagolva a foglalat szemszögéből útjának folytatását tartalmazó, ugyanahhoz a ponthoz csatlakozó foglalatot, ha van ilyen. Legyen A út, B út, N pont, ASN egy A és N közti foglalat, BSN egy B és N közti foglalat. Látható, hogy A és B utak ekkor szomszédosak N-en keresztül. Ekkor B út A folytatása, ha valamelyik feltétel teljesül ezek közül:

- B útnak van száma és ugyanaz, mint A száma, és csak A-nak és B-nek van ilyen száma az N-hez csatlakozó foglalatok útjai közt.
 - Ez néha ad hamisan igaz eredményt, pl. ahol a számozott út alárendelt, és a számozás a védett úton megy tovább (103-as út Bajnánál jó példa erre).
- B út irányának fokának és A út irányának fokának a különbségének legfeljebb 45 fok, vagy $|180 - [A \text{ és } B \text{ fokának különbsége}]|$ legfeljebb 45 fok, hogy a gráfbeli irány ne számítson. A cél az, hogy topológiailag egyenes vagy egyeneshez elég közeli legyen a két útból alkotott szakasz.

Ebből következik, hogy ha A út B folytatása, akkor B út A folytatása, ez a reláció szimmetrikus.



33. ábra: heurisztikák út folytatásának meghatározására a tervrajzon: útszám szerint és topológia szerint

- **getConnectedSlot(): NodeSlot**
 - A foglalat útjának ellentétes végén szereplő foglalat.
- **getOtherSlotsOfNode(): List<NodeSlot>**
 - A foglalat pontjához tartozó többi pontfoglalat.
- **addSign(boolean againstSlot, double distance, RoadSign sign)**
 - A foglalat útja mentén, a foglalattól *distance* méterre, de legfeljebb az út végére elhelyez egy jelzőtáblát. Ha *againstSlot* hamis, akkor a foglalat felé fog nézni az új tábla, ha igaz, akkor a foglalatnak háttal lesz. A metódus a foglalat *getDistanceFromNode* metódusát használja az út *addSign* metódusának meghívása előtt a hely megtalálásához.
- **getAccessibleSlots(): List<NodeSlot>**
 - Azok a foglalat pontjához csatlakozó pontfoglalatok (magán a metódust futtató foglalaton kívül), melyek a forgalom számára elérhetők a pontból (tehát nem egyirányú forgalmú utak a pont felé).
- **isInBranching(): boolean**
 - Igaz, ha a foglalat pontja elágazás (legalább 3 foglalata van).
- **isInIntersection(): boolean**

- Igaz, ha a foglalat pontja kereszteződés (elágazás, de nem fonódási pont, sem körforgalom csatlakozási pontja).
- **isInMerge(): boolean**
 - Igaz, ha a foglalat fonódási pont része.
- **isNodeIntermediate(): boolean**
 - Igaz, ha a foglalat közbenső pont része (tehát csatlakozó útjainak száma 2).
- **getSpeedLimit(): Optional<SpeedLimit>**
 - A foglalat útjának sebességhatárítása `Optional` objektumba, ha van, egyébként `Optional.empty()`.
- **getWeightRestriction(): Optional<WeightRestriction>**
 - A foglalat útjának össztömeg-korlátozása `Optional` objektumba, ha van, egyébként `Optional.empty()`.
- **hasRestriction(restriction: Restriction): boolean**
 - Igaz, ha a foglalat útjának van a `restriction` argumentummal egyenlő korlátozása.
- **getIntersectionCenter(): Optional<NodeSlot>**
 - Keres a 3 pontos környezetében egy kereszteződést, nem az útja, hanem a pontja és az azzal szomszédos többi pont felé indulva.
- **isRoundaboutMerge(): boolean**
 - Igaz, ha a foglalat pontja körforgalom csatlakozásában található.
- **isSolitary(): boolean**
 - Igaz, ha a metódust futtató foglalat a pontjának egyetlen foglalata.
- **isRegularRoad(): boolean**
 - Igaz, ha a foglalat útjának `isRegularRoad()` metódusa is igazat ad vissza (3.2.5.5.1. fejezet).
- **getIncomingLanes(): List<Lane>**
 - A foglalat útjának a foglalat felé haladó sávjai, balról jobbra rendezve.
- **getOutgoingLanes(): List<Lane>**
 - A foglalat útjának a foglaltól haladó sávjai, balról jobbra rendezve.
- **hasOutgoingSpecialUseLane(): boolean**

- A foglalat útján a foglalattól indulva megy legalább egy nem általános típusú sáv (kerékpársáv, buszsáv vagy buszsáv engedélyezett kerékpáros forgalommal).
- **hasBranchingWithin(distance: double, reverse: boolean)**
 - Igaz, ha a foglalattól indulva hamis `reverse` argumentum esetén az út felé, igaz `reverse` argumentum esetén a foglalat pontjának többi útja felé van `distance` távolságon belül (az utak hosszát mérve) elágazás.
- **getSlotsStraight(): List<NodeSlot>**
 - A foglalat pontjának azon foglalatai, amelyek útjai a metódust végrehajtó foglalat útjához képest egyenesen indulnak a pontból ($\pm 32,5^\circ$).
- **getSlotsLeft(): List<NodeSlot>**
 - A foglalat pontjának azon foglalatai, amelyek útjai a metódust végrehajtó foglalat útjához képest balra indulnak a pontból ($\pm 32,5^\circ$).
- **getSlotsRight(): List<NodeSlot>**
 - A foglalat pontjának azon foglalatai, amelyek útjai a metódust végrehajtó foglalat útjához képest jobbra indulnak a pontból ($\pm 32,5^\circ$).
- **seekBranchingByNodeCount(depth: int, reverse: boolean): Optional<NodeSlot>**
 - A foglalattól indulva hamis `reverse` argumentum esetén az út felé, igaz `reverse` argumentum esetén a foglalat pontjának többi útja felé legfeljebb `depth` pont távolságra elágazást keres.
- **seekBranchingByDistance(distance: double, reverse: boolean): Optional<NodeSlot>**
 - A foglalattól indulva hamis `reverse` argumentum esetén az út felé, igaz `reverse` argumentum esetén a foglalat pontjának többi útja felé legfeljebb `distance` távolságra (az utak hosszát mérve) elágazást keres.
- **containsPriorityRegulation(): boolean**
 - Igaz, ha a foglalat tartalmaz STOP vagy GIVE_WAY címkét (tehát valamilyen elsőbbségadási kötelezettséget).
- **isSignalized(): boolean**
 - Igaz, ha a foglalat jelzőlámpát tartalmaz az út felé (tehát TRAFFIC_SIGNALS címkét tartalmaz).

3.2.5.5.8 A jelzőtáblák

A jelzőtáblákat a **RoadSign** osztály valósítja meg. Ez az osztály a táblaadatokkal nem rendelkező, csupán egy kódból azonosítható táblákat modellezi, a testre szabható táblák a **RoadSign** leszármazottjai, ahol az adatok plusz adattagként jelennek meg.

A konstruktoron át mindenképpen meg kell adni a tábla kódját, az egyik csak kódot kér, a másik kódot és egy egész számot, mely a tábla precedenciája (minél nagyobb, annál magasabb helyen lesz egy jelzőtábla-kompozícióban). Ha a csak kódot argumentumként fogadó konstruktorral jön létre a jelzőtábla, automatikusan kap egy precedenciát a kódja alapján (a konstruktor a tábla kategóriáját próbálja kód alapján kideríteni, a „B” kezdetű kódok elsőbbséget szabályzó táblákhoz tartoznak, így nagyon magas a precedenciájuk, mivel magasabbra szokás helyezni őket az oszlopon, de az „A”-val kezdődő veszélyt jelző táblák is magasabban vannak, mint a „C”-vel jelzett tiltó és korlátozó táblák, stb.).

Adattagok:

- **code:** `String`: a tábla kódja.
- **precedence:** `int`: a tábla precedenciája.

A precedencia csak a szerver tervrajzkomponensének modelljében van használva, a tábla adatátviteli objektummá konvertálásakor már nem marad a jelzőtáblában, így a válaszként küldött (és a grafika elkészítésekor a kérésben elvárt) JSON-objektumban nem szerepel, a kód azonban igen, sőt a prezentációs komponens alapvetően az alapján határozza meg a jelzőtábla típusát. Épp ezért fontos, hogy olyan kódokat kapjanak a jelzőtáblák, amiket a prezentációs komponens támogat, tehát van hozzá kapcsolódó név és jelkép.

A jelzőtáblákat a tervrajzon elkészítést követően a táblázó program helyezi el (3.2.5.6. fejezet).

A táblák megfelelő típusú adatátviteli objektummá alakítása érdekében implementálják (mind a szülőosztály, mind a leszármazottjai) a látogató tervezési mintát, így a **RoadSign** osztály és mindegyik altípusa a **visit** metóduson keresztül pontosan elérhető (tehát egy táblalistában altípus esetén nemcsak **RoadSign**ként) egy, a **RoadSignVisitor** interfészt megvalósító osztályból (ez az `api.dto` csomagban a `SignDtoMapper`).

3.2.5.5.9 Testre szabható jelzőtáblák típusai

A testre szabható jelzőtáblák osztályai kiterjesztik a **RoadSign** osztályt, de mivel egy táblatípust valósítanak meg egyedi adatokkal, csak az adatokat várják el konstruktoraikban, amikben az adott táblatípus kódjával (vagy kóddal és precedenciával) hívják meg az őssztály konstruktorát. A táblák osztályainak kapcsolatát a 34. ábra: a jelzőtáblák modelljeinek osztálydiagramja^{34. ábra szemlélteti.}

- **LaneArrangementSign:**

- Sávbesorolás rendjét jelző tábla egyedi sávelrendezéssel
- Kód: „E-SAVBESOR”⁸
- Adatok:
 - **lanes:** **List<SignLane>**
- A **SignLane** osztály egy táblán lévő sávot reprezentál, adattagok:
 - **laneDirections:** **Set<LaneDirection>**: a sáv megengedett irányai
 - **type:** **LaneType**: a sáv típusa (**GENERAL**: általános, **BICYCLE**: kerékpársáv, **BUS**: buszsáv, **BUS_BICYCLE**: buszsáv megengedett kerékpáros közlekedéssel)

- **TrafficSignalSet:**

- Jelzőlámpák csoportja
- Kód: „JL”
- Adatok:
 - **signals:** **List<TrafficSignal>** (a jelzőlámpák balról jobbra)
- A **TrafficSignal** felsorolástípus, mely a jelzőlámpa fényjelzőjében szereplő ikont határozza meg, ami lehet kör alakú jelzés/telizöld (**FULL** esetén), kerékpáros jelzés (**BICYCLE** esetén) vagy valamilyen nyíl alakú jelzés

⁸ A vonatkozó útügyi műszaki előírásban [24] a különböző típusú sávbesorolást jelző tábláknak több különböző kódja van, így a tábla az adatai alapján mindig más-más kódnak felelt volna meg. Végül ezért lett E-SAVBESOR (E, mert az összes sávbesorolás rendjét jelző tábla „E” kategóriájú (különleges szabályt jelző) tábla).

(többi érték esetén). Az osztályt a prezentációs komponens is használja, így a `commons.sharedtypes` csomagban található. A teljes lista:

- `FULL, STRAIGHT, LEFT, RIGHT, LEFT_RIGHT, STRAIGHT_LEFT, STRAIGHT_RIGHT, REVERSE, BICYCLE`

- A konstruktor használata mellett a `fromLaneConfiguration(laneConfiguration: List<Lane>)` statikus metódussal készíthető egy jelzőlámpacsoport a lámpával ellátandó sávok alapján.

- **ShieldSign:**

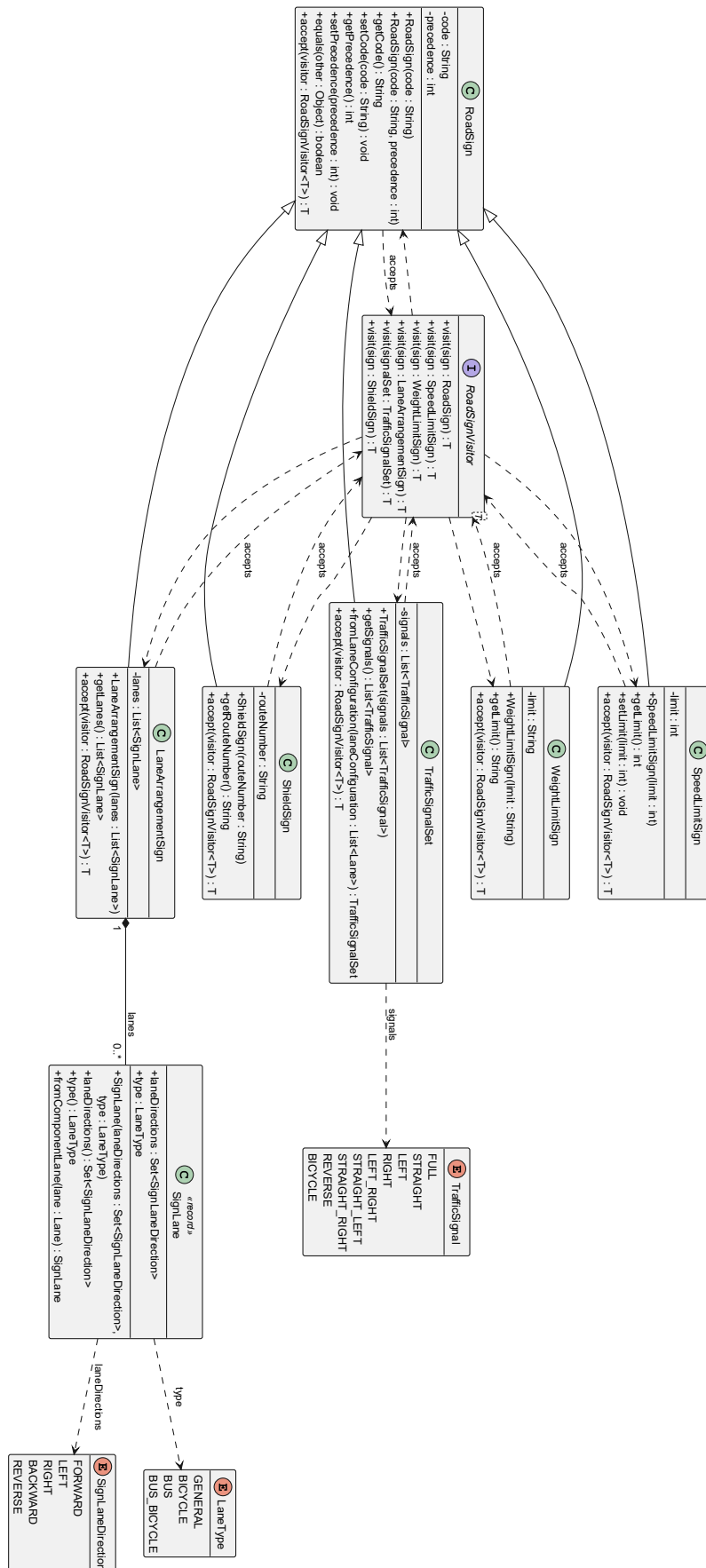
- Számozott út táblája
- Kód: „G-152” [25]
- Precedencia: 49 (hogy közvetlen az 50-es precedenciájú „Főútvonal” tábla alatt legyen)
- Adatok:
 - `routeNumber: String` (a táblán feltüntetendő útszám)

- **SpeedLimitSign:**

- „Sebességkorlátozás” tábla
- Kód: „C-033” [22]
- Adatok:
 - `limit: int` (tábla címkéje, egész szám kell legyen sebességhatár feltüntetése esetén)

- **WeightLimitSign:**

- „Össztömeg-korlátozás” tábla
- Kód: „C-023” [22]
- Adatok:
 - `limit: String` (tábla címkéje, egész vagy tört szám, a lebegőpontos számokból eredő potenciális hibák elkerülése miatt `String` típusú, nem `double` vagy `float`)



3.2.5.6. A táblázó program osztályai és működése

3.2.5.6.1 Táblázási szabályok

Egy jelzőtábla elhelyezésének szabályait a **SignageRule** osztály valósítja meg.

Az alábbi adattagokkal rendelkezik, példányosításnál mindet meg kell adni:

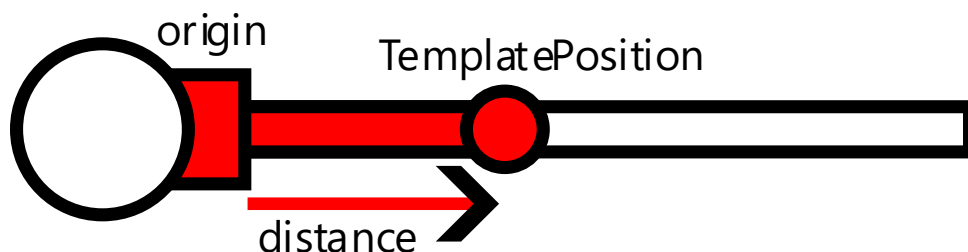
- **trigger**: Predicate<NodeSlot>
 - A tábla elhelyezésének szükségét kiváltó, egy pontfoglalatra alkalmazható predikátum. Ha a `trigger.test(slot)` metódus visszatérési értéke igaz egy `slot` foglalatra, akkor `slot` aktiválja a `trigger`-t.
- **placer**: Consumer<TemplatePosition>
 - A táblának a pozíciókereső által talált helyen történő elhelyezését elvégző fogyasztó függvény.
- **distance**: double
 - A távolság, amit a pozíciókereső használ a tábla elhelyezésének szükségét kiváltó (a `trigger`-t aktiváló) pontfoglalatától indulva az elhelyezés pozícióinak keresésére, amiket a `placer` adattag használhat.
- **seeker**: PositionSeeker
 - A pozíciókereső, mely a `trigger`-t aktiváló foglalatától a foglalat útjának irányába megkeresi az elhelyezési pontot vagy pontokat az előző argumentumban megadott távolságra.

A táblázási szabállyal a tervrajz pontfogladatait lehet vizsgálni, hogy szükséges-e egy adott helyre a szabály által lehelyezhető tábla. Ennek a vizsgálatnak azért a pontfoglalat az „alapegysége”, mert az modellezi az út és a pont kapcsolatát, így sok információt lehet hatékonyan lekérdezni a tervrajz ottani kontextusáról, az út, a pont és a környező utak viszonyáról (3.2.5.5.7. fejezet). Ha egy pontfoglalatra a `trigger` igazat ad vissza, a `seeker` kereső a `trigger`-t aktiváló foglalatától az út irányába elkezd keresni megfelelő helyeket a `distance` mezőben tárolt távolságra. Ezeket a helyeket egy listában adja vissza. A lista lehet üres is, ekkor nincs megfelelő hely a tábla elhelyezésére, a `placer` pedig nem aktiválódik. Ha egy foglalat közvetlen maga mögé igényel jelzőtáblát (pl. „Kijelölt gyalogos-átkelőhely” az átkelő mellett), akkor érdemes egy, az út folytatását követő pozíciókeresőt alkalmazni egy nagyon alacsony távolsággal (a program szabályszoftvertől az implementációjában ez az

érték 5, a „Kijelölt gyalogos-átkelőhely” esetén pedig 2, így a tábla helyesen 2 méterre van az átkelő pontjától, (JETSZ [1] 18.9. a) pontja) vagy az „Elsőbbségadás kötelező” 5 méterre az elsőbbségadási kötelezettség pontjától, ami előtt el kell helyezni (JETSZ 11.3. ac) pontja)).

A kereső `TemplatePosition` formájában visszaadta a talált helyeket, amiket a rekord a 35. ábra által mutatott módon határoz meg. Adattagjai:

- **origin**: `NodeSlot`: a legutolsó kereső által érintett foglalat, melynek útján ettől a foglalatától `distance` távolságra van a tervrajzon a pozíció.
- **distance**: `double`: a pozíció távolsága az `origin` pontfoglalattól az út mentén.



35. ábra: a `TemplatePosition` szemléltetése

Ezeket a pozíciókat pedig a `Consumer<TemplatePosition>` típusú letevő, a `placer` adattag használja fel a tábla elhelyezésére. Azért nem hívja meg az osztály egy példánya egyszerűen a pozícióban talált `origin` foglalat `addSign` metódusát, mert így elhelyezés előtt további vizsgálatokat lehet végezni a pozíció környezetében, és a testre szabható jelzőtáblák adatainak elkészítéséhez szükség van a környezet vizsgálatára (pl. `origin` foglalatból induló sávok elemzése a sávbesorolás rendjét jelző tábla elkészítéséhez).

3.2.5.6.2 A pozíciókereső

A tervrajz pozíciókeresőjét a **`PositionSeeker`** osztály valósítja meg. A pozíciókereső egy adott foglalatból indulva, metódushívás paraméterétől függően a pontjának vagy az útjának irányába indulva bejárja a tervrajz gráfját egy bizonyos távolságig, egy pontból a látogatandó foglalatokat az osztály példányosításakor meghatározott feltételek alapján kiválasztva.

Egy pozíciókereső létrehozásakor számos paramétert be lehet állítani, de gyakran ezeknek csak egy részére van szükség, a többinél pedig elfogadható valamilyen általános alapérték is. A **`PositionSeekerBuilder`** osztállyal építhető egy pozíciókereső úgy, hogy a metódusainak hívásával beállított paraméterek egyediek legyenek, a többinél pedig

alapértéket vegyen fel. A `PositionSeekerBuilder` egy példányán az alábbi metódusokkal tudjuk beállítani a paramétereket, melyek a `PositionSeeker` adattagjai:

- **`slotSelector(slotSelector: Function<NodeSlot, List<NodeSlot>>)`**
 - A `slotSelector` adattagot állítja be.
 - Ha a kereső elér egy pontfoglalathoz, ez alapján választja ki a következő foglalatokat, amiknek az útjain továbbmegy (kivéve, ha a foglalat pontja már látogatva volt).
 - Alapérték: a foglalat **`getOthersSlotsOfNode`** metódusa, ami a foglalat pontjának a többi foglalatát adja vissza (tehát a kereső a foglalathoz tartozó ponttól minden másik irányban folytatja a bejárást).
- **`stopAndAccept(stopAndAccept: Predicate<NodeSlot>)`**
 - A `stopAndAccept` adattagot állítja be.
 - Ha a kereső egy pontfoglalatra ér, amire a `stopAndAccept` predikátum igaz, a keresés az adott irányban leáll, és a foglalat útján **`stopAndAcceptDistance`** távolságra lévő pont bekerül a talált pozíciók listájába.
 - Alapérték: konstans hamis függvény, nincs olyan feltétel, amivel a kereső egy ágon megáll és elfogad egy pontot.
- **`stopAndAcceptDistance(stopAndAcceptDistance: double)`**
 - A **`stopAndAcceptDistance`** adattagot állítja be.
 - Ha a kereső a `stopAndAccept`-tel megáll egy foglalatnál, az útján a foglalatától **`distance`** távolságra, de legfeljebb az út túlsó végén lévő pozíciót adja a talált helyek listájához.
 - Alapérték: 5
- **`maintainDirection(maintainDirection: boolean)`**
 - A `maintainDirection` adattagot állítja be.
 - Csak akkor releváns, ha a `stopAndAccept` is be van állítva. Ha igaz, akkor a `stopAndAccept` adattagban tárolt predikátumot csak olyan esetben ellenőrzi, ha a pontfoglathoz érkezés iránya egyenlő a keresés indításának irányával. Ha egy foglalatától indulva út felé keresünk, akkor egy pont

foglalatainak vizsgálatakor, az út bejárása előtt hajtódik végre a `stopAndAccept` predikátum ellenőrzése; ha pedig pont felé kezdődik a keresés, akkor egy út bejárása után a ponthoz érkezve, az utat és pontot összekötő foglalatot van ellenőrzés. Ha hamis, akkor minden bejárt pont-út kapcsolat foglalatán végbemegy a `stopAndAccept` predikátum ellenőrzése.

- Alapérték: hamis
- **`acceptAtDistanceEnd(acceptAtDistanceEnd: boolean)`**
 - Az `acceptAtDistanceEnd` adattagot állítja be.
 - Ha igaz, a bejáráskor `distance` távolságra lévő pozíciókat elfogadja a kereső. Ha hamis, akkor csak a `stopAndAccept` predikátummal tud a kereső helyeket találni, ha a következő elemzendő foglalat messzebb van, mint a hátralévő keresési távolság, az ág végéig szakad és nem adódik a talált helyek listájához pozíció. Éppen ezért, ha a `stopAndAccept` alapérték (tehát konstans hamis függvény) és az `acceptAtDistanceEnd` hamis, a pozíciókereső soha semmilyen helyet nem fog találni.
 - Alapérték: igaz

A `PositionSeeker` egy példányával keresést a két nyilvános metódus egyikével lehet indítani:

- **`seekSignPositions(distance: double, startSlot: NodeSlot, towardsRoad: boolean)`**
 - A kereső paraméterei alapján helyeket keres a `startSlot` foglaltól indulva `distance` távolságra (vagy amíg a `stopAndAccept` predikátum nem igaz). Ha a `towardsRoad` igaz, a foglaltól az út felé, ellenkező esetben a csatlakozó pont felé kezdődik a bejárás; ez befolyásolja azt is, hogy hol kerül ellenőrzésre a `stopAndAccept` predikátum, ha a `maintainDirection` adattag igaz.
- **`seekSignPositions(distance: double, startSlot: NodeSlot, towardsRoad: boolean, skipStartNode: boolean)`**
 - Ha a `skipStartNode` hamis, ugyanazt teszi, mint a három argumentumos változat. Ha igaz, akkor

- igaz `towardsRoad` érték esetén annyi a különbség, hogy a kezdő foglalatra nem tesztel a `stopAndAccept`, és ha a `distance` értéke kisebb, mint az út, amin elindul a keresés (szóval nem ér el a szomszédos pontig), a visszatérési érték egy üres lista, hiszen a kiindulási pont helyválasztás tekintetében „át van ugorva”, a következőt pedig nem éri el a bejárás,
- hamis `towardsRoad` esetén pedig ugyanígy jár el, csak a kezdő foglalat pontjánál kiválasztja a foglalatokat, és azoknál nem tesztel a `stopAndAccept` predikátumra a keresés.
- Igaz `skipStartNode` esetén nem kerül bele induláskor a kiinduló foglalat pontja a látogatott pontok listájába.
- A kezdési pont átugrása akkor lehet célszerű, ha semmiképp sem a kiinduló foglalat pontjának valamelyik útján szeretnénk helyet találni, vagy ha tudjuk, hogy a kiindulási foglalat vagy a kiindulási pont valamelyik foglalata biztosan beteljesíti a `stopAndAccept` predikátumot, és nem szeretnénk, hogy egy, csak a kiindulási helyet tartalmazó listával térjen vissza a keresés.

A keresés rekurzívan történik, a `seekSignPositions` privát változatának meghívásával, ami a nyilvános metódusok argumentumai mellett (kivéve a `skipStartNode`-ot) tartalmazza a látogatott pontok listáját. Ha bejáráskor a kereső elér egy olyan foglalathoz, aminek a pontja benne van a listában, a bejárás azon az ágon megszakad. Ha egy ágon a bejárásnak vége, mert

- a foglalat útjának hossza kisebb, mint a hátralévő távolság,
- vagy a `stopAndAccept` predikátum ellenőrzéskor igaz volt,

akkor a metódus visszatér egy egyelemű `TemplatePosition` példányt tartalmazó listával, ami a talált pozíciót tartalmazza (vagy üres listával, ha távolság alapján ér véget az ág és a kereső `acceptAtDistanceEnd` mezője hamis), ami egy pontnál a további kiválasztott foglalatokon indított bejárások eredményével összevonódik.

A `roadtemplate.signalization.utils.PositionSeekers` osztályban található néhány pozíciókereső nyilvános adattagként, amik akár több jelzőtábla elhelyezésénél is hasznosak lehetnek.

- **INTERSECTION_SIGNALISATION_SEEKER:** egy útszakaszon megkeresi a kereszteződéshez tartozó, és a kereszteződés felé érkező forgalomra vonatkozó **GIVE_WAY**, **STOP** vagy **TRAFFIC_SIGNALS** címkével rendelkező foglalatot. Hasznos a kereszteződés viszonyainak felderítéséhez, ha egy kereszteződésre vonatkozó jelzőlámpa/elsőbbségadási kötelezettség nem a kereszteződés pontjának a foglalatában, hanem kicsit messzebb van címkézve.
- **ALONG_ROAD_SEEKER:** út mentén haladó kereső (a foglalat `getContinuation` metódusa alapján), magányos pontra érve hozzáadja az út végét és leáll.
- **INTERSECTION_WITH_PRIORITY_SEEKER:** egy kereszteződés foglalatából kiindítva célszerű alkalmazni, visszaad egy helyet a metódushíváskor adott távolságban, ha a foglalat útján elindulva a távolságon belül nem talál még egy elágazást vagy a kiinduló foglalat felé érkező forgalomra vonatkozó jelzőlámpát vagy elsőbbségadási kötelezettséget.

3.2.5.6.3 Szabályhalmazok

Egy szabályhalmaz táblák elhelyezési szabályainak halmaza, ami képes magát alkalmazni egy tervrajzra annak tulajdonságait elemelve.

A tervrajzokra alkalmazható szabályhalmazokat a **RuleSet** interfész absztrahálja, amelynek egyetlen metódusa van, az **applySignage(template: RoadTemplate)**. Az implementációkban a metódus mellékhatásaként kell a `template` argumentumként átadott tervrajzon elhelyezni az implementáció alapján a jelzőtáblákat. Az interfészt implementálva lehet további fejlesztési lehetőségként új szabályhalmazokat készíteni, melyeket alkalmazni lehet saját algoritmus alapján a tervrajzokra.

A **RuleSet** egyetlen implementációja, a **RuleSetImpl** egyetlen konstruktora kettő argumentumot vár:

- **urbanRules:** `List<SignageRule>` (lakott területen belülrre vonatkozó táblázási szabályok listája)
- **ruralRules:** `List<SignageRule>` (lakott területen kívülrre vonatkozó táblázási szabályok listája)

Az **applyTemplate** metódus alkalmazása az argumentumként átadott tervrajzon egyesével végigmegy az összes pont összes foglatán, és attól függően, hogy a tervrajz lakott területen belüli-e (amit a **RoadTemplate** osztály **isTemplateUrban** adattagja határoz meg), alkalmazza rá sorban a lakott területen belüli vagy lakott területen kívüli szabályok (3.2.5.6.1. fejezet) listáját.

3.2.5.6.4 Szabályhalmaz-szolgáltatók

Egy szabályhalmaz-szolgáltató olyan osztály, mely képes elkészíteni egy szabályhalmazt. A **RuleSet** osztály és alosztályai a modell része, tervrajz elkészítésekor a tervrajz modelljét is kezelő osztálynak (a szerverhez érkező kérések esetén ez a **MapToTemplateConversionService**) a **RuleSetProvider** interfész egy implementációját kell befecskendezni (aminek egyetlen implementálandó metódusa van, a **getRuleSet()**, aminek vissza kell adnia egy **RuleSet**-implementációt). Az interfész egyetlen implementációja a **RuleSetProviderImpl** alkalmazás hatókörű bean, ami egy **RuleSetImpl** példányt konstruál a konkrét táblázási szabályokat tartalmazó listákból.

A **RuleSetProviderImpl** a **generateRules(isContextUrban: boolean)** metódussal készíti el a **RuleSetImpl** elkészítéséhez szükséges lakott területen belüli (21 szabályból álló) és lakott területen kívüli (21 + 3 csak lakott területen kívül alkalmazott szabályból álló) listát. A lista elkészítéséhez az osztály befecskendezi és a metódus felhasználja az **application.properties** fájlban beállítható paramétereket a táblák elhelyezését kiváltó ponttól való elhelyezési távolság beállítására. Az alábbi attribútumok módosítják a szabályhalmaz szabályait:

- **signalization.warning.distance.urban**: veszélyt jelző táblák távolsága a táblázást kiváltó helytől lakott területen belül (alapérték: 75, JETSZ [1] 8.1.1. pontja alapján)
- **signalization.warning.distance.rural**: veszélyt jelző táblák távolsága a táblázást kiváltó helytől lakott területen kívül (alapérték: 200, JETSZ [1] 8.1.1. pontja alapján)
- **signalization.priorityroad.distance**: „Főútvonal” tábla (és számpajzs táblája) elhelyezési távolsága kereszteződés után (alapérték: 15, sem a JETSZ, sem a vonatkozó útügyi műszaki előírás nem ír külön értéket, szóval én választottam valami

megfelelőt: a döntés azért esett erre, mert a valóságban is gyakran helyezik a „Főútvonal” táblát kicsit messzebb a kereszteződéstől, és így a tervrajz különböző ábrázolásain sem tömörül a többi, kereszteződéshez közelebb, ezzel növelve az átláthatóságot, de nem feláldozva a szabályosságot).

- **signalization.default.distance:** jelzőtáblák távolsága a táblázást kiváltó helytől alapesetben (ha a kiváltó foglalatnál kell közvetlenül elhelyezni a jelzőtáblát) (alapérték: 5, hogy keresztezésekénél, amikor ez a távolság leggyakrabban van használva, ne teljesen a metszéspontban legyen elhelyezve a jelzőtábla, ahogy a valóságban sem lehet pontban az utak találkozásának metszéspontjában elhelyezni egy jelzőtáblát).
- **signalization.crossing.distance:** „Kijelölt gyalogos-átkelőhely” jelzőtábla elhelyezési távolsága az átkelővel címkézett foglaltól (alapérték: 2, JETSZ [1] 18.9. a) pontja alapján)

Az egyes táblákra vonatkozó pontos szabályokat a `RuleSetProviderImpl` `generateRules` metódusának törzse tartalmazza. A többször használt pozíciókeresőket és triggereket a `PositionSeekers` és `TriggerCatalog` osztályokból veszi át. A „Kötelező haladási irány” jelzőtáblák elhelyezésének triggereit az **obligatoryTurnTriggerCatalog** osztály tárolja.

A `RuleSetProviderImpl` által szolgáltatott szabályhalmaz az alábbi jelzőtáblák modelljeinek elhelyezésére ad meg szabályokat:

- Gyalogosátkelés (A-020)
- Útkereszteződés alárendelt úttal (balra és jobbra) (A-028)⁹
- Útkereszteződés alárendelt úttal (balra) (A-029)⁹
- Útkereszteződés alárendelt úttal (jobbra) (A-030)⁹
- Körforgalom előjelzése (A-056)
- Elsőbbségadás kötelező (B-001)

⁹ Csak lakott területen kívüli tervrajzokra alkalmazza a szabályhalmaz

- ÁLLJ! Elsőbbségadás kötelező (B-002)
- Főútvonal (B-003)
- Főútvonal vége (B-004)
- Behajtani tilos (C-001)
- Össztömeg-korlátozás (C-023)
- Sebességkorlátozás (C-033)
- Kötelező haladási irány (egyenesen) (D-001)
- Kötelező haladási irány (balra) (D-002)
- Kötelező haladási irány (jobbra) (D-003)
- Kötelező haladási irány (egyenesen és balra) (D-004)
- Kötelező haladási irány (egyenesen és jobbra) (D-005)
- Kötelező haladási irány (balra és jobbra) (D-010)
- Körforgalom (D-017)
- Egyirányú forgalmú út (E-012)
- Kijelölt gyalogos-átkelőhely (E-038)
- Sávbesorolás rendjét jelző tábla (E-SAVBESOR)
- Számozott út azonosító táblája (G-152)
- Fényjelző készülék(ek) (JL)

A szabályhalmaz nem tökéletes, számos helyzettel nem számol (pl. autópályák és gyorsforgalmi utak sajátos helyzetének kezelése), de ad egy elégséges alapot arra, hogy a tervrajz modelljének és a táblázó programnak a táblázásra való képességét demonstrálja. A meglévő szabályrendszer mintájára a `RuleSetProviderImpl` által adott szabályrendszer kiegészíthető számos új táblával, a meglévő szabályok finomíthatók, de akár új szabályszoftverek is írhatók például más országok táblázási szabályrendszerének megvalósítására.

3.2.6. Prezentációs komponens

A prezentációs komponens osztályai a **presentation** csomagban találhatók.

3.2.6.1. A *SignController* osztály és a végpontok

A `presentation.api.controller` csomagban található **SignController** osztály gyűjti össze az összes végpontot, amin keresztül kéréseket lehet küldeni feldolgozásra a

prezentációs komponensnek. A végpontok mindegyike egy-egy metódusnak felel meg, melyek visszatérési értéke `jakarta.ws.rs.core.Response` típusú, viselkedésük és a bemenetek kezelése a végpontokról szóló részben van leírva. A `Response` osztály példányait az építő tervezési mintával lehet megvalósítani, a státuszkódot a `Response.status(status: ResponseStatus)` hívással lehet beállítani, a törzsben a válaszként küldött adatok pedig az `entity(o: Object)` hívással vannak hozzáadva.

A nevek a **`SignInDto`** példányaként vannak hozzáadva a kódban a válaszhoz, a jelzőtáblák SVG formátumú grafikái bájtömbként. A nevek lekéréséhez az osztály befecskendezi a táblák listáját tartalmazó CSV-fájlt kezelő `SignInService` egy példányát (**`signInService`**).

Az osztály befecskendezi magába az `ObjectMapper` egy példányát (**`mapper`**) a `/signs/{code}/image` végpontra POST-metódusú kérések törzsben küldött JSON-struktúrájának értelmezéséhez, hogy egyszerű jelzőtáblák esetén a kérés törzsének hiánya vagy az üres JSON-objektum (`{}`) ne eredményezzen 400-as státuszkódú választ (mint ahogy az történik a paraméterek alapértelmezett JSON-elemzésénél).

A `SignInController` a `SignInDtoMarshallerService`-ből is befecskendez egy példányt (**`signInDtoMarshallerService`**), ami a táblák jelképeinek elkészítésére irányuló kéréseket „eligazítja” a `SignInGraphicsService` megfelelő metódusához.

3.2.6.2. A táblatípus-elosztó (*`SignInDtoMarshallerService`*)

A **`SignInDtoMarshallerService`** alkalmazás hatáskörű bean, melynek célja a táblák kódja alapján a különböző adatokkal rendelkező jelzőtáblák megfelelő kezelése. Mivel ez felelős a jelzőtáblák jelképéről készült grafikák elkészíttetésére is, befecskendezi magába a `SignInGraphicsService` bean egy példányát **`graphicsService`** nevű adattagként, ami bájtömböt ad vissza a különböző grafikákat elkészítő metódusok visszatérési értékeként, melyet aztán a `SignInDtoMarshallerService` visszaad a `SignInController`-nek, ami a HTTP-válasz törzsébe teszi az SVG formátumú vektorgrafikát.

Az osztály két nyilvános metódussal rendelkezik:

- **`getGraphicDataFromDto(code: String, dto: SignInDataDto)`**: a tábla kódja alapján azonosítja a jelzőtábla típusát. Ha egyszerű tábláról van szó, kód alapján elkéri

a megfelelő grafikát a `graphicsService`-től, testre szabható táblák esetén a `SignDataDto` típusú adatátviteli objektumot explicit módon konvertálja az adott tábla típusához tartozó adatstruktúrává (amiknek fájljai a `COMMONS` csomagban vannak), tehát

- C-023 (Össztömeg-korlátozás), C-033 (Sebességkorlátozás), G-152 (Számozott út azonosító táblája) esetén `LabelDataDto` (mivel ezek adatai mind egy címkeként reprezentálhatók),
- E-SAVBESOR (Sávbesorolás rendjét jelző tábla) esetén `LaneArrangementSignDto`,
- JL (Fényjelző készülék(ek)) esetén pedig `TrafficSignalSetDataDto`

típusra konvertálja az argumentumként fogadott adatátviteli objektumot, és azt adja át a `graphicsService` megfelelő táblatípust elkészítő metódusának. Ha a kód alapján az adatstruktúra konverziója nem sikerül, a metódus `IllegalArgumentException` típusú kivételt dob. A grafika elkészítésének megghiúsulásakor dobott `GraphicCreationFailedException` kivételt a metódus tovább dobja, ezt kezelni kell.

- **`isCodeForSpecialSign(code: String)`**: igaz, ha a tábla kódja testre szabható táblához tartozik.

3.2.6.3. Jelzőtábla-grafika elkészítésének szolgáltatásosztálya

A jelzőtáblák átalakításáért egy táblakódból (és testre szabható jelzőtáblák esetén a tábla megfelelő típusú adataiból) a jelképet ábrázoló grafikává a **`SignGraphicsService`** alkalmazás hatókörű bean felelős.

A jelzőtáblák, valamint testre szabható jelzőtáblák esetén a grafika külön elemei mind a jelzőtáblák tervezésére vonatkozó jelenleg (2026. április) hatályos útügyi műszaki előírások alapján készültek, melyek megtalálhatók az irodalomjegyzékben az alábbi indexekkel: [19], [20], [21], [22], [23], [24], [25], [26]. A jelképek grafikáinak elkészítéséhez az Inkscape programot használtam.

Ez az osztály gyűjti össze az `application.properties` fájlban beállítható paramétereket és a vektorgrafikák fájljait tartalmazó könyvtárak útvonalakat, melyeket mind adattagként fecskendez be.

- **`presentation.graphics.char.baseheight` (`baseCharHeight`):** A karakterek SVG-grafikáinak verzálmagassága (túlnyúló elemek nélküli nagybetűk, pl. „A” magassága) pixelekből (alapérték: 1000)
- **`presentation.graphics.speedlimit.labelheight` (`speedLimitLabelHeight`):** A „Sebességhatár” jelzőtábla címkéjének magassága pixelekből (alapérték: 700, a jelzőtábla alapjának 0,35-szöröse). A karakterek ilyen magasságúra lesznek átméretezve.
- **`presentation.graphics.weightrestriction.labelheight` (`weightRestrictionLabelHeight`):** Az „Össztömeg-korlátozás” jelzőtábla címkéjének magassága pixelekből (alapérték: 535, a jelzőtábla alapjának 0,292-szerese előírás alapján [22]). A karakterek ilyen magasságúra lesznek átméretezve.
- **`presentation.graphics.trafficsignal.spacing` (`trafficSignalSpacing`):** A jelzőlámpák grafikájának létrehozásakor a csoport egyes lámpái közt kihagyott rés szélessége pixelekből (alapérték: 30).
- **`presentation.graphics.lanearrangement.baseheight` (`laneArrangementSignBaseHeight`):** Az elkészített sávbesorolás rendjét jelző tábla magassága pixelekből (alapérték: 1000).
- **`presentation.graphics.lanearrangement.spacing` (`laneArrangementSignSpacing`):** A jelzőtáblán az egyes sávok között hagyott rés szélessége pixelekből (alapérték: 100).
- **`presentation.graphics.signs.path` (`signsPath`):** Egyszerű jelzőtáblák és testre szabható jelzőtáblák alapjainak könyvtárának relatív útvonala (alapérték: `signs`).
- **`presentation.graphics.chars.white.path` (`whiteCharsPath`):** Fehér színű karakterkészlet könyvtárának relatív útvonala (alapérték: `signs/white_characters`).

- **presentation.graphics.chars.black.path** (**blackCharsPath**): Fekete színű karakterkészlet könyvtárának relatív útvonala (alapérték: `signs/black_characters`).
- **presentation.graphics.laneArrangement.elements.path** (**laneArrangementElementsPath**): Sávbesorolás rendjét jelző tábla elemeit tartalmazó könyvtár relatív útvonala (alapérték: `signs/lane_arrangement_elements`).
- **presentation.graphics.trafficSignals.elements.path** (**trafficSignalsElementsPath**): Jelzőlámpákat tartalmazó könyvtár relatív útvonala (alapérték: `signs/signals`).

Az osztály nyilvános metódusai bájtömbként adják vissza a kódnak és az adatoknak megfelelő jelzőtáblák grafikáit. A grafika elkészítésének megghiúsulása esetén `SignGraphicsCreationException` típusú kivételt dobhat mindegyik ilyen metódus, ezt kezelni kell.

- **getLaneArrangementSignAsBytes(dataDto: LaneArrangementSignDataDto): byte[]**: Az adatok alapján elkészíti a `dataDto` argumentumban található, sorrend szerint balról jobbra sávok reprezentációját tartalmazó, `SignLaneDto` típusú lista alapján a sávbesorolás rendjét jelző táblát. Először elkészítetteti a sávok iránya és típusa alapján a sávokat a **laneSupplier** `LaneSupplier` típusú objektummal, ami az argumentumban átadott útvonalon (`laneArrangementSignPath`) megkeresi és az elemekből elkészíti a sávokat. A megfelelő helyekre elválasztók grafikáját automatikusan elhelyezi (általános és kerékpársáv közé sárga, busz- és egyéb másik sáv közé fehér szaggatott vonalat). Ezt végül egy, a sávelrendezés szélessége alapján méretezett, `laneArrangementSignBaseHeight` pixel magas kék alapú, fehér szegélyű téglalapra helyezi, ezt adja vissza.
- **getSimpleSignAsBytes(code: String): byte[]**: A kód alapján megkeresi a jelzőtáblák könyvtárában a jelzőtábla kódjával elnevezett SVG-fájlt, ezt visszaadja bájtömbként. Ha nem talál a kóddal megegyező nevű fájlt az útvonalon, egy `placeholder.svg` nevű szürke kört ad vissza.

- **getWeightRestrictionSignAsBytes(dataDto: LabelDataDto): byte[]:** A címkét tartalmazó adatátviteli objektum alapján elkészíti az „Össztömegkorlátozás” jelzőtáblát. Az 1000 pixel magas alap (a táblák könyvtárában `restriction_base.svg` nevű fájl) közepére `weightRestrictionLabelHeight` magasságú, fekete karakterekből álló címkét tesz, a szám tört része és a mértékegységet jelölő „t” betű a címke magasságának 0,7-szeresére van méretezve a tábla tervezésének előírása alapján. [22]
- **getSpeedLimitSignAsBytes(dataDto: LabelDataDto): byte[]:** A címkét tartalmazó adatátviteli objektum alapján elkészíti a „Sebességkorlátozás” jelzőtáblát. Az 1000 pixel magas alap (a táblák könyvtárában `restriction_base.svg` nevű fájl) közepére `speedLimitLabelHeight` magasságú, fekete karakterekből álló címkét tesz.
- **getSignalSetAsBytes(dataDto: TrafficSignalSetDataDto): byte[]:** Az adatátviteli objektumban szereplő jelzőlámpák listája alapján a lista sorrendje szerint balról jobbra, `trafficSignalSpacing` pixel szélességű réssel elkészíti az egyes jelzőlámpák jelképéből álló jelzőlámpa-csoport grafikáját, és visszaadja azt.
- **getShieldSignAsBytes(dataDto: LabelDataDto): byte[]:** A címkét tartalmazó adatátviteli objektum alapján elkészíti a számozott út azonosító jelzőtábláját. Az 1000 pixel magas alaphoz először méretezi a fehér karakterekből álló címkét ($\text{alap karaktermagasság} * 7 * 0,7 / 24$ a tervezési előírás alapján), [25] majd elhelyezi azt a táblák könyvtárában található `shield_base.svg` nevű alap közepére.

3.2.6.4. A grafika modellje

Egy grafika általános meghatározása absztrakt metódusokkal a **GraphicImplementation** osztályban található. A **GraphicImplementation** osztályból leszármaztatott osztályok célja a grafika kezelése az egyes grafikai elemek, azok méretezése és a grafika koordinátarendszerén belüli pozíciója szintjén.

Egy **GraphicImplementation**-implementáció az alábbi metódusokat kell, hogy megvalósítsa:

- **getWidth(): double**: a grafikaimplementáció szélessége.
- **getHeight(): double**: grafikaimplementáció magassága.
- **getScaleX(): double**: a grafikaimplementáció szélességének jelenlegi mérete, ha 1 az eredeti szélesség.
- **getScaleY(): double**: a grafikaimplementáció magasságának jelenlegi mérete, ha 1 az eredeti magasság.
- **getIds(): List<String>**: az implementáción szereplő elemek azonosítói. Egy grafikaimplementációhoz hozzáadhatók más, **GraphicImplementation** típusú elemek; a hozzáadás módja és reprezentációja az implementációra van bízva, de a hozzáadott elemeknek azonosíthatóknak kell lenniük legalább egy **String** típusú azonosító által, amit az **addElement** metódusban lehet megadni.
- **addElement(element: GraphicImplementation, id: String, x: double, y: double)**: egy másik grafikaimplementáció beágyazása **id** azonosítóval az (x, y) helyre a már meglévő grafika koordinátarendszerén.
- **moveRelativeElement(id: String, dx: double, dy: double)**: az **id** azonosítójú elem eltolása a koordinátarendszerben (dx, dy)-nal.
- **moveRelativeAll(dx: double, dy: double)**: az implementáció minden elemének eltolása (dx, dy)-nal.
- **removeElement(id: String)**: az **id** azonosítójú elem törlése a grafikaimplementációról.
- **setScale(x: double, y: double)**: a grafikaimplementáció méretének beállítása, ha 1 az eredeti méret. A méret beállítása módosítja a **getWidth** és **getHeight** visszatérési értékeit, azoknak mindig a jelenlegi méretezés szerinti szélességet és magasságot kell visszaadnia.

A **GraphicImplementation** egyetlen megvalósítása az **svgGraphicImplementation**, mely egy SVG-dokumentum implementációját tartalmazza. Magát az SVG-dokumentumot a védett **document** adattagban tárolja, ami

`org.w3c.dom.Document` típusú, és maga a dokumentum egy DOM-faként¹⁰ van reprezentálva, melynek gyökerére az **`svgRoot`** `org.w3c.dom.Element` típusú adattag mutat.

Az egyes nyilvános metódusok is ezt a dokumentumot manipulálják. A fa struktúrának köszönhetően egy másik implementáció hozzáadása annyiból áll, hogy a másik grafikaimplementáció `document` adattagját lemásoljuk, a gyökérből kivesszük a metaadatokat, hozzáadjuk az azonosítót és a koordinátákat, ahova elhelyezzük, majd a SVG-dokumentum fájának gyökeréhez hozzáadjuk.

3.2.6.4.1 Az `SvgGraphicImplementation` tulajdonságai az SVG-dokumentumban

Az osztály által megvalósított grafika gyökere `<svg>` címkéjű elemnek felel meg. A dokumentum szélessége és magassága (`getWidth()` és `getHeight()` értékei) a gyökér `<svg>`-címké `width` és `height` attribútumainak felelnek meg. Egy adott elem hozzáadásakor az `x` és `y` argumentumok a dokumentum fájához hozzáadott elem címkéjének az `x` és `y` attribútuma lesz beállítva, elem relatív mozgatasakor is ezt az értéket módosítja az implementáció osztálya.

Az `SvgGraphicImplementation` fontos sajátosságai az exportálási lehetőségek. Két opció van, melyek az alábbi nyilvános metódusok hívásával érhetők el:

- **`export(fileName: String)`**: a képet `[fileName].svg` néven exportálja a szerver gyökérkönyvtárba.
- **`exportAsBytes()`**: `byte[]`: az SVG-dokumentumot bájtömbként visszaadja. A `GraphicsService` osztály is ezt a metódust használja a jelzőtáblák grafikáinak visszaadásához.

3.2.6.4.2 Téglalapok grafikaimplementációi

Az `SvgGraphicImplementation` osztályt terjeszti ki az `SvgBorderedRectangleImpl` osztály, mellyel grafikaimplementációként el lehet

¹⁰A Document Object Model egy HTML-, SVG- és XML-dokumentumokat faként leíró struktúra modellje [27]

készíteni egy, a konstruktor argumentumain keresztül testre szabható, szegéllyel rendelkező téglalapot:

- **width:** `double`: a téglalap szélessége pixelekben
- **height:** `double`: a téglalap magassága pixelekben
- **color:** `RgbColor`: a téglalap alapjának színe
- **borderwidth:** `double`: a szegély szélessége pixelekben
- **borderColor:** `RgbColor`: a szegély színe

Egy másik konstruktorral megadható csupán az első három paraméter, ekkor nincs az elkészült téglalapnak szegélye.

3.2.6.4.3 Szolgáltatók

A `presentation.model.supplier` csomagban találhatók a jelképek grafikáinak elkészítéséhez szükséges egyes elemeket szolgáltató osztályok, akik a `GraphicImplementation` példányaként adják vissza az argumentum alapján a megfelelő elemet. Két interfész van, amit a szolgáltatást megvalósító osztályok implementálhatnak:

- **CharacterGraphicSupplier:**
 - Implementálandó metódus:
 - **getGraphicFromChar**(character: `char`):
`GraphicImplementation`
 - Karakterek grafikáit szolgáltatja a `getGraphicFromChar` metódus argumentuma alapján.
- **LaneSupplier**
 - Implementálandó metódusok:
 - **getLaneFromDto**(dto: `SignLaneDto`):
`GraphicImplementation`
 - **getSeparator**(separatorType: `LaneSeparatorType`):
`GraphicImplementation`
 - A `getLaneFromDto` egy adott sáv DTO-adatstruktúrája alapján visszaadja a megfelelő irányú, különleges sáv esetén a megfelelő piktogrammal ellátott sáv

grafikaimplementációját, a `getSeparator` pedig a **LaneSeparatorType** felsorolástípus megfelelő eleméhez tartozó elválasztó elem grafikaimplementációját adja vissza. A **LaneSeparatorType** elemei:

- **BUS** (buszsáv elválasztója)
- **BICYCLE** (kerékpársáv elválasztója)

A **CharacterGraphicsSupplier** interfész implementációja a **CharacterGraphicsSupplierImpl**. A konstruktor argumentumában megadott könyvtárban (amit a **SignGraphicsService** a befecskendezett `application.properties`-attribútumból ad át) az alábbi szabály szerint keresi meg a megfelelő SVG-fájlt, amit beolvas és visszaad **GraphicImplementation**-példányként:

- Szám vagy nagybetű: `[karakter].svg`
- Kisbetű: `[karakter]_min.svg`
- Szóköz: `space.svg`
- Vessző: `comma.svg`

Ha nem talál megfelelő nevű fájlt a szolgáltató egy karakter grafikájának létrehozásához, feltételezi, hogy az argumentumban átadott karakter nem része a könyvtár karakterkészletének, és **IllegalArgumentException** típusú kivételt dob.

A **LaneSupplier** interfész implementációja a **LaneSupplierImpl**. A konstruktor argumentumában megadott könyvtárban (amit a **SignGraphicsService** a befecskendezett `application.properties`-attribútumból ad át) az alábbi szabály szerint keresi meg a `getLaneFromDto` metódus a sáv objektumának megengedett irányai alapján a megfelelő nyilat ábrázoló SVG-fájlt, amit beolvas:

- „`forward_left_right.svg`”, ebben a sorrendben a nem megengedett irányokat kihagyva a fájlnevből (pl. egyenes: `forward.svg`, egyenes-jobb: `forward_right.svg`).

Ha nem talál megfelelő nevű fájlt a szolgáltató a sávnyíl grafikájának létrehozásához, akkor **SignGraphicsCreationException** típusú ellenőrzött kivételt dob, így ezt a kivételt a metódus hívásakor kezelni kell. Ha sikeres volt a fájlbeolvasás, és a sáv típusa nem általános,

elhelyezi rajta a megfelelő típus ikonját, amit ugyanabban a mappában keres a sávtípus `LaneType`-beli elemének a csupa kisbetűs nevével + a „.svg” fájlkiterjesztéssel.

Hasonló módon a `getSeparator` metódus a `LaneSeparatorType` felsorolástípus elemének a kisbetűs nevét keresi a könyvtárban a „.svg” kiterjesztéssel. Mivel ez is `SignGraphicCreationException` típusú ellenőrzött kivételt dob, ha nem találja a fájlt, a kivétel ennek a metódusnak hívásakor is kezelendő.

3.2.6.4.4 Grafikai elemek absztrakciója és címkék

A **GraphicElement** absztrakt osztály tartalmaz magában egy implementációt, egy `getter` metódust az implementációra, valamint egy-egy metódust az implementáció magasságának és szélességének a lekérdezésére és az implementációban szereplő azonosítók listájára. A **GraphicElement** kiterjesztését olyan grafikát modellező osztályok megvalósítására célszerű használni, amik nem az implementáció elemeinek közvetlen kezelésére szolgálnak, hanem komplexebb műveleteket absztrahálnak a metódusaikkal, vagy akár kiegészítő adattagokkal is rendelkeznek, amik befolyásolják az implementáció elemeit vagy az implementáción végrehajtható műveleteket.

A prezentációs komponensben egy osztály terjeszti ki a **GraphicElement**-et, a **Label**. Az osztály konstruktora elvár

- egy verzálmagasságot pixelben (**lineHeight**: `double`)
- egy szolgáltatót a karakterek grafikáihoz (**characterGraphicsSupplier**: `CharacterGraphicsSupplier`)

A **Label** osztály az egyes karaktereket a **lineHeight** értékéhez relatívan képes meghatározni a karakterek tetejének a verzálmagassághoz képesti függőleges eltolódását a `Map<Character, Supplier<Double>>` típusú **offsetMap** szótárral. A szótár eltolódási értékeinek számítása a magyarországi jelzőtáblák betűtípusát veszi alapul. [19]

Az **add(c: char)** metódussal lehet új karaktert adni a címkéhez. A karakterek közti betűköz a verzálmagasság hetede. Ha szükség van a grafikaimplementációra (pl. exportáláshoz), a **getImpl()** metódussal lehet megszerezni.

3.2.6.5. Jelzőtáblák kód-név listája

A jelzőtáblák kód-név listája a **presentation.names.path** attribútummal adható meg (alap esetben a szerver erőforrásainak gyökérkönyvtárában a `sign_names.csv` fájl). Ez egy CSV-fájl, mely pontosvesszővel elválasztva tartalmazza soronként a szerver által ismert jelzőtáblák **kódját** és **nevét**.

A CSV-fájl útvonalát a **SignListService** alkalmazás hatókörű bean fecskendezi be, amely felhasználja a benne lévő adatokat a nyilvános metódusaiban:

- **getSignName(code: String): Optional<SignNameDto>**: visszaadja az argumentumban átadott jelzőtáblakódhoz tartozó nevet egy **Optional**-példányba csomagolva, ha jelen van a listában, ellenkező esetben a visszatérési érték **Optional.empty()**. Ha nem található vagy hibás formátumú a fájl, **RuntimeException** típusú, hibaüzenettel ellátott kivétel dobódik.
- **getAllCodes(): List<String>**: visszaadja a listában található jelzőtáblák kódjainak listáját, a CSV-fájl sorainak sorrendjében. Ha nem található vagy hibás formátumú a fájl, **RuntimeException** típusú, hibaüzenettel ellátott kivétel dobódik.

3.3. Alkalmazás

3.3.1. Áttekintés

Az asztali alkalmazás a szervertől külön indítható, de szorosan összedolgozik vele, kéréseken keresztül a szerverrel kommunikálva készített el a tervrajzokat és jeleníti meg a jelzőtáblákat, valamint onnan szerzi meg a táblák elérhető listáját is a prezentációs komponens CSV-fájljából (3.2.6.5. fejezet). A felhasználói alkalmazás tartalma, célja és használata a felhasználói dokumentációban található a 2.3. fejezetben.

3.3.1.1. Alkalmazott technológiák

Az alkalmazás Electron keretrendszerben fut, mely beágyazza és futtatható asztali alkalmazásként teszi elérhetővé a beágyazott Chromium böngészőmotorral azt, hogy webalkalmazások technológiáit használva valósuljon meg az alkalmazásfejlesztés. [28] Ezt az utat választva a JavaScripthez írt, a nyelv népszerűsége miatt nagyon nagy számú, korszerű keretrendszer használható a kezelőfelület megvalósításához. Felmerült az architektúra

tervezésekor a szerver és az asztali alkalmazás egy, tisztán Java-projektként történő fejlesztése (a Java Swinget használva a kezelőfelülethez), azonban ez a két részre osztott modell

- növeli a modularitást, a szerverrel végpontokon keresztül kommunikál az asztali alkalmazás, és a szerver így önállóan is működik: ugyanezeket a végpontokat akár más alkalmazások is igénybe vehetik,
- továbbá, ahogy fentebb említettem, lehetővé teszi a JavaScripthez készített keretrendszerek nagyon széleskörű és korszerű ökoszisztémájának használatát a kezelőfelület elkészítéséhez.

Az Electron futó webalkalmazásos technológiákat alkalmazó kódbázis TypeScript nyelven íródott. A TypeScript egy, a JavaScriptet kiegészítő és azzal kompatibilis, erősen típusos programozási nyelv. A TypeScripittel a típusok megjelölése könnyebben értelmezhetővé teszi a kódot a fejlesztők számára, és az így lehetségessé váló statikus típusellenőrzés segít kiküszöbölni a dinamikus típusosságból eredő futásidejű problémákat: például egy paraméterként számot váró függvényt nem hívhatunk meg véletlenül string paraméterrel, mert a statikus típusellenőrzés felismeri a változók típusát és a függvény paraméterlistájában a típusok közti különbséget. [29] Ez a TypeScript-kód fordításakor jelzésre kerül, de a nyelvet támogató integrált fejlesztői környezetek már a kód szerkesztésekor jelzik a hibát.

A tervrajz ábrázolásához gráfként működő modellje miatt célszerűnek találtam a GoJS használatát. A GoJS egy interaktív gráfok beágyazására készült JavaScript-könyvtár [30] (amihez van TypeScript-támogatás típusokkal), melyben, a tervrajzot egy gráfnak megfelelően, könnyen lehetett ábrázolni a tervrajz szerver felől JSON-objektumként megkapott modelljét. Szintén kedvező, hogy az elemek pozícióira a GoJS egy kétdimenziós koordinátarendszert használ, épp úgy, ahogy a tervrajz modellje is.

Az alkalmazás továbbá a Quasar keretrendszert is használja. A kezdő fájlstruktúra is a Quasar parancssori konfigurációs menüjével készült, amiben be lehetett állítani többek közt a nyelvet (az alkalmazás esetében TypeScript) és hozzá lehet adni a gyakori használt könyvtárakat (Axios, Pinia, stb.), majd létrejön a beállítások alapján a projekt váza úgy, hogy a kiválasztott funkciók rögtön működnek, és ki van alakítva egy jó könyvtárstruktúra is, ami megkönnyíti a fejlesztést. [31] A Quasar magába foglalja a Vue.js keretrendszert, mely az egyes oldalakat és vizuális komponenseket egy-egy `.vue` kiterjesztésű fájlba különíti, amiben szerepel a

megjelenítés, az annak működtetéséhez szükséges kód és a stíluslap. Az egyes fájlok így nincsenek réteg szerint kategorizálva, hanem Vue-komponensenként, ahol a megjelenésért felelő HTML-kód, a stíluslap és a komponens logikájáért felelős TypeScript-kód egy fájlban található. [32] A Quasar tartalmaz számos könnyen használható és testre szabható vizuális elemet is.

A szerver felé végzett API-hívásokat az Axios végzi, az alkalmazás futása során tárolandó, több komponens által is használandó adatokat a Pinia által kezelt store-okban¹¹ tároltam.

A stílusokhoz SCSS-t használtam, ami a Sass CSS-előfeldolgozó nyelve, főként azért, mert többek közt változókat is bevezet, ami például lehetővé teszi a globálisan elérhető konfigurációs értékek (mint a színek) használatát.

3.3.2. Az alkalmazás általános stílusa

Az asztali alkalmazás egy egységes színsémát használ, ami SCSS-es változók formájában érhető el minden CSS-kódban.

A színséma monokróm, fő színe a sárga, ennek különböző árnyalatait használja a felület (egy-két kivétellel). Az `src/css/quasar.variables.scss` fájlban vannak definiálva a színek:

-  Elsődleges szín: #EDB72E (\$primary)
-  Másodlagos szín: #FFDB80 (\$secondary)
-  Kiemelés színe: #9F7200 (\$accent)
-  Sötét szín: #1D1D1D (\$dark)
-  Negatív szín: #FF001E (\$negative)

¹¹ A „store” szó magyar megfelelőjére, mint olyan komponensre egy alkalmazásban, mely kliensoldalon tárol univerzálisan elérhető helyen az alkalmazás futásának ideje alatt adatokat, nem találtam utalást magyar nyelvű szakirodalomban, és mivel egy gyakran használt, közismert fogalomról van szó, nem szerettem volna saját módon magyarázni.

Ezekon kívül a felület használja a fehér (#FFFFFF) és fekete (#000000) színt. Kitűnik a negatív szín, mint az egyetlen a listában, ami nem a sárga egy árnyalata, ez a törlési műveletek kiemelése és a felhasználói felületek hagyománya miatt piros.

Az alkalmazásban minden szöveg betűtípusa a **Verdana**, egy talp nélküli betűtípus, mely különösen könnyen olvasható akár kis méretben is. [33] A Verdana betűtípust emellett személy szerint esztétikusnak tartom.

3.3.3. Az alkalmazás típusdefiníciói

A TypeScript egy előnye, hogy a JavaScript-objektumokat külön, névvel és típussal rendelkező adattagokból álló adatstruktúrát megkövetelő típusokkal lehet ellátni. A szervernek küldött kérések és a szervertől válaszokban kapott objektumok típusainak struktúrái megfelelnek a szerver komponenseinek **dto** nevű csomagjaiban található adatátviteli objektumok struktúráinak, amit a szerver használ adatok küldésére és fogadására. Az típusdefiníciók az **src/types** mappában találhatók, és almappákra vannak osztva:

- A **diagram** mappában találhatók a **diagram-types.ts** fájlban a Tervrajz lapon a tervrajz megjelenítéséhez GoJS-diagram által használt típusok.
- A **signage** mappában a **road-sign-types.ts** fájl tárolja a jelzőtáblákat és azok adatait reprezentáló típusokat.
- A **template** mappában találhatók szerep szerint további külön fájlokba szedve a tervrajz modelljének típusai. Az **index.ts** fájl összegyűjt és exportál minden, a tervrajzzal kapcsolatos, külön fájlokban definiált típust (beleértve a szomszédos **signage** mappában található jelzőtáblákkal kapcsolatos típusokat, mivel a tervrajzok jelzőtábla-kompozícióinak részét képezik), hogy kevesebb **import** utasítással lehessen importálni a tervrajz elemeit.

3.3.4. API-hívások

Az alkalmazás működése erősen függ a szervertől, amivel az Axios API-hívásain keresztül kommunikál a végpontokkal. A szerver címét az Axios az alkalmazás indulásakor végrehajtott **defineBoot()** függvényében a **getApiBaseUri()** megpróbálja az alkalmazás erőforrásai közül beolvasni az **app-config.json** fájlból a szerver címét. Ha nem sikerül (pl. nincs jelen a fájl), a szerver az Axios a **localhost:8080** címen próbálja elérni.

A `services/sign/index.ts` fájlban található a `signs/*` útvonalra irányuló hívások függvényei:

- **getImage**(code: string, data: (SignData | undefined)): `signs/[„code” értéke]/image` POST metódusú HTTP-kérés küldése a szervernek (3.2.3.5). Siker esetén a válasz `data` adattagja tartalmazza a jelképet tartalmazó SVG-dokumentumot, ami felhasználható stringként.
- **getName**(code: string): `signs/[„code” értéke]/name` GET metódusú HTTP-kérés küldése a szervernek (3.2.3.2. fejezet). Siker esetén a válasz `data` adattagja `SignName` típusú.
- **getList**(): `/signs` GET metódusú HTTP-kérés küldése a szervernek (3.2.3.3. fejezet). Siker esetén a válasz `data` adattagja `string[]` típusú.

A `services/template/index.ts` fájlban található a szerver `/template` végpontjára irányuló kérés a tervrajz elkészítéséhez:

- **getTemplate**(bbox: BoundingBox, hasSignage: boolean): `/template` GET metódusú HTTP-kérés küldése a szervernek (3.2.3.1. fejezet). Siker esetén a válasz adattagja `RoadTemplate` típusú, és a szerver felől érkező tervrajz adatátviteli objektumát tartalmazza.

3.3.5. Tervrajz tárolása az alkalmazás futása során

A táblák jelképére vagy nevére irányuló hívások válaszát az alkalmazás a hívást intéző Vue-komponensben tárolja (3.3.6. fejezet), azonban a tervrajzot egy Pinia által induláskor inicializált store-ban tárolja, ami a program bármely komponenséről elérhető, és oldalak váltása vagy egy oldal újratöltése során sem veszik el, csak akkor, amikor az alkalmazás leáll (ekkor későbbi betöltéshez menteni ajánlott a tervrajzot), vagy a store-ban tárolt tervrajz felülírásra kerül.

3.3.5.1. Tervrajz store-ja

A tervrajz store-ja az **`src/store/template-store.ts`** fájlban van definiálva, ahol a **`template`** nevű `RoadTemplate` típusú reaktív adattagban tárolódik a tervrajz. A **`store.fetchTemplate(bbox: BoundingBox, hasSignage: boolean, errorCallback: string => void)`** függvényével lehet végrehajtani az Axios-hívást a szerver felé, és siker

esetén eltárolni a tervrajzot. Az elérhető jelzőtáblák kódjainak listája szintén egy store-ban tárolódik. A visszatérési érték sikeres hívás esetén igaz, sikertelen esetén hamis (és aktiválódik hibaüzenettel az `errCallback` függvény).

3.3.5.2. Jelzőtáblák listájának store-ja

A táblák listájának store-ja a `src/store/sign-list-store.ts` fájlban van definiálva, a lista a `signList` nevű `Map<string, string>` típusú reaktív adattag (egy szótár, ahol a kulcs a tábla kódja, az érték a tábla neve). A store `fetchList()` függvénye Axios-hívásokkal összerakja a táblalista szótárát (először lekérdezi az elérhető kódokat, majd az egyes kódokhoz tartozó neveket), majd azt eltárolja. Visszatérési értéke siker esetén igaz, ellenkező esetben hamis.

3.3.6. Oldalak és nézetkomponensek

Az alkalmazást összetevő Vue egyfájlos komponensek (single file component) az `src` mappán belül a szerepüknek megfelelő almappában találhatók:

- Minden nézet a `layouts` mappában található `MainLayout.vue` fájlt használja, ami tartalmazza az oldalsó menüt, a fejléct, és beágyazza a jelenlegi nézet oldalát.
- Az oldalak a `pages` mappában találhatók. Az alkalmazás három lapjához egy-egy oldal tartozik:
 - Térkép: `MapPage.vue`
 - Tervrajz: `TemplatePage.vue`
 - Táblaszerkesztő: `SignEditorPage.vue`
 - Hiba esetén a Vue oldalak közti navigációt kezelő router komponense ez `ErrorNotFound.vue` oldalra irányít át.

Az oldalak közti navigációt az `src/router/routes.ts` fájlban található `routes` konstans definiálja. A fő komponens mindig a `MainLayout`, ebbe helyezi el az útvonal alapján a megfelelő oldalt.

Az oldalakba beágyazható, újra felhasználható komponensek a `components` mappában vannak.

3.3.6.1. Oldalak

3.3.6.1.1 Térkép lap

A Térkép lap oldala a **MapPage**. Egyetlen eleme a kiválasztáshoz használt térkép komponense, a **SelectMap**.

A **SelectMap** komponens egy térképet valósít meg a Leaflet használatával. Az OpenStreetMap hivatalos csempéit (a térképet alkotó négyzet alakú képek) használja a megjelenítéshez a **tile.openstreetmap.org**ról. A térkép inicializációjakor a nézet Magyarország közepére van beállítva (47,180086; 19,503736).

A Leaflet **L** konstansának **map** metódusa végzi el a Leaflet-térkép példányosítását, ami a **map** any típusú változóban tárolódik el (a Leaflet nem támogatja a TypeScriptet, így itt nincs típusosság). A térkép inicializációja a Vue-komponens összeállítását követően (az **onMounted** meghívásán keresztül) történik az **initializeMap()** függvényben. A Leaflet nem támogatja a Vue.js keretrendszert, így körülményesnek bizonyult a **map** objektum létrehozása, főleg, mivel a Leaflet dokumentációja [34] nem részletezi az **L** konstans pontos típusát és szerepét, csak a rajta keresztül meghívható függvényeket. Bár létezik egy **vue-leaflet** nevű csomag, mely a Leaflet Vue-ben történő integrálását valósítja meg, elhagyatottnak tűnik a projekt, mivel 3 éve nem volt frissítve béta státusza ellenére. Emellett telepítést követően nem működött a Leaflet-térképet megvalósító Vue-komponens sem. Végül az eredeti Leaflet csomagot használtam fel, melynek Vue-kompatibilis implementációjához a Copilotot használtam segédeszközként, amitől kértem egy Vue-beli, a Vue.js 3 konvencióihoz lehetőleg jobban ragaszkodó Leaflet-implementációt demonstráló referenciát az egyes konstansok és változók magyarázatával. Ebből a referenciából kiindulva hoztam létre a **SelectMap**ben a térképet. Ezután a térkép egyes eseményeit kezelő függvényeket, melyek a következő bekezdésben kerülnek tárgyalásra, már a referenciától függetlenül saját magam implementáltam a Leaflet dokumentációja alapján. [34]

A **handleMouseDown**, **handleMouseMove** és **handleMouseUp** függvények a térkép **mousedown**, **mousemove** és **mouseup** eseményeit kezelik, paraméterként megkapják a kurzor helyén található szélességi és hosszúsági fokokat, így az egérgomb lenyomásakor és felengedésekor rögzített koordináták meghatározzák a felhasználó által kijelölt befogó téglalapot. Ez a **drawnRectangle** adattagban tárolódik, aminek az égtájankénti határait

használja a **confirmSelection()** függvény, ami a tervrajz store-jával (amire a **templateStore** tárol a komponensben referenciát) lekérdezteti a befogó téglalapban, a táblázást beállító `<q-toggle>` elemhez kötött **hassignage** igaz-hamis értékkel a tervrajzot. Sikertelen esetben a **templateStore** **fetchTemplate** függvényébe átadott kód alapján a hibaüzenettel megjelenik egy modális ablak, siker esetén pedig a Vue routerén keresztül a felhasználó átirányításra kerül a **TemplatePage** oldalra, ami megjeleníti az imént a store-ral lekérdeztetett tervrajzot.

Ezt a **confirmSelection** függvényt a „Terület kiválasztása” címkéjű gomb aktiválja. A másik, térképpel történő interakció módját váltó gomb az **isDrawing** értékét váltja igaz és hamis között. A befogó téglalap rajzolását csak akkor kezeli a komponens, ha ez igaz. Ekkor kikapcsolja a térkép egérrel történő mozgatását is (**map.dragging**), amit böngésző módba történő visszakapcsoláskor újra engedélyez.

3.3.6.2. Tervrajz lap komponensei

A Tervrajz lap oldala a **TemplatePage**. Tárol egy referenciát a tervrajz store-jában található példányra (**template**).

Az oldal sarkában egy `<div>` elemben elhelyezett két gomb meghívja a **saveTemplate()** és **loadTemplate()** függvényeket, előbbi a tervrajz JSON-leírásának stringjét menti ki egy fájlba az ablak (**window**) **showSaveFilePicker** függvényének segítségével, utóbbi pedig egy ilyen JSON-t tároló szövegfájlt tölt be az ablak **showOpenFilePicker** függvényét használva, és ha a beolvasott JSON érvényes, a belőle készített objektumra cseréli a store-ban tárolt tervrajzot (ha nem, modális ablakot jelenít meg hibaüzenettel).

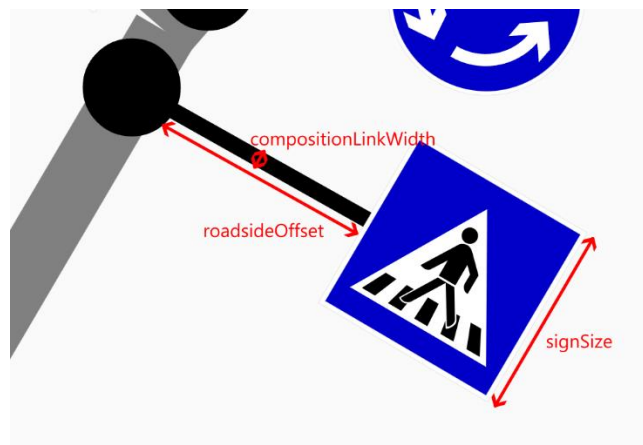
Az oldal beágyazza a **TemplateComponent** komponenst, aminek átadja paraméterként a store tervrajzában referenciáját, valamint a **diagramKey** kulcsot, ami új tervrajz esetén megváltozik, jelezve a Vue-nek, hogy a tervrajzot megjelenítő komponenst is újra kell tölteni.

A **TemplateComponent** jeleníti meg a store-ban tárolt tervrajzot (amihez attribútumban megadva a **TemplatePage**-en keresztül fér hozzá). A **diagram** **Go.Diagram** típusú adattag a diagram, ami a komponens HTML-kódjában a **diagramDiv** **ref** attribútummal rendelkező `<div>` elemben jelenik meg. Ez az elem elérhető a kódban (ugyanazzal a névvel),

és a diagram konstruktorának paramétereként kerül összeköttetésre a GoJS-diagram objektumával.

A Vue-komponens `<script>` elemében az import utasítások után található néhány konstans, ami a diagram beállításakor állít be néhány paramétert:

- **roadsideOffset:** number: jelzőtábla-kompozíciók megjelenítésének távolsága oldalra az úttól (alapérték: 5)
- **signSize:** number: diagramon megjelenített jelzőtáblák magassága (alapérték: 6)
- **scaleFactor:** number: diagram alapértelmezett nagyítási értéke (alapérték: 5)
- **compositionLinkWidth:** number: jelzőtábla-kompozíciók és az úton található helyüket reprezentáló pont közti egyenes vastagsága (alapérték: 0,5)



36. ábra: a konstansok szerepe a megjelenítésben

3.3.6.2.1 A GoJS-diagram

Egy GoJS diagram pontokból (`Go.Node`) és azokat összekötő élekből (`Go.Link`) áll. Egy diagram többféle pontot és élet definiálhat, mindegyik típusnak megadva egy sémát, ami tartalmazza a beállításokat (pl. mozgatható-e), a vizuális elemeket és az adatkötéseket, amikkel a modellből készített, `diagram-types.ts`-ben (3.3.3. fejezet) definiált, diagramra szabott típusok valamelyikét felhasználva prezentálja a felhasználónak.

Az alábbi él- és csúcssémákat használja a GoJS a diagram elkészítéséhez:

- **roadNodeTemplate:** az utak pontjainak GoJS-pontként való reprezentációja. Nem látszódnak, az utakat reprezentáló élek összekötésére szolgálnak, nem lehet őket

kiválasztani vagy mozgatni. Adatkötéssel átveszik a tervrajz modelljében található egyes pontok koordinátáit.

- **signCompositionPointTemplate:** a táblakompozíciók helyének reprezentációja GoJS-pontként, mely egy 2 egységnyi átmérőjű fekete körként van megjelenítve. Adatkötéssel átveszik a táblakompozíciók koordinátáit, és mivel a táblakompozíciókat a táblázó program az útra helyezi, mindig az utakat ábrázoló éleken található. Nem kijelölhetők, nem törölhetők közvetlenül és nem mozgathatók, de a diagram eseményfigyelője kezeli a dupla kattintásokat rá, ami megfordítja a táblakompozíció út menti irányát.
- **signCompositionTemplate:** a táblakompozíciók vizuális reprezentációja GoJS-pontokként. Megjelenése a kompozíció tábláinak jelképei egymás alatt **signSize** magasságban. Nem kijelölhető, de mozgatható. Közvetlenül a diagramon nem törölhető. Kattintásra esemény váltódik ki: egy kattintás a vágólapra másolja a kompozíciót, kettő kattintás beállítja kiválasztott táblakompozíciónak (azaz hozzárendeli a **selectedSignComposition** reaktív objektum értékéhez), amire megnyílik a kompozíciós menü. Pozíciójuk és forgatásuk az általuk reprezentált táblakompozíció adataihoz van kötve, a pozíciót meghatározáskor a **calculateRoadsideOffset** függvény tolja el az úttal merőlegesen **roadsideOffset** távolságra.
- **roadLinkTemplate:** az utak reprezentációja GoJS-élként, amik a **roadNodeTemplate** sémájú útpontok között futnak. Színük szürke, vastagságuk annyi egységnyi, ahány sávból áll a tervrajzban szereplő út, amit reprezentál a GoJS-él. Nem kijelölhetők, nem törölhetők.
- **signCompositionLinkTemplate:** a táblakompozíciók helyeit (**signCompositionPointTemplate** sémájú pontok) és a táblakompozíciókat (**signCompositionTemplate** sémájú pontok) összekötő él. Színük fekete, vastagságukat a **compositionLinkwidth** adja meg. Nem törölhetők, nem kijelölhetők, a táblakompozíciók pontjainak mozgatásával mozgathatók.

Az egyes sémák pontos meghatározása a komponensben található.

A diagram elkészítését és inicializálását az **initDiagram()** függvény végzi, amit a Vue a komponens betöltése után hív meg (ezt az **onMounted()** függvényben történő meghívásával intézi). Az **initDiagram()** az alábbi lépésekkel állítja össze a diagramot:

1. A konstruktor, ami megkapja a diagramot megjelenítő **<div>** elem objektumát és a kezdő beállításokat, inicializálja a diagramot.
2. A diagramhoz hozzáadja a dupla klikkre aktiválódó eseményfigyelőt, mely
 - a. útra kattintás esetén hozzáad egy új táblakompozíciót a kattintott helyre, mely tartalmazza a vágólap tábláit, vagy üres vágólap esetén egy „Elsőbbségadás kötelező” jelzőtáblát, a módosítást elvégzi a store tervrajzán, majd egy tranzakció keretén belül¹² a tervrajz diagramján;
 - b. egy jelzőtábla-kompozíció úton lévő helyét jelölő pontra kattintva pedig megfordítja a kompozíciót az út mentén, a módosítást elvégzi a store tervrajzán, majd egy tranzakció keretén belül¹² a tervrajz diagramján.
3. A diagramhoz hozzárendeli a fentebb leírt sémákat egy-egy kategóriához (egy él és egy pont sémája tartozhat ugyanabba a kategóriába, hiszen egyértelmű az elem típusa és a kategória alapján, hogy melyik séma tartozik rá):
 - a. **roadNodeTemplate**: **road**
 - b. **roadLinkTemplate**: **road**
 - c. **signCompositionLinkTemplate**: **signComposition**
 - d. **signCompositionTemplate**: **signComposition**
 - e. **signCompositionPointTemplate**: **signCompositionPoint**
4. Létrehozza az egyes kategóriákhoz és elemtípusokhoz tartozó listákat, amiket a modell egyes elemeiből **map()** függvénnyel diagrammal kompatibilis elemmé átalakított objektumokkal tölt meg. Az útpontok azonosítói (diagramelem **key** attribútuma) a modellből vannak átemelve, a táblakompozíciók és azok helyeit reprezentáló pontok azonosítóját a kompozíció objektumából a **generateSignCompositionKey(sc:**

¹² A diagram nincs összeköttetésben a modellel, új elemek hozzáadását, valamint elemek módosítását és törlését tranzakciók keretén belül kell elvégezni, ami a diagram **startTransaction(tname: (string | undefined))** függvényének hívásával indítható, és a **commitTransaction(tname: (string | undefined))** függvénnyel véglegesíthető.

`SignComposition`) és a `generateSignCompositionPointKey(sc: SignComposition)` függvények generálják. Az azonosítók alapján lehetséges az elemek későbbi módosítása (így történik pl. a kompozíciók forgatása is a helyet jelölő pontra történő dupla klikkeléskor), valamint az élek összekötése (a tervrajz modelljében benne van az utak kezdő- és végpontjainak az azonosító, szóval ez elég egyszerű feladat). A táblakompozíciók tábláinak jelképeit is ekkor szerzi meg stringként tárolt SVG-dokumentumként (amit a GoJS képes használni) a `fetchSignSvgUrl(sign: RoadSign)` függvénnyel, ami aztán a tábla kódjával és (testre szabható tábla esetén) adataival az Axiossal lekérdezi a jelkép grafikáját a szervertől.

5. Hozzáad a diagramhoz egy eseményfigyelőt, ami a nézetet a diagram közepére igazítja.

3.3.6.2.2 Vágólap

Ahogy a `signCompositionTemplate` leírásánál is említve volt, egy kattintás lemásolja a táblakompozíció tábláit a vágólapra. A vágólap a `signsClipboard: Ref<RoadSign[] | null>` reaktív adattagban tárolódik, mely vagy egy jelzőtáblákból álló lista, vagy nullérték. Alapesetben a `signsClipboard` értéke null. Miután egy táblakompozícióból jelzőtáblákat kapott a vágólap, azok listáját tárolja, és mivel már nem nullérték, megjelenik a jobb alsó sarokban a `SignClipboard` Vue-komponenssel megvalósított panel, mely a vágólapot a `signs-clipboard` attribútumon keresztül kapja meg, és a Vue `v-for` direktívájával és a `fetchSignSvgUrl(sign: RoadSign)` függvénnyel megszerzi a listán szereplő táblák jelképeit a szervertől, és egyesével `<img>` HTML-elemekbe helyezi őket; így egy vizuális előnézetet mutat a felhasználónak a vágólapról. Az útra történő dupla kattintás eseménykezelője, amit az `initDiagram()` hoz létre (3.3.6.2.1. fejezet), a vágólapon lévő táblalista alapján tölti fel jelzőtáblákkal az új kompozíciót (és ha null a vágólap értéke, akkor hagyatkozik az „Elsőbbségadás kötelező” táblára). A vágólap törölhető a `SignClipboard`-ban implementált piros, „X” ikonnal jelölt gombbal: ez nullra állítja a `signsClipboard` értékét, és emiatt a vágólap előnézete is eltűnik a sarokból.

3.3.6.2.3 Jelzőtábla-kompozíció kiválasztása és a táblakompozíció menüje

A jelzőtábla-kompozíciók menüje a Quasar beépített, modális ablakot létrehozó elemével (`<q-dialog>`) van megvalósítva, és akkor jelenik meg, ha van kiválasztott táblakompozíció (a `selectedSignComposition` értéke nem null). Ebbe a modális ablakba van beágyazva

a **SignCompositionMenu** eleme, aminek a kiválasztott táblakompozíció értékét tároló reaktív objektumot a `sign-composition` attribútumon keresztül adja át.

A **SignCompositionMenu** `sign-composition` attribútumán keresztül adott kompozíciót `signComposition` adattagban tárolja. A kompozíció tábláinak a jelképeiről készült SVG-stringek egy listában vannak (`signSources`) ugyanolyan sorrendben, mint a `signComposition` jelzőtábla-kompozíciónak a táblái, így az n indexű tábla jelképe az n indexű kép a `signSources` listában.

A táblakompozíció menüje bal oldalt egy kártyában (aminek háttérszíne az alkalmazás másodlagos színe) mutatja a komponens táblakompozíciójának a tábláit `<img>` HTML-elemként, mely a `signSources` listáját használja a jelképek megjelenítésére. Kattintásra a kiválasztott index az adott tábla indexe lesz. (A **selectedIndex** értéket állítja be.)

A „Tábla törlése” gomb a **deleteFromComposition(index: number)** függvénnyel törölteti a kiválasztott indexű jelzőtáblát a jelképek közül és a kompozíció modelljének listájából is. Az indexet csökkenti eggyel, ha az utolsó elemet törölte a felhasználó.

Az „Új tábla alá” vagy „Új tábla fölé” gombok a kiválasztott indexre (kiválasztott tábla fölé) vagy a kiválasztott index fölé eggyel (kiválasztott tábla alá) beszúr egy új, „B-001” kódú („Elsőbbségadás kötelező”) táblát. A kompozíció modellje mellett a képek listáját is frissíti. A függvény végére a kiválasztott index az új tábláé.

Az éppen kiválasztott jelzőtáblát a jobb oldalt megjelenő **SignMenu** Vue-komponenssel lehet exportálni és szerkeszteni. A Táblaszerkesztő lapon is ugyanez a komponens van jelen a szerkesztett tábla kezelésére, leírása a 3.3.6.3.1 részben található.

3.3.6.2.4 Diagram frissítése táblakompozíció szerkesztését követően

A **TemplateComponent** a beágyazott **SignCompositionMenu** komponens bezárásakor a **handleDialogModelUpdate(isOpen: boolean)** függvényben a **syncSelectedSignCompositionNode()**-ot meghívva szinkronizálja egy diagramtranzakcióban a kiválasztott táblakompozíció változásait, beleértve a törlését, ha nincs több jelzőtábla a kompozícióban, majd törli a kiválasztást (`null`-ra állítja a `selectedComposition` értékét).

3.3.6.3. A Táblaszerkesztő lap és a táblamenü

A Táblaszerkesztő lapot a **SignEditorPage** Vue-komponens valósítja meg. A lapon egy jelzőtáblát lehet megtekinteni, szerkeszteni és exportálni. A jelzőtábla modelljét a **sign** `ref<RoadSign>` típusú reaktív objektum tárolja, melynek értéke a szerkesztett tábla modellje. Alapértéke egy „C-033” kódú jelzőtábla, melynek adata egy „50” feliratú címkéből áll (50-es sebességhatárt jelölő „Sebességkorlátozás” jelzőtábla). A **signSource** reaktív `ref`-objektum tartalmazza a jelzőtábla jelképét SVG-stringként, amit a nézet bal oldalán lévő quasaros `<q-card>` elembe beágyazott `<img>` HTML-elem használ forrásként a tábla megjelenítéséhez. A Vue **watch()** függvényének meghívásával a komponens figyeli a szerkesztett tábla modelljének változásait, és ekkor a szerverről a `fetchSignSvgUrl` meghívásával megszerzi a frissített tábla jelképének SVG-sztringjét, amit hozzárendel a **signSource** értékéhez.

3.3.6.3.1 A táblamenü

A táblák menüje a Táblaszerkesztő lap jobb oldalán és a Tervrajz lapon jelzőtábla-komponensek szerkesztőjében is megjelenik a kompozíció jelenleg szerkesztett táblájához. A **SignMenu** komponens valósítja meg.

A táblamenü az alábbi elemeket tartalmazza egymás alatt elhelyezve:

- Tábla megnevezése és exportálás gombja
- Táblatípus kiválasztó vezérlője
- Tábla szerkesztő komponense (táblatípus szerint)
- „Alkalmaz” gomb

A táblamenüben a létrehozáskor a **sign** attribútumaként átadott modell a **sign** reaktív `ref`-objektum értékeként tárolódik, ami akkor frissül, amikor az éppen szerkesztett jelzőtáblát alkalmazza a felhasználó az „Alkalmaz” gombra kattintva az **updateSign()** függvényen keresztül, ami pedig a **replaceSign()** függvényben másolja át a szerkesztett tábla (**editedSign** értéke) adatait az attribútumon keresztül elért tábla helyére.

A jelenleg szerkesztett jelzőtábla az **editedSign** `ref`-objektum értékeként tárolódik. Ennek a kódját változtatja a `<q-select>` Quasar-féle kiválasztó vezérlő, ami a szerverről a táblák listájának store-jából meríti a kiválasztható elemeket a **signListStore** néven tárolt

referenciája a boltnak. A Vue-komponens **onMounted()** függvénye hívja meg a `store fetchSignList` függvényét, ami megszerzi a listát a szervertől a store-ba (3.3.5.2. fejezet).

A táblatípus változása esetén az `src/utis/road-sign-utis.ts` fájlban található

A szerkesztett tábla kódja alapján dönti el a táblamenü, hogy melyik szerkesztőt jeleníti meg az ezt tároló `<q-card>` HTML-elemben. A szerkesztők a testre szabható jelzőtáblák adatainak szerkesztését teszik lehetővé. Az alábbi táblakódokhoz tartoznak szerkesztő típusú komponensek, melyek neve „Editor”-ra végződik és a `src/components/editor` mappában találhatók:

- **Kód:** C-023 („Össztömeg-korlátozás”)
 - **Szerkesztő:** **WeightLimitEditor**
 - Attribútumként a szerkesztett táblát **WeightRestrictionSign** típusúvá konvertálva kapja meg. Egy, a címkét beállító szövegdobozból áll, melynek szövege legfeljebb 4 karakter lehet, és csak számokat vagy vesszőt tartalmazhat. A szövegdoboz frissülésekor a címke értéke (`data.label`) a szövegdoboz tartalma és a mértékegységet jelölő „t” (szóköz nélkül).
- **Kód:** C-033 („Sebességkorlátozás”)
 - **Szerkesztő:** **SpeedLimitEditor**
 - Attribútumként a szerkesztett táblát **SpeedLimitSign** típusúvá konvertálva kapja meg. Egy bemenetként legfeljebb három darab számjegyet megengedő szövegdobozból áll, aminek értéke össze van kötve a tábla címkéjével (a tábla `data.label` értéke).
- **Kód:** G-152 („Számozott út azonosító táblája”)
 - **Szerkesztő:** **ShieldEditor**
 - Attribútumként a szerkesztett táblát **ShieldSign** típusúvá konvertálva kapja meg. Bemenetként legfeljebb három darab számjegyet megengedő szövegdobozból áll, aminek értéke össze van kötve a tábla címkéjével (a tábla `data.label` értéke).
- **Kód:** E-SAVBESOR („Sávbesorolás rendjét jelző tábla”)
 - **Szerkesztő:** **LaneArrangementSignEditor**

- Attribútumként a szerkesztett táblát **LaneArrangementSign** típusúvá konvertálva kapja meg. A jelzőtábla adataiban szereplő sávok listáját egy `<q-list>` HTML-elemben jeleníti meg. A listában minden egyes sáv rendelkezik egy saját vezérlőpanellel, ami az adott indexű sáv megengedett irányainak és a sáv típusának beállítását teszi lehetővé, a lista értékei a szerkesztett tábla adataiban meghatározott sávokat tükrözik (`data.lanes`).
- A sáv megengedett irányait egy `<q-select>` elemben lehet kiválasztani; a vezérlő több irány kiválasztását is megengedi. Az egyes irányokat az **applicableLaneDirections** Record (kulcs-érték párok listája) képviseli a vezérlőben, ahol a megjelenített magyar név („Egyenesen”, „Balra”, „Jobbra”) stringje egy rekord kulcsa, a **LaneDirection** irányt reprezentáló felsorolástípus pedig az értéke. A lista frissülésekor az **updateDirections(index: number, labels: string[])** függvény az **applicableLaneDirections** kiválasztott kulcsait és a szerkesztett sáv indexét használva a rekordlista megfelelő értékeivel tölti fel a szerkesztett tábla (**sign** reaktív ref-objektum értéke) adatainak megfelelő indexű sávjához tartozó megengedett irányainak halmazát. Legalább egy irányt ki kell választani, ha kiürül a kiválasztott irányok listája, automatikusan bekerül az egyenes irány („Egyenesen”/LaneDirection.FORWARD).
- A sáv típusát hasonló módon egy `<q-select>` elemmel lehet megadni, mely ugyanúgy kulcs-érték párok egy listájával működik. A kulcsok itt is a felhasználó számára könnyen olvasható stringek: „Általános”, „Kerékpár”, „Busz”, „Busz (kerékpárral)”. A különbség annyi, hogy mivel egy sávnak egy típusa lehet, csak egy elemet lehet egyszerre kiválasztani és a kiválasztást végrehajtó **updateType(index: number, label: string)** is csak egy rekordkulcsot vár.
- A panel jobb oldalán lévő, kukát ábrázoló ikont tartalmazó gomb törli a `<q-list>` adott eleméhez társított index helyén lévő sávot a tábla adatai közül (tehát a gombot tartalmazó vezérlőpanel sávját).
- **Kód:** JL („Fényjelző készülék(ek)“)

- **Szerkesztő: TrafficSignalSetEditor**

- Attribútumként a szerkesztett táblát TrafficSignalSet típusúvá konvertálva kapja meg. A jelzőlámpa-csoport adataiban szereplő egyes lámpák listáját egy `<q-list>` HTML-elemben jeleníti meg. A listában minden egyes jelzőlámpa rendelkezik egy saját vezérlőpanellel, ami az adott indexű jelzőlámpa típusának kiválasztását, valamint törlését teszi lehetővé.
 - A jelzőlámpa típusát egy `<q-select>` elemmel lehet megadni, mely kulcs-érték párok egy listájával működik, ahol a megjelenített magyar név (az adott jelzőlámpa fajtájának megnevezése) stringje egy rekord kulcsa, a TrafficSignal jelzőlámpa fajtáját reprezentáló felsorolástípus pedig az értéke. Egyszerre egy értéket lehet kiválasztani. Az `updateType(index: number, label: string)` frissíti a kiválasztott listaelem kulcsa és a szerkesztett jelzőlámpa indexe alapján a modellt.
 - A panel jobb oldalán lévő, kukát ábrázoló ikont tartalmazó gomb törli a `<q-list>` adott eleméhez társított index helyén lévő jelzőlámpát a jelzőlámpa-csoport adatai közül (tehát a gombot tartalmazó vezérlőpanel lámpáját).

3.4. Tesztelés

A szoftver teszteléséhez két fő módot választottam:

- Az alkalmazás tesztelése „given-when-then” formájú felhasználói történetekkel.
- Az elkészített és táblázott tervrajz pontosságának tesztelése, hibák és pontatlanságok kimutatása.

3.4.1. A tervrajz táblázási pontosságának és pontatlanságainak tesztelése

Bár a tervrajz modellje és a táblázó program sok helyzetet felismer és helyesen tábláz, működése korántsem tökéletes. A táblázó program jóságának a mértéke nehezen állapítható meg egy konkrét értékként, és ennek a „jóságnak” a definiálásához is számos dolgot meg kell fontolni:

- **Mennyire vehetjük viszonyítási alapként a valós táblázást?** Egyrészt az utak igazi táblázása lehet hiányos vagy rossz, másrészt rengeteg opcionális szabály van, amikor

egy tábla elhelyezhető, de nem elhelyezendő. A veszélyt jelző táblákat a táblázó program elhelyezi mindig a veszélyek előtt, ha az út hossza eléri az előjelzési távolságot, de a legtöbb veszélyt jelző tábla mind az útügyi műszaki előírás, mind a táblák elhelyezésére vonatkozó jogszabály alapján elhelyezhető, de nem elhelyezendő. [1][18] Egy erősen forgalomcsillapított, alacsony sebességhatárú területen valószínű, hogy nem helyeznének el egy kijelölt gyalogos-átkelőhelyre figyelmeztető „Gyalogátkelés” táblát, de ugyanakkor ez nem tilos, szóval nem helytelen a táblázó program viselkedése.

- **Hogyan mérjük a heurisztikák helyességét?** Vannak bizonyos helyzetek, amikor a táblázó program valamilyen szabályt nem egy jogszabályban vagy előírásban található pont alapján határoz meg, hanem legalább részben a jelzőtáblák általános elhelyezési szokásai alapján, amik nem szerepelnek explicit módon leírva. Ilyen például a jelzőtáblák precedenciája: bár egy-két sorrendre van szabály, például a „Körforgalom” tábla helyére az „Elsőbbségadás kötelező” jelzőtábla alatt (JETSZ [1] 11.3. *af*) pont), de más táblák esetén ez nincs így. Nem találtam külön szabályt arra, hogy a veszélyt jelző táblák a tilalmi és korlátozó táblák felett legyenek, sem arra, hogy a „Főútvonal vége” jelzőtábla egyéb elsőbbséget szabályzó táblák felett legyen, de az esetek nagy többségében mégis így van, ezért ezt számon tartja a táblázó program. Más esetekben viszont a heurisztika szembe mehet a szabályokkal: a „Főútvonal” táblát lakott területen belül elég a becsatlakozó alárendelt utakról belátható helyekre, de legfeljebb 3 kereszteződésenként, lakott területen kívül azonban minden szilárd útburkolatú kereszteződés után el kell helyezni (JETSZ [1] 11.1. *a*) pont). A „Főútvonal” táblák megismétlését végül úgy oldottam meg, hogy akkor helyezi el a program a táblát, ha a kereszteződés után 80 méteren belül nem talál másikat. Nem tudhatjuk, hogy tényleg mennyire belátható a kereszteződésből a következő „Főútvonal” tábla, így akár minden kereszteződés után is elhelyeztethettem volna, de a lakott területen belül akár egymástól 10-20 méteren belül elhelyezkedő keresztezések után mindig kitenni a „Főútvonal” táblát sehogy nem lett volna reális a legtöbb esetben, tehát így közelítettem meg a felesleges táblák elvetését. Ez a heurisztika, bár lakott területen kívül minden szilárd burkolatú úttal kereszteződés után elhelyezendő és ugyanúgy minden kereszteződés után elhelyezhető a tábla, érvényes külterületi részekre is, mivel az esetek nagy többségében ha 80 méteren belül két út is csatlakozik külön ponton egy

főúthoz, az valószínűleg egy út csatlakozása középszigetekkel vagy egy eltolt útkereszteződés (két becsatlakozó ág a főútba ugyanannak a csomópontnak a része, csak pár méterrel odébb csatlakoznak), így ez egy jó heurisztika, ami lehet, hogy időnként tud rossz eredményt produkálni (pl. ha tényleg van két kereszteződés lakott területen kívüli főúton ilyen távolságon belül, de ez viszonylag ritka).

- **Hogyan mérjük a távolságok helyességét?** A távolságok az előírásban és a jogszabályban legtöbb esetben nem konkrétak. [1][18] Olyasmi intervallumok szoktak lenni, hogy az „Elsőbbségadás kötelező” vagy „ÁLLJ! Elsőbbségadás kötelező” táblát a keresztező utak széleinek metszéspontjától legfeljebb 8 méterre lehet elhelyezni (e-UT 04.02.11. [18] 60.1. *bb*) pont), vagy a veszélyt jelző táblák elhelyezésére vonatkozó tág intervallumok: lakott területen belül 50-100, lakott területen kívül 150-250 méterre a veszélyforrástól (JETSZ [1] 8.1.1. pont). A „Főútvonal” kereszteződés utáni elhelyezésének távolságára pedig már semmilyen intervallum nincs. Ugyanakkor egyértelmű, hogy a tervrajz nem alkalmas távolságok különösen pontos mérésére, hiszen például az utaknak sincs szélessége, a jelzőtáblák távolságát oldalra az úttól pedig ezért nem is lehet értelmezni, így út menti távolságoknál is elfogadható a megközelítőleg jó távolság; de szerencsére a távolságokat illető laza jogszabályi és előírásbeli háttér miatt könnyű szabályosnak maradni.

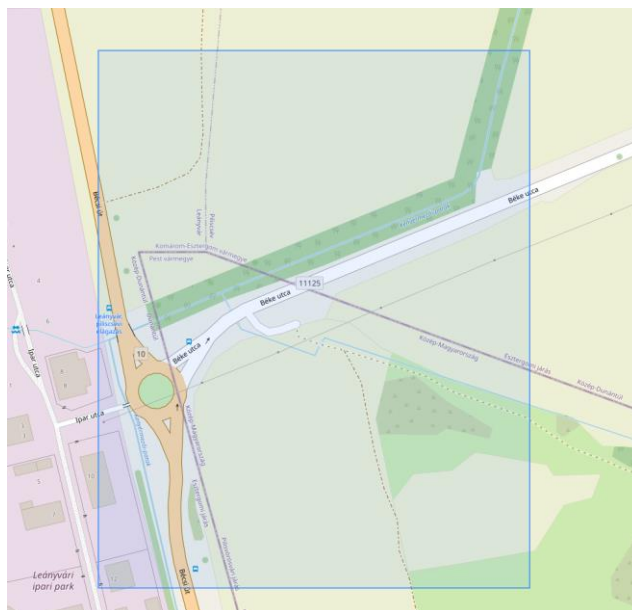
A fenti kérdések megfontolását követően Magyarországon található kiválasztott úthálózati részletek alapján fogom a szerver képességét értékelni adott térképrészletek táblázott tervrajzzá alakítására. A tervrajz területének kiválasztását, majd elemzését az alkalmazáson keresztül végzem. Az egyes kiválasztott részek tervrajzáról a fenti három kérdéshez tartozó gondolatok figyelembevételével szöveges értékelést készítek, mely röviden összegzi a táblázás pontosságát, amibe a táblázó program által nem ismert táblák (tehát minden olyan jelzőtábla, mely nincs jelen a 3.2.5.6.4. fejezet listájában) hiánya nem számít bele. Ami pontatlanságnak számít, az

- a táblázó program egy ismert táblájának hiánya ott, ahol kellene (racionális heurisztikára hivatkozás nélkül),
- egy jelzőtábla jelenléte ott, ahol nem kéne lennie,
- vagy egy jelzőtábla egyértelműen rossz távolságban vagy pozícióban (pl. fordítva, kereszteződés rossz oldalán) elhelyezve.

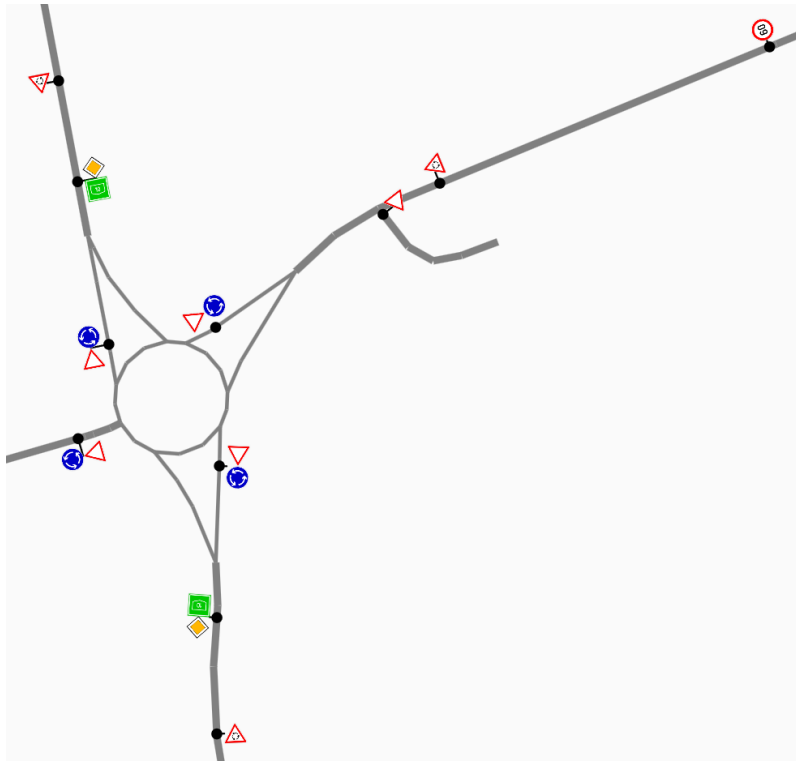
A jól működő részeknél nem részletezek minden aktív szabályt és heurisztikát, ellenben a hibáknál mindig megpróbálok magyarázatot adni a pontatlanságra.

3.4.1.1. 1. tervrajzteszt

Egy körforgalmat, és annak környezetét választom ki a 10-es számú főúton Leányvár mellett (37. ábra).



37. ábra: 1. tervrajz kiválasztott területe térképen



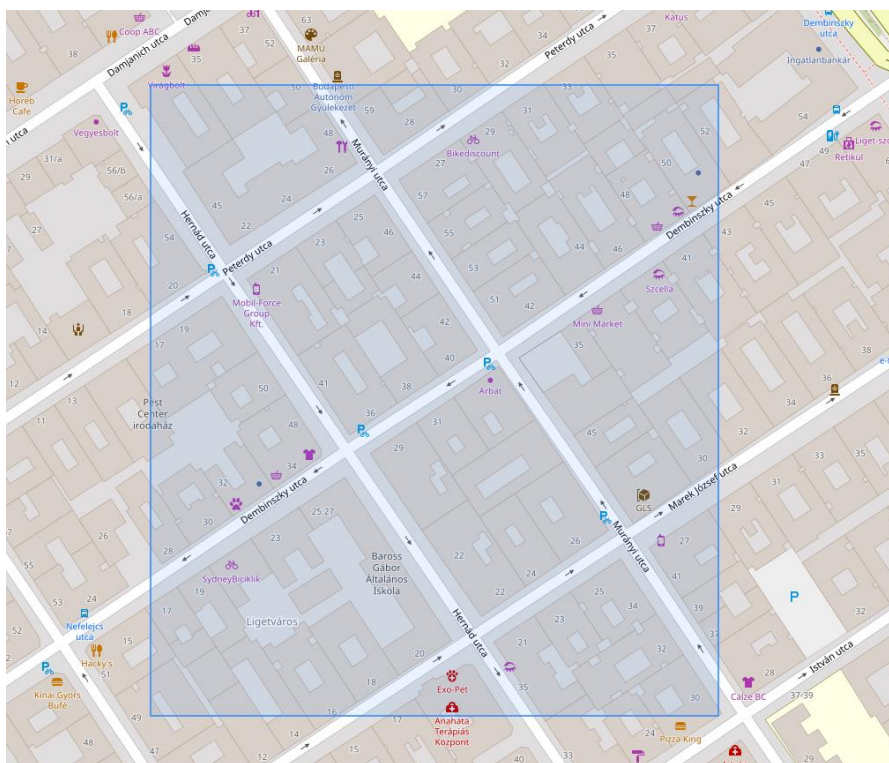
38. ábra: 1. tervrajz megjelenítése az alkalmazásban

Az eredmény (38. ábra) majdnem teljesen pontos. Az elhelyezendő táblák mind el vannak helyezve: a körforgalom bejáratainál az „Elsőbbségadás kötelező” és alatta „Körforgalom” jelzőtáblák helyesen el vannak helyezve, a 10-es főútvonal „Főútvonal” táblája és a számpajzsa is megvan, a körforgalom három irányból elő van jelezve, a negyedikben a tervrajz véget ér az előjelzési távolság előtt, így tábla nem került elhelyezésre. A csonka becsatlakozó úton a jobbra menő útra „Elsőbbségadás kötelező” tábla lett helyesen elhelyezve, és a szintén jobb oldali ágon a körforgalom előtti (tervrajz területén egyetlen) 60 km/h-s sebességkorlátozás is tükröződik a táblázáson.

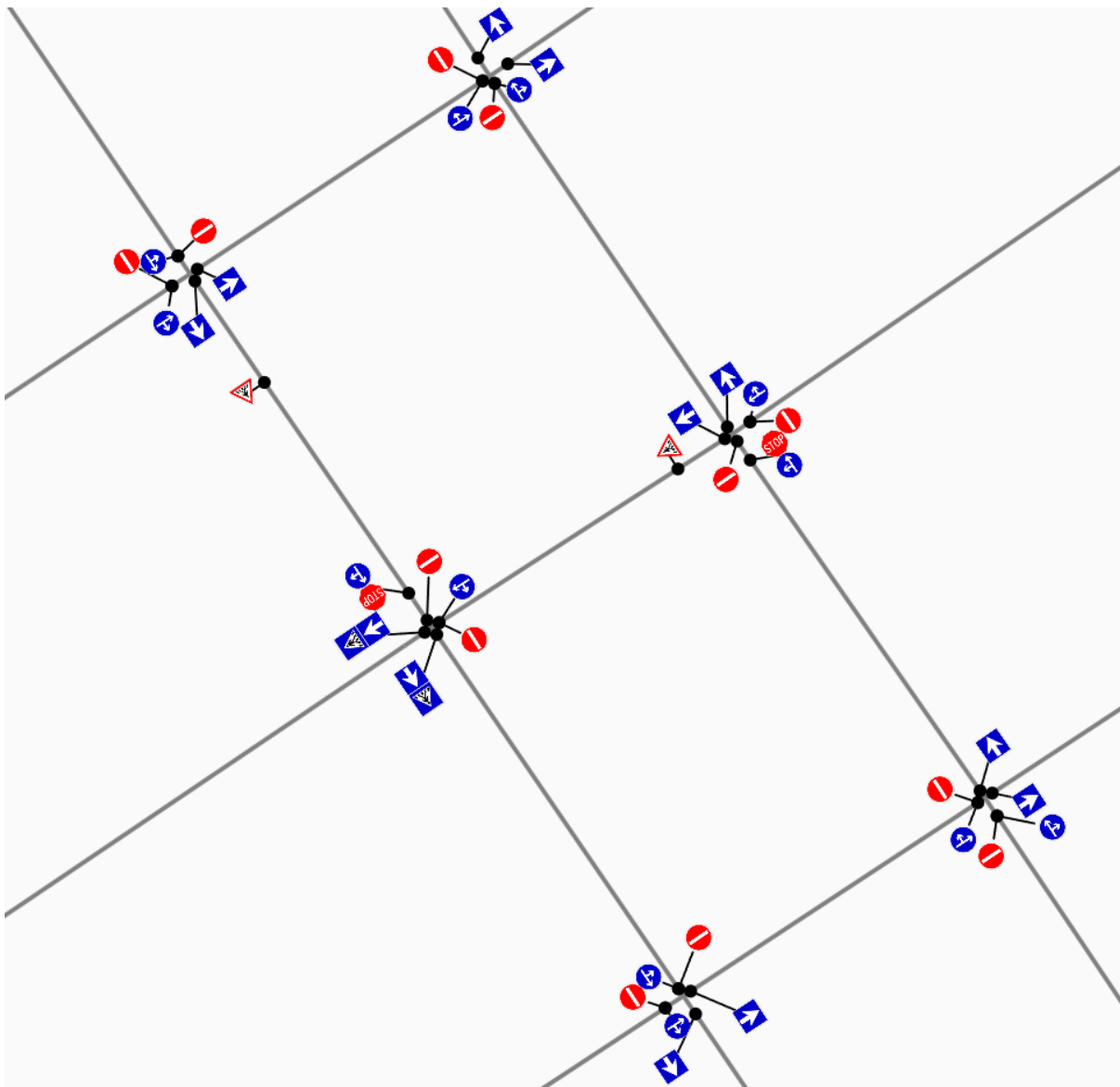
Az egyetlen hiba az az, hogy a körforgalom bal oldali ágának útját Ipar utcának hívják, az „utca” elnevezés pedig heurisztika a lakott területre, így a tervrajzon a körforgalmat előjelző tábla a körforgalomtól csupán 75 méterre lett letéve, ami a veszély előjelzési távolsága lakott területen belül, és nem 150-250 m-re, ahogy a JETSZ 8.1.1. b) pontja megszabja. Ezt a hibát egyébként az alkalmazásban könnyen javíthatja a „Körforgalom előjelzése” táblák törlésével és újak elhelyezésével messzebb.

3.4.1.2. 2. tervrajzteszt

Budapest belvárosának (a Dózsa György út közelében) egy egyirányú utcákból álló részletét alakítom át tervrajzzá (39. ábra).



39. ábra: 2. tervrajz kiválasztott területe a térképen

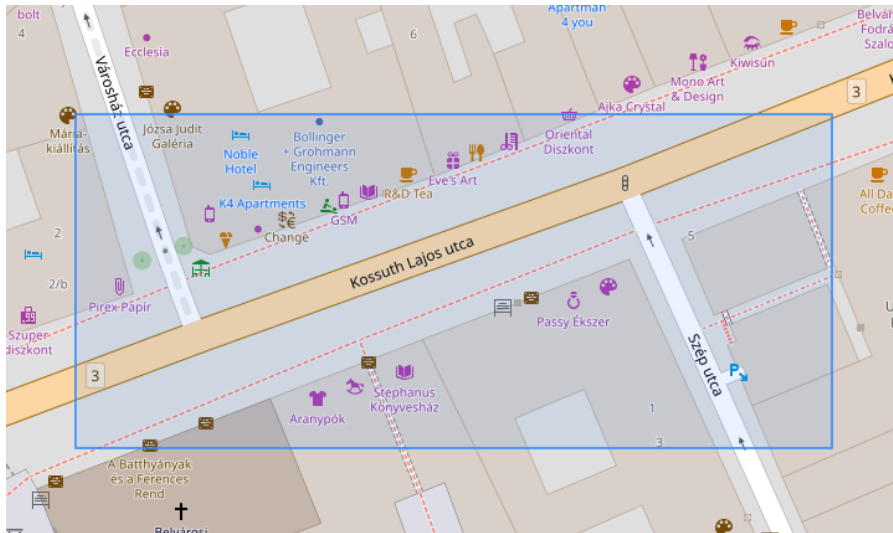


40. ábra: a 2. tervrajz megjelenítése az alkalmazásban.

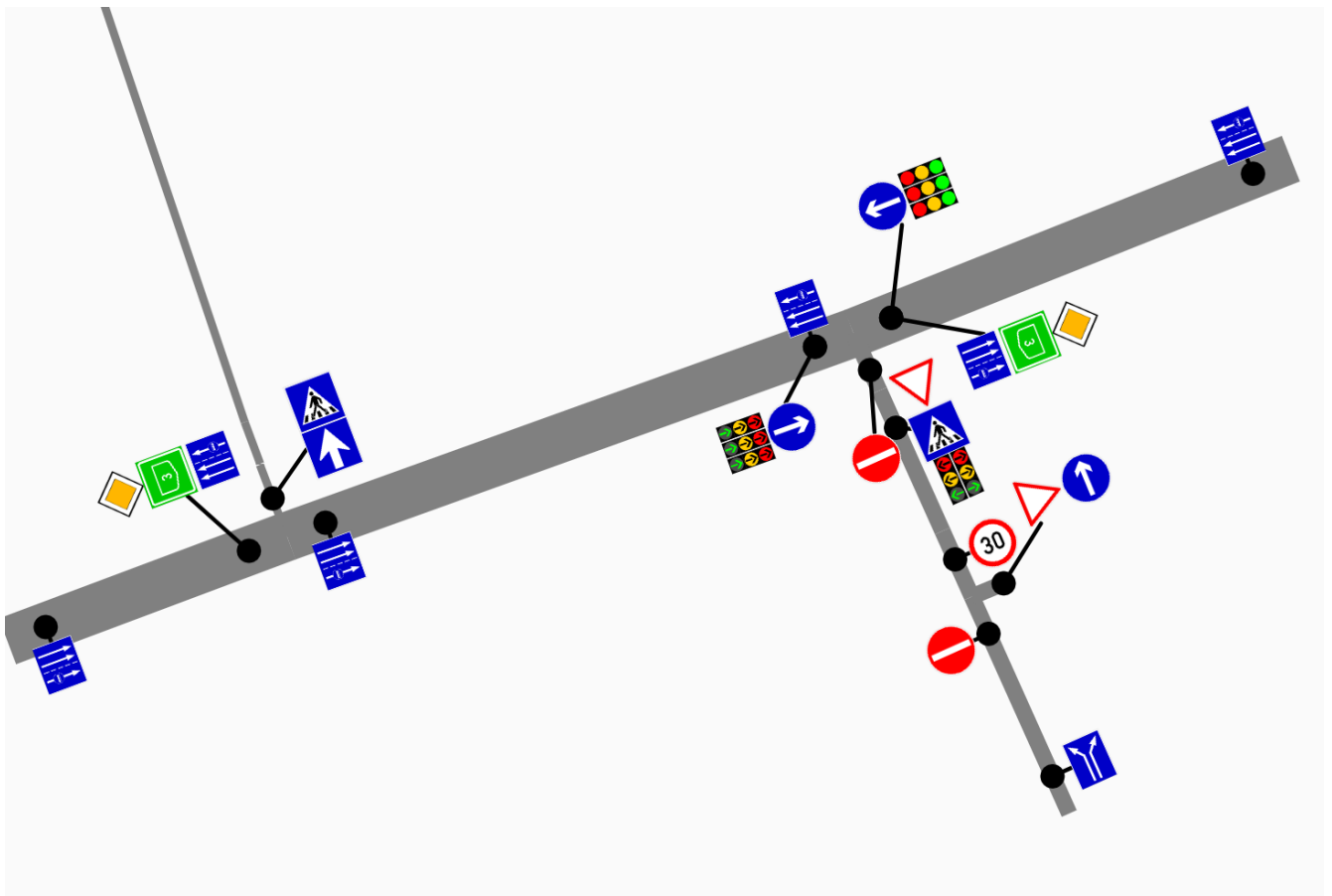
Ezen a tervrajzon (40. ábra) a táblázó program hiba nélkül táblázott. Az egyes egyirányú utcáknál helyesen el vannak helyezve egyik végen az „Egyirányú forgalmú út”, másik végen a „Behajtani tilos” táblák. A „Kötelező haladási irány” táblák csak megengedett irányokból érkezőknek vannak elhelyezve, ugyanez igaz a „Kijelölt gyalogos-átkelőhely”, „Gyalogosátkelés” és „ÁLLJ! Elsőbbségadás kötelező” táblákra (a csak kerékpárosoknak engedélyezett irányokat nem számítva; az nem a tervrajz modelljének része, de jó lehetőség továbbfejlesztéshez). Ahol elsőbbségadási hierarchia van, ott el vannak helyezve az „ÁLLJ! Elsőbbségadás kötelező” táblák, az egyenrangú kereszteződések nem táblázta ki hibásan a transzformációs gráf elsőbbségadási kötelezettséget címkéző heurisztikája (3.2.5.4.2. fejezet listájának 1. pontja).

3.4.1.3. 3. tervrajzteszt

Szintén Budapest belvárosában a Kossuth Lajos utca Astoria és Erzsébet híd közti szakaszából választottam egy részletet (41. ábra).



41. ábra: 3. teszt területe térképen



42. ábra: a 3. teszttervrajz

Az elkészült tervrajz (42. ábra) nagyrészt pontos.

- A jobb oldalt a becsatlakozó úton táblával jelezve van a besorolási rend.
- Jelezve van „Behajtani tilos” jelzőtáblával az egyirányúság is.
- A „Sebességkorlátozás” tábla is helyes, bár a valóságban itt övezetet használnak (de a program nem ismeri az övezeteket).
- A jobb oldali mellékutcába csatlakozó mélygarázs kijárata el van látva megfelelő „Kötelező haladási irány” és „Elsőbbségadás kötelező” táblával.
- A széles úton haladó két általános sáv és egy buszsáv helyesen jelezve van sávbesorolás rendjét jelző táblával.
- A Kossuth Lajos utca főútvonal jellege jelezve van.
- A jobb oldali lámpás kereszteződésben a helyesek a jelzőlámpák: a főúton csak egyenesen lehet haladni, és egyébként nincs további elsőbbségadási kötelezettség, ez jelezhető nyíl alakú lencsével, de megfelelő „Kötelező haladási irány” tábla mellett telizölddel is (FISZ [36] 6.3.3. pont): a valóságban az utóbbi módszert használták (43. ábra), de akár a fővárosban is találhatunk példát az előbbire is. A heurisztika egyik irányban kör, másik irányban nyíl alakú jelzésre jutott. Még egy különbség, hogy a tervrajz mindhárom sávhoz létrehoz egy-egy jelzőlámpát, míg a valódi helyen csak kettő van: ez a tervrajz részéről lehet redundáns, de nem tilos minden sávhoz jelzőlámpát helyezni (több azonos irányítású forgalmi sáv esetén az a kikötés, hogy legalább egy ismétlő jelző kell, FISZ [36] 8.6. pont).
- A jobb oldali lámpás kereszteződésben a becsatlakozó ágon egy kompozícióra van elhelyezve az „Elsőbbségadás kötelező”, „Kijelölt gyalogos-átkelőhely” tábla és a jelzőlámpacsoport. Megfelelő a sorrend: minden jelzőtábla a lámpák felett van, legfelül pedig az elsőbbséget szabályozó tábla. A „Kijelölt gyalogos-átkelőhely” jelzőtábla nincs kihelyezve ezen a helyen a valóságban, de mivel a jelzőlámpa mögött valóban van egy átkelő, ez egyáltalán nem tiltott, sőt, ha a lámpa bármikor tartósan üzemben kívül van, kötelező kihelyezni a táblát (FISZ [36] 20.5. pont).



43. ábra: A Kossuth Lajos utca-Szép utca csomópont lámpázása az Erzsébet híd felé (a 37. ábra tervrajzán a jobb oldali kereszteződés a főút mentén jobbról). A valóságban csak két lámpa van a három sávra

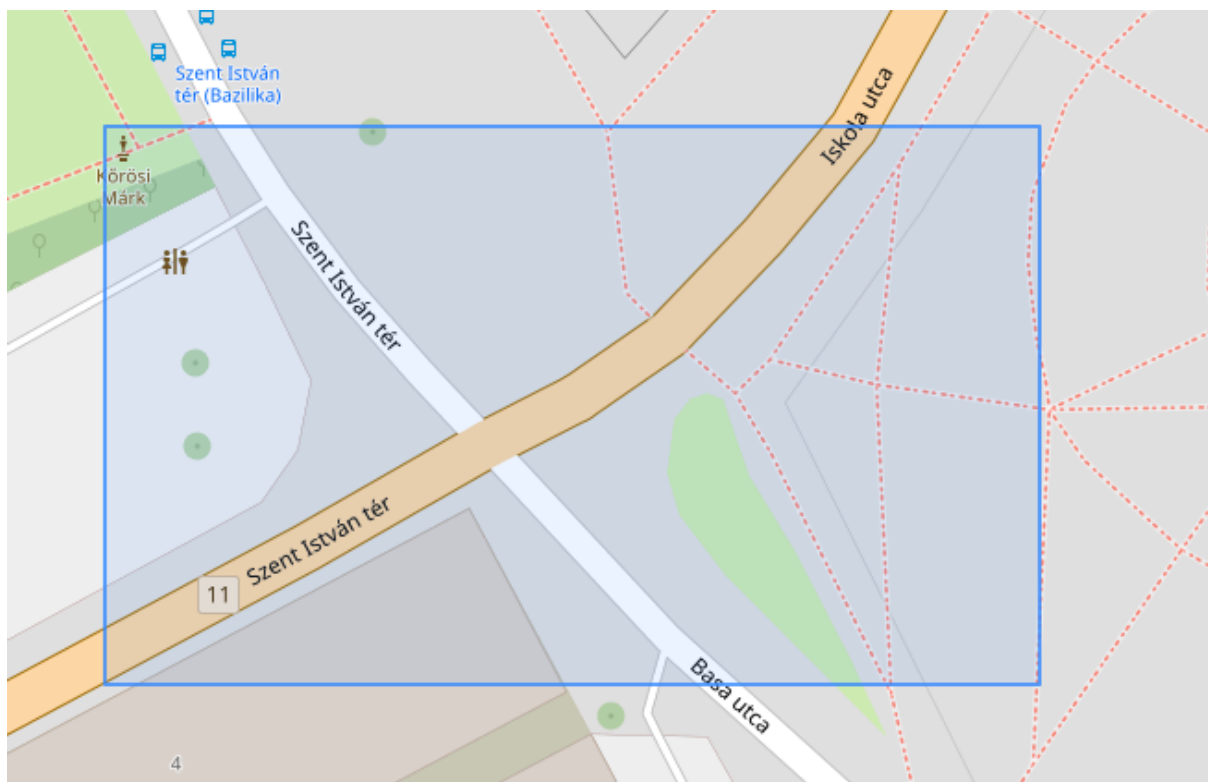
Azonban van néhány pontatlanság is a tervrajzban:

- A valóságban Budapest belső részein nincs számpajzzsal jelölve a főutak száma, de ez egy nagyon specifikus és konvenció, amire pontos meghatározást sem láttam. Azonban mivel a Kossuth Lajos utca tényleg a Clark Ádám térről induló 3-as főút része, nem hibás a táblázás.
- A bal oldali kereszteződésben, bár a sávok rendjét jelző táblák mutatják, hogy egy sávból sem lehet balra kanyarodni a főúton balról érkezőknek, hiányzik egy „Kötelező haladási irány” tábla.

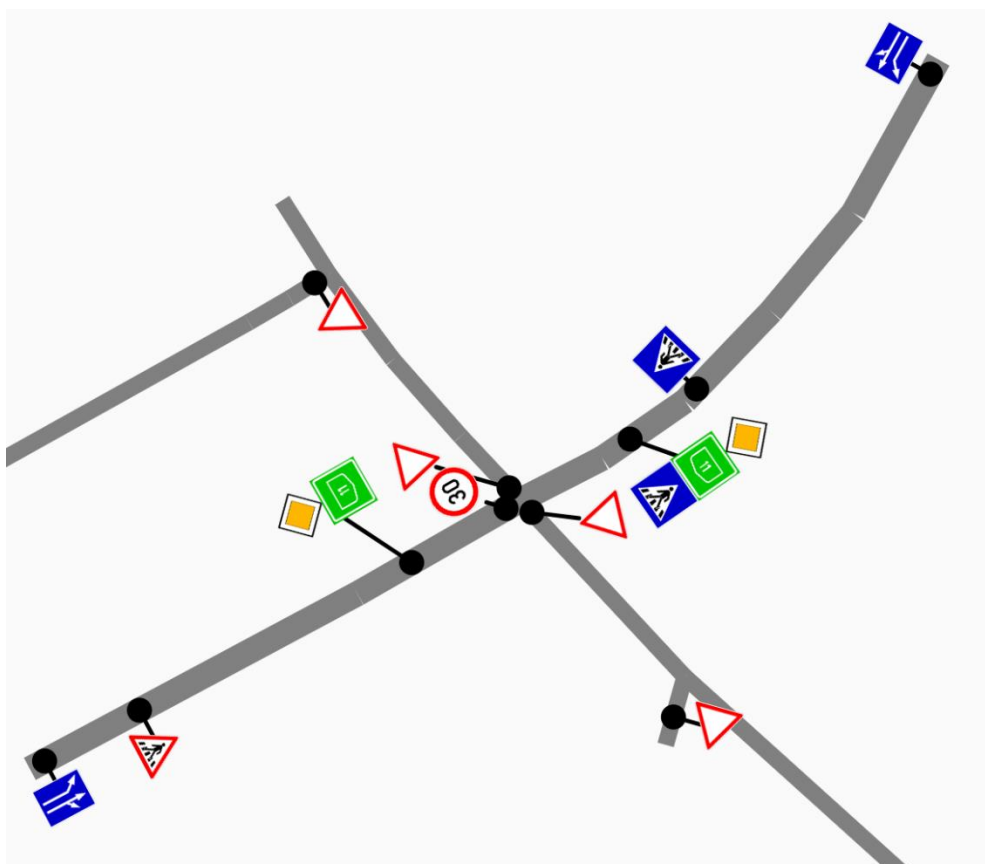
Ezek a pontatlanságok a tervrajz szerkesztésével az alkalmazásban könnyen javíthatók.

3.4.1.4. 4. tervrajzteszt

A 11-es főútvonal mentén Esztergom egyik kereszteződését vizsgálom meg (44. ábra: a 4. tervrajzteszt területe a térképen).



44. ábra: a 4. tervrajzteszt területe a térképen



45. ábra: a 4. teszt tervrajza

A teszt tervrajza (45. ábra) itt is nagyrészt pontos.

- Minden szükséges „Elsőbbségadás kötelező” jelzőtáblát elhelyezett. Bár az OpenStreetMap gyakran jelöl elsőbbségadási kötelezettséget „give_way” vagy „stop” címkéjű pontokkal, ezen a környéken nincsenek jelen a jelölések, ami azt jelenti, hogy az elsőbbségadási kötelezettségeket a transzformációs gráf helyezte a tervrajzra az utak precedenciájának ellenőrzésével a csomópontokban, helyesen.
- A sávbesorolás rendjét jelző táblák helyesen tükrözik a 11-es főúton az egyes sávok megengedett irányait.
- A 11-es főútvonal főút jellege jelezve van, számpajzzsal együtt.
- A kijelölt gyalogos-átkelőhely jelezve és előjelezve van.

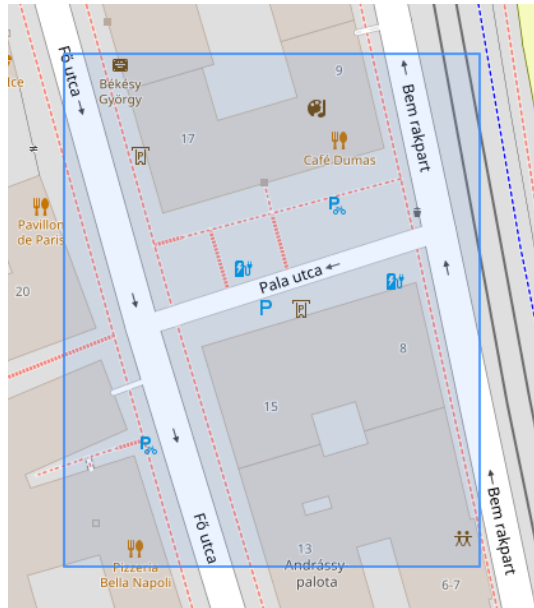
Ennek a tervrajznak egy felesleges és egy hiányzó jelzőtáblája van, mindkettő az OSM-adatok pontatlanságából ered (a teszt 2026 április 28-án készült, azóta lehet, hogy javításra került):

- A 11-es főúton a kereszteződéstől balra el van helyezve egy 30-as sebességkorlátozás, ami nincs igazából jelen. Az OpenStreetMap ezen az útszakaszon valóban 30-as korlátot jelez. Ilyen „fantom-sebességkorlátozások” viszonylag gyakoriak az OSM-adatokban.
- A 11-es útba csatlakozó, balra fel haladó úton viszont van 30-as korlátozás, de az viszont az OpenStreetMapen nincs jelölve (50-es korlátozást jelez, ami a standard lakott területen belül, így nincs rá jelzőtábla a tervrajzon).

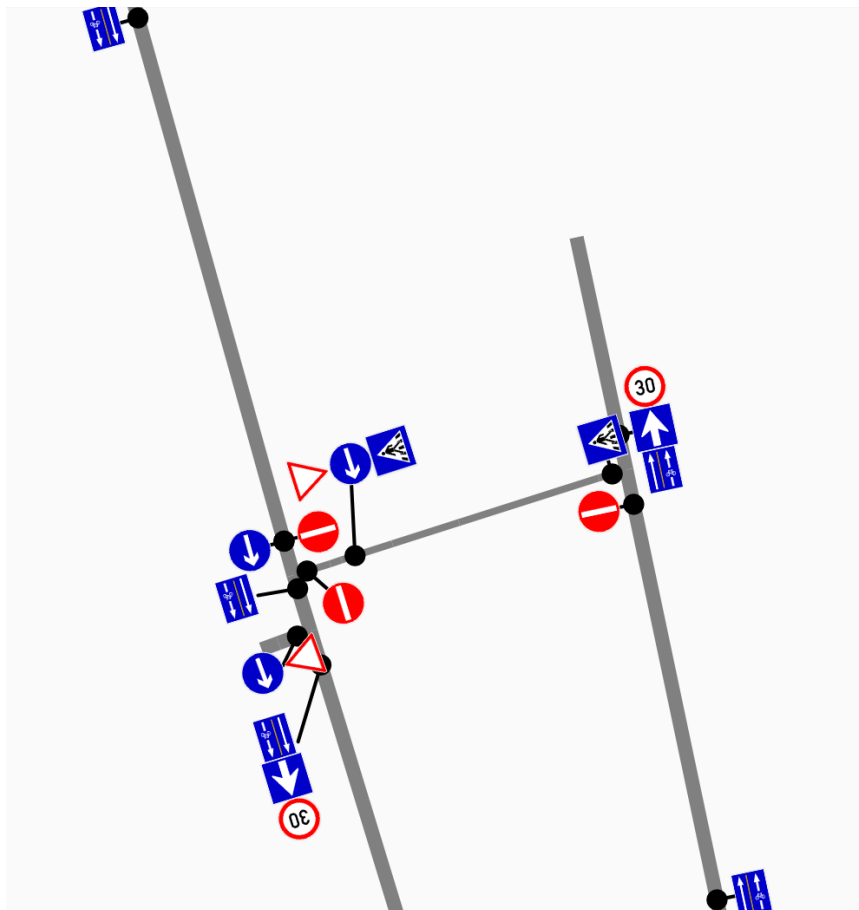
A fent felsorolt hibákat a Tervrajz lapon az alkalmazásban itt is egyszerűen lehet javítani a felesleges „Sebességkorlátozás” tábla törlésével és újak hozzáadásával a fenti pontban említett útra.

3.4.1.5. 5. tervrajzteszt

Most Budapesten a Fő utca és a vele párhuzamos Bem rakpart egy részletét vizsgálom meg (46. ábra).



46. ábra: az 5. teszt területe a térképen



47. ábra: 5. teszt tervrajza

A tervrajzon (47. ábra) megnézhetjük azt, hogy hogyan kezel a táblázó program kerékpársávokat.

- Mint a 2. tesztenél, az egyirányú utcákat szabályzó „Egyirányú forgalmú út”, „Behajtani tilos” és „Kötelező haladási irány” jelzőtáblák következetesen vannak elhelyezve.
- A „Sebességkorlátozás” jelzőtáblák helyesen vannak elhelyezve, csak azokban az irányokban, ahol van engedélyezett forgalom.
- Az alsóbbrendű utca és a garázs kijárat csatlakozásánál van „Elsőbbségadás kötelező” jelzőtábla.

Egy dolgot azonban kiemelnék, ami pontatlanság:

- A tervrajzon a vízszintes irányú utca két végén látunk „Kijelölt gyalogos-átkelőhely” táblákat, miközben valójában ott nincsenek. Ez azért van, mert a tervrajzon tényleg fel vannak címkézve az utca két végén a foglalatok a „CROSSING” címkével, ugyanis az OSM-adatok gyakran a járda meghosszabbított vonalát egy utcára merőlegesen átkelőként tüntetik fel. Ezt az alkalmazásban a „Kijelölt gyalogos-átkelőhely” táblák törlésével könnyen lehet orvosolni.

3.4.1.6. Tesztek összegzése

Látható, hogy sok helyzetet jól kezel a táblázó program, de bonyolultabb helyzetekben képes hibákat véteni, amelyek forrása lehet a táblázó program működésének rendellenessége, valamelyik heurisztika az OSM-adatok elemzésekor, vagy akár maguknak az OSM-adatoknak a helytelensége. Azonban összességében még mindig állítható, hogy a szoftver nagyrészt jól tábláz, a prezentációs komponens az adatok alapján minden esetben helyesen készíti el a jelképeket, és a szerver tervrajzkomponensének hibái viszonylag könnyen javíthatók az alkalmazásban.

3.4.2. Az alkalmazás tesztelése felhasználói történetekkel

Az alkalmazás teszteléséhez tesztelhető felhasználói történeteket használtam. A felhasználói történetek egy **tételmondat**tal kezdődnek, melyek formája:

Mint [cselekvő], szeretnék [tevékenységet végrehajtani], hogy [valami megvalósuljon].

A tételmondat arra szolgál, hogy tisztázza, hogy mi a motiváció a tételmondathoz tartozó jelenetek mögött. Minden tételmondat után egy vagy több jelenet leírása található. A jelenetek tesztelhetőségét a rendszerek teszteléséhez gyakran „given-when-then” struktúra biztosítja, mely rendelkezik egy konkrét adott kontextussal, mint kiindulási pont (**given**), egy

tesztelésként végrehajtható cselekvéssel vagy cselekvéssorozattal (**when**) a cselekvő szemszögéből, és az annak következtében elvárt végállapottal vagy eseménnyel (**then**). Ennek a struktúrának a három eleme tesz ki egy jelenetet, melynek szükségessége a teszteléshez visszaellenőrizhető a tételmondatokkal, hogy a cselekvő szerepéhez illik-e a cselekvés (a „when” rész), és hogy a cselekvő motivációjával egybevág-e az elvárt esemény vagy állapot (a „then” rész).

A „when” részben leírt cselekvéseket a leírt sorrendben kell elvégezni a teszteléshez, a többi résznél a felsorolt állapotok vagy események sorrendje nem számít.

Az alkalmazás esetén nincsenek különböző szerepű felhasználók, így az egyetlen cselekvőtípus a „felhasználó”.

3.4.2.1. Kijelölés térképen

Mint felhasználó, szeretnék kijelölni a térképen egy területet.

3.4.2.1.1.1 1. jelenet

- **Given:**
 - Az alkalmazás fut.
 - Az alkalmazás eléri az internetet.
 - A Térkép lapon vagyok, böngésző módban.
- **When:**
 - A Térkép lap térképén a kijelölni kívánt területre navigálok.
 - Rajzoló módba kapcsolok.
 - A bal egérgomb nyomva tartása közben kijelölök egy téglalapot.
- **Then:**
 - A térképen megjelenik a kijelölt terület, a vezérlőpanelen elérhetővé válik a kijelölt terület kiválasztása.

3.4.2.2. Kijelölt terület kiválasztása

Mint felhasználó, szeretnék kiválasztani egy, a térképen kijelölt területet.

3.4.2.2.1 1. jelenet

- **Given:**
 - Az alkalmazás fut.

- Az alkalmazás eléri az internetet és a szerveret.
- A szerver eléri az Overpass API-t.
- A Térkép lapon vagyok, rajzoló módban, kijelöltem egy területet.
- **When:**
 - Rányomok a „Terület kiválasztása” címkéjű gombra.
- **Then:**
 - Az alkalmazás átirányít a Tervrajz lapra, amin megjelenik a kiválasztott területről készült tervrajz.

3.4.2.2.2 2. jelenet

- **Given:**
 - Az alkalmazás eléri az internetet.
 - Az alkalmazás nem éri el a szerveret.
 - A Térkép lapon vagyok, rajzoló módban, kijelöltem egy területet.
- **When:**
 - Rányomok a „Terület kiválasztása” címkéjű gombra.
- **Then:**
 - Egy modális ablak tájékoztat a tervrajz betöltésének sikertelenségéről, melynek üzenete leírja, hogy az alkalmazás nem tudott választ szerezni a szerver felől.
 - Rajzoló módban maradok.
 - A kijelölt terület megmarad.
 - Az ablak bezárását követően lehetőség van a terület kiválasztásának újbóli megpróbálására.

3.4.2.2.3 3. jelenet

- **Given:**
 - Az alkalmazás eléri az internetet és a szerveret.
 - A szerver a kijelölt területet nem képes feldolgozni (az Overpass API elérhetetlenségének, feldolgozható utak hiánya vagy a szerver kódjának hibája miatt).
 - A Térkép lapon vagyok, rajzoló módban, kijelöltem egy területet.
- **When:**
 - Rányomok a „Terület kiválasztása” címkéjű gombra.

- **Then:**
 - Egy modális ablak tájékoztat a tervrajz betöltésének sikertelenségéről, melynek üzenete leírja a szerver által küldött válasz alapján, hogy miért volt sikertelen a művelet.
 - Rajzoló módban maradok.
 - A kijelölt terület megmarad.
 - Az ablak bezárását követően lehetőség van a terület kiválasztásának újbóli megpróbálására.

3.4.2.3. Tervrajz mentése

Mint felhasználó, szeretnék elmenteni egy tervrajzot, hogy az alkalmazás máskori futtatása során egy fájlból betölthető legyen.

3.4.2.3.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok.
 - Az alkalmazás tárol egy tervrajzot, amit megjelenít a Tervrajz lapon.
- **When:**
 - Rákattintok a Tervrajz lap vezérlőpanelén a „Tervrajz mentése” feliratú gombra.
 - A megjelenő ablakban kiválasztom a fájl mentési helyét és a nevét.
- **Then:**
 - Az fájl mentésére szolgáló ablakban megadott helyen és névvel létrehozásra kerül egy, a tervrajzot leíró JSON-fájl.
 - A létrehozott fájl használható a mentéskor tárolt tervrajz betöltésére.

3.4.2.4. Tervrajz betöltése

Mint felhasználó, szeretnék betölteni egy tervrajzot, hogy megtekinthessem és szerkeszthessem a Tervrajz lapon.

3.4.2.4.1 1. jelenet

- **Given:**
 - Nem töltöttem be tervrajzot (területkiválasztással vagy fájl megnyitásával).
 - A Tervrajz lapon vagyok.

- **When:**
 - Rákattintok a „Jelenleg nincs mutatható tervrajz” üzenet alatt opcióként felkínált „Tervrajz betöltése” gombra.
 - Kiválasztok a megjelenő fájlböngésző ablakban egy JSON-fájlt, mely egy tervrajzot tartalmaz (pl. egy korábban elmentett tervrajz).
- **Then:**
 - Az alkalmazás tárolja a kiválasztott fájlból beolvasott tervrajzot.
 - A Tervrajz lap frissül, megjeleníti a betöltött tervrajzot.

3.4.2.4.2 2. jelenet

- **Given:**
 - Van alkalmazásban tárolt tervrajz.
 - A Tervrajz lapon vagyok, megjeleníti a tárolt tervrajzot.
- **When:**
 - Rákattintok a vezérlőpanelen a „Tervrajz betöltése” gombra.
 - Kiválasztok a megjelenő fájlböngésző ablakban egy JSON-fájlt, mely egy tervrajzot tartalmaz (pl. egy korábban elmentett tervrajz).
- **Then:**
 - Az alkalmazás által tárolt tervrajz lecserélődik a kiválasztott fájlból beolvasott tervrajzra.
 - A Tervrajz lap frissül, a most betöltött tervrajzot jeleníti meg.

3.4.2.4.3 3. jelenet

- **Given:**
 - Van alkalmazásban tárolt tervrajz.
 - A Tervrajz lapon vagyok, megjeleníti a tárolt tervrajzot.
- **When:**
 - Rákattintok a vezérlőpanelen a „Tervrajz betöltése” gombra.
 - Kiválasztok a megjelenő fájlböngésző ablakban egy JSON-fájlt, mely nem egy tervrajzot tartalmaz, vagy egy hibásan kódolt tervrajzot tartalmaz.
- **Then:**
 - Egy modális ablak tájékoztat a sikertelen betöltésről, az üzenet leírja, hogy érvénytelen formátumú a JSON-fájl.

- A tárolt tervrajz az eddigi marad, Tervrajz lap ábrázolása nem frissül.

3.4.2.5. Jelzőtábla-kompozíció jelképeinek mozgatása a Tervrajz lapon

Mint felhasználó, szeretném mozgatni egy jelzőtábla-kompozíció képeit, hogy tisztábban lássam a tervrajzot.

3.4.2.5.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
- **When:**
 - A bal egérgomb nyomva tartásával a kurzorral odébb húzom egy jelzőtábla-kompozíció jelképeit.
- **Then:**
 - A táblakompozíció ábrája, annak minden jelképével oda helyeződik, ahova a kurzorral húztam.
 - A táblakompozíció út mentén lévő helyét jelző pont ugyanott marad.
 - A táblakompozíció nem forgatódik el, eredeti irányában marad.
 - Továbbra is egy egyenes köti össze a táblakompozíció helyét jelző fekete pontot és az áthelyezett ábráját (hacsaknem olyan közel vannak egymáshoz, hogy a vonal nem kerül renderelésre, de ekkor egyértelmű, hogy az ábra melyik ponthoz tartozik).

3.4.2.6. Jelzőtábla-kompozíció megfordítása a Tervrajz lapon

Mint felhasználó, szeretnék egy jelzőtábla-kompozíciót megfordítani az út mentén, hogy a jelzőtáblák az út ellentétes irányára vonatkozzanak a tervrajzon.

3.4.2.6.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - A tervrajz tartalmaz egy (általam megfordítani kívánt) jelzőtábla-kompozíciót.
- **When:**
 - Duplán rákattintok a táblakompozíció helyét jelölő pontra az úton.
- **Then:**
 - A kompozíció ábrája 180 fokkal elfordul a tervrajz megjelenítésén.

- A kompozíció ábrája az út (a kompozíció új iránya szerinti) jobb oldalra kerül.
- A változás rögzül a tervrajz tárolt modelljében, a Tervrajz lap újratöltésekor, vagy mentéskor az új irány kerül rögzítésre.

3.4.2.7. Jelzőtábla-kompozíció másolása a vágólapra

Mint felhasználó, szeretnék egy tervrajzon megtalálható jelzőtábla-kompozíciót a vágólapra másolni, hogy elhelyezzek azonos tartalmú új táblakompozíciókat.

3.4.2.7.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - A tervrajz tartalmaz egy (általam vágólapra másolni kívánt) jelzőtábla-kompozíciót.
- **When:**
 - Rákattintok bal egérgombbal egyszer a másolni kívánt jelzőtábla-kompozícióra.
- **Then:**
 - A lap jobb alsó sarkában megjelenik a vágólapra másolt táblakompozíció ábrája.
 - Egy új, elhelyezett jelzőtábla-kompozíció (ha nem lett törölve a vágólap) a másolt jelzőtábla-kompozíció tábláit tartalmazza.

3.4.2.8. Új jelzőtábla-kompozíció elhelyezése

Mint felhasználó, szeretnék elhelyezni a tervrajzon egy új jelzőtábla-kompozíciót, hogy céljaim szerint szerkeszsem a tervrajzot.

3.4.2.8.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
- **When:**
 - Rákattintok bal egérgombbal duplán a tervrajz úthálózatának arra a pontjára, ahova el szeretném helyezni a jelzőtábla-kompozíciót.
- **Then:**
 - A tervrajz megjelenítésén a kattintott helyen létrejön a táblakompozíció helyét jelző pont.

- A pont mellett megjelenik a táblakompozíció ábrája.
- A jelzőtábla-kompozíció egy „Elsőbbségadás kötelező” jelzőtáblát tartalmaz, ha a vágólap üres; ha nem üres, akkor a vágólapon lévő táblákat.
- A változás rögzül a tervrajz tárolt modelljében, a Tervrajz lap újratöltésekor, vagy mentéskor ott marad az új táblakompozíció.

3.4.2.8.2 2. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
- **When:**
 - Rákattintok bal egérgombbal duplán egy olyan helyre, ami nem egy út.
- **Then:**
 - Nem kerül elhelyezésre jelzőtábla-kompozíció.

3.4.2.9. Jelzőtábla-kompozíció menüjének megnyitása

Mint felhasználó, szeretném megnyitni egy jelzőtábla-kompozíció menüjét, hogy kezelhessem az egyes jelzőtábláit.

3.4.2.9.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - A tervrajz tartalmaz egy (általam szerkeszteni kívánt) jelzőtábla-kompozíciót.
- **When:**
 - Rákattintok bal egérgombbal kétszer annak a jelzőtábla-kompozíciónak az ábrájára, aminek a menüjét meg szeretném nyitni.
- **Then:**
 - Megnyílik a kiválasztott jelzőtábla-kompozíció menüje.
 - A menüben a jelzőtáblák sorrendje és tartalma megegyezik a tervrajzon kiválasztott táblakompozíció ábráján látottal.

3.4.2.10. Jelzőtábla kiválasztása a kompozíció menüjében

Mint felhasználó, szeretnék kiválasztani egy táblát a jelzőtábla-kompozíció menüjéből, hogy a vezérlőpanelen különböző műveleteket hajtsak végre vele (törlés, szerkesztés, beszúrás alá/fölé, exportálás).

3.4.2.10.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - Egy jelzőtábla-kompozíció menüje meg van nyitva, a kiválasztott jelzőtábla (aminek a jelképe sötét kerettel van ellátva a kompozíció nézetében) nem az, amit kezelni szeretnék.
- **When:**
 - Rákattintok az általam kiválasztani kívánt jelzőtábla jelképére a bal oldali nézetben.
- **Then:**
 - A sötét keret átkerül arra a jelképre, amire kattintottam.
 - A vezérlőpanel adatai frissülnek.
 - A tábla típusa frissül az imént kiválasztott jelzőtáblára
 - A gombok akciói is a most kiválasztott táblára vonatkoznak.
 - Testre szabható tábla esetén megjelenik a megfelelő szerkesztő a kiválasztott tábla adatait tükrözve.

3.4.2.11. Új jelzőtábla hozzáadása a kompozícióhoz

Mint felhasználó, szeretnék egy jelzőtábla-kompozícióhoz új táblát hozzáadni, hogy céljaim szerint szerkeszsem a tervrajzot.

3.4.2.11.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - Egy jelzőtábla-kompozíció menüje meg van nyitva.
- **When:**
 - A menü jobb oldalán lévő vezérlőpanelen rákattintok a „Tábla alá” címkéjű gombra.
- **Then:**
 - A kompozícióban kiválasztott jelzőtábla alatt megjelenik egy új, alapértelmezetten „Elsőbbségadás kötelező” jelzőtábla.
 - A menü bezárását követően megjelenik a kompozíció új táblája a Tervrajz lap ábráján.

- A táblakompozíció új állapota az alkalmazásban tárolt tervrajz modelljében is rögzül, mentéskor megmarad.

3.4.2.11.2 2. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - Egy jelzőtábla-kompozíció menüje meg van nyitva.
- **When:**
 - A menü jobb oldalán lévő vezérlőpanelen rákattintok a „Tábla fölé” címkéjű gombra.
- **Then:**
 - A kompozícióban kiválasztott jelzőtábla felett megjelenik egy új, alapértelmezetten „Elsőbbségadás kötelező” jelzőtábla.
 - A menü bezárását követően megjelenik a kompozíció új táblája a Tervrajz lap ábráján.
 - A táblakompozíció új állapota az alkalmazásban tárolt tervrajz modelljében is rögzül, mentéskor megmarad.

3.4.2.12. Jelzőtábla-kompozíció törlése

Mint felhasználó, szeretnék egy jelzőtábla-kompozíciót törölni, hogy a tervrajzról eltűnjön.

3.4.2.12.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - A tervrajz tartalmaz egy (általam törölni kívánt) jelzőtábla-kompozíciót.
- **When:**
 - Megnyitom a jelzőtáblakompozíció menüjét (3.4.2.9.1. fejezet jelenete)
 - A menüben kezdésképp kiválasztott jelzőtábla jobb oldali vezérlőpaneljén rákattintok a „Tábla törlése” címkéjű gombra
 - Megismétlem az előző lépést a törlések után következő kiválasztott táblákkal egészen addig, amíg üres nem lesz a táblakompozíció a menüben
 - Kilépek a menüből az „OK” gombbal, vagy menü ablakán kívülre kattintással
- **Then:**

- A tervrajzon már nem jelenik meg a jelzőtábla-kompozíció; sem a táblák ábrái, sem a helyét jelző pont az úton.
- A táblakompozíció törlődik a tárolt tervrajz modelljéből, így mentéskor sem marad a kompozíció.

3.4.2.13. Jelzőtábla exportálása

Mint felhasználó, szeretnék egy jelzőtáblát exportálni, hogy a szoftver keretein túlmutató célokra használjam fel (pl. kinyomtassam fényvisszavető fóliára).

3.4.2.13.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, ami épp megjeleníti a tárolt tervrajzot.
 - Egy jelzőtábla-kompozíció menüje meg van nyitva.
 - A menüben kiválasztottam az exportálni kívánt táblát (3.4.2.10.1. fejezet jelenete).
- **When:**
 - A vezérlőpanelen rányomok a tábla neve körül található „Exportálás” címkéjű gombra.
 - A fájlböngésző ablakban megadom az exportált fájl helyét és nevét.
- **Then:**
 - A fájlböngészőben megadott helyen és néven létrejön egy SVG-dokumentum az exportált jelzőtábla jelképével.

3.4.2.13.2 2. jelenet

- **Given:**
 - A Táblaszerkesztő lapon vagyok, amin a szerkesztett tábla az, amit exportálni szeretnék.
- **When:**
 - A vezérlőpanelen rányomok a tábla neve körül található „Exportálás” címkéjű gombra.
 - A fájlböngésző ablakban megadom az exportált fájl helyét és nevét.
- **Then:**

- A fájlböngészőben megadott helyen és néven létrejön egy SVG-dokumentum az exportált jelzőtábla jelképével.

3.4.2.14. Jelzőtábla feliratának módosítása

Mint felhasználó, szeretném egy felirattal rendelkező jelzőtábla szövegét módosítani, hogy az ábra a számomra megfelelő feliratot tüntesse fel.

3.4.2.14.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, egy jelzőtábla-kompozíció menüje meg van nyitva.
 - Az általam módosítani kívánt jelzőtábla van kiválasztva.
 - A kiválasztott jelzőtábla típusa egyike a következőknek:
 - „Össztömeg-korlátozás” (C-023)
 - „Sebességkorlátozás” (C-033)
 - „Számozott út azonosító táblája” (G-152)
- **When:**
 - Átírom a felirat szövegdobozának értékét.
 - Rákattintok az „Alkalmaz” címkéjű gombra.
- **Then:**
 - A menü nézetén frissül a jelzőtábla jelképe.
 - Az kompozíció menüjének bezárását követően tervrajz megjelenítésén frissül a kompozícióban a jelzőtábla jelképe.
 - Az kompozíció menüjének bezárását követően az alkalmazás által tárolt tervrajz modelljében is frissül a jelzőtábla adata.

3.4.2.14.2 2. jelenet

- **Given:**
 - A Táblaszerkesztő lapon vagyok.
 - A szerkesztett jelzőtábla típusa egyike a következőknek:
 - „Össztömeg-korlátozás” (C-023)
 - „Sebességkorlátozás” (C-033)
 - „Számozott út azonosító táblája” (G-152)
- **When:**

- Átírom a felirat szövegdobozának értékét.
- Rákattintok az „Alkalmaz” címkéjű gombra.
- **Then:**
 - Frissül a szerkesztett jelzőtábla jelképe.

3.4.2.15. Új sáv hozzáadása sávbesorolás rendjét jelző táblához

Mint felhasználó, szeretnék egy új sávot hozzáadni egy sávbesorolás rendjét jelző tábla jelképéhez, hogy a tábla a számomra megfelelő sávkonfigurációt reprezentálja.

3.4.2.15.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, egy jelzőtábla-kompozíció menüje meg van nyitva.
 - Az általam módosítani kívánt jelzőtábla van kiválasztva.
 - A kiválasztott jelzőtábla típusa „Sávbesorolás rendjét jelző tábla” (E-SAVBESOR)
- **When:**
 - Rákattintok a vezérlőpanelen a szerkesztő nézet listája alján lévő, „Új sáv” feliratú gombra.
 - (Opcionális) A listában megjelent új bejegyzésben beállítom a vezérlőkön keresztül a sáv irányait és típusát.
 - Rákattintok az „Alkalmaz” címkéjű gombra.
- **Then:**
 - A menü nézetén frissül a jelzőtábla jelképe az új sávval és a szükséges elválasztó elemekkel.
 - Az kompozíció menüjének bezárását követően tervrajz megjelenítésén frissül a kompozícióban a jelzőtábla jelképe.
 - Az kompozíció menüjének bezárását követően az alkalmazás által tárolt tervrajz modelljében is frissül a jelzőtábla adata.

3.4.2.15.2 2. jelenet

- **Given:**
 - A Táblaszerkesztő lapon vagyok.
 - A szerkesztett jelzőtábla típusa „Sávbesorolás rendjét jelző tábla” (E-SAVBESOR)

- **When:**
 - Rákattintok a vezérlőpanelen a szerkesztő nézet listája alján lévő, „Új sáv” feliratú gombra.
 - (Opcionális) A listában megjelent új bejegyzésben beállítom a vezérlőkön keresztül a sáv irányait és típusát.
 - Rákattintok az „Alkalmaz” címkéjű gombra.
- **Then:**
 - A menü nézetén frissül a jelzőtábla jelképe az új sávval és a szükséges elválasztó elemekkel.

3.4.2.16. Új jelzőlámpa hozzáadása jelzőlámpák csoportjához

Mint felhasználó, szeretnék egy új jelzőlámpát hozzáadni egy jelzőlámpacsoport jelképéhez, hogy a tábla a számomra megfelelő jelzőlámpákat reprezentálja.

3.4.2.16.1 1. jelenet

- **Given:**
 - A Tervrajz lapon vagyok, egy jelzőtábla-kompozíció menüje meg van nyitva.
 - Az általam módosítani kívánt (jelzőtáblaként reprezentált) jelzőlámpacsoport van kiválasztva.
 - A kiválasztott jelzőtábla típusa „Fényjelző készülék(ek)” (JL).
- **When:**
 - Rákattintok a vezérlőpanelen a szerkesztő nézet listája alján lévő, „Új jelzőlámpa” feliratú gombra.
 - (Opcionális) A listában megjelent új bejegyzésben beállítom a vezérlőn keresztül a jelzőlámpa típusát.
 - Rákattintok az „Alkalmaz” címkéjű gombra.
- **Then:**
 - A menü nézetén frissül a jelzőlámpák csoportjának jelképe az új lámpával.
 - Az kompozíció menüjének bezárását követően tervrajz megjelenítésén frissül a kompozícióban a jelzőlámpacsoport jelképe.
 - Az kompozíció menüjének bezárását követően az alkalmazás által tárolt tervrajz modelljében is frissül a jelzőlámpacsoport adata.

3.4.2.16.2 2. jelenet

- **Given:**
 - A Táblaszerkesztő lapon vagyok.
 - A kiválasztott jelzőtábla típusa „Fényjelző készülék(ek)” (JL).
- **When:**
 - Rákattintok a vezérlőpanelen a szerkesztő nézet listája alján lévő, „Új jelzőlámpa” feliratú gombra.
 - (Opcionális) A listában megjelent új bejegyzésben beállítom a vezérlőn keresztül a jelzőlámpa típusát.
 - Rákattintok az „Alkalmaz” címkéjű gombra.
- **Then:**
 - A menü nézetén frissül a jelzőlámpák csoportjának jelképe az új lámpával.

3.4.2.17. Konklúzió a felhasználói történetek tesztelésekor

Az alkalmazás mindegyik jelenet tesztelésekor az elvárt, „then” részben leírtakat produkálja.

3.5. Mesterséges intelligencia használata a fejlesztés során

A mesterséges intelligencia használatának eseteit részleteztem az érintett részekről szóló fejezetekben (pontosabban a 3.2.5.3.3., illetve 3.3.6.1.1. fejezetben). A fejlesztés során sem generált kódot, sem MI eszköz által felvetett ötletet nem alkalmaztam annak ellenőrzése és kritikus átgondolása nélkül. A dolgozat tartalmáért a felelősséget vállalom, beleértve azokat a részeket, melyeknek a megvalósításához MI-t használtam segédeszközként.

4. Összefoglalás és további fejlesztési lehetőségek

A dolgozatban elkészített tervrajzmodell képes tárolni számos alapvető adatot a közutak környezetéről, amit a bevezetőben (1. fejezet) felvetett, megvalósított táblázó program (793.2.5.6. fejezet) képes feldolgozni, és elhelyezni rajta 24 különböző típusú jelzőtáblát, melyből 5 az elhelyezés környezete alapján egyedi adatokkal rendelkezik. A tervrajztesztek alapján (3.4.1. fejezet) a táblázó program a jelzőtáblák nagy részét jól helyezi el, egyes úthálózatokban hiba nélkül, de máshol, főleg bonyolultabb közúti csomópontok esetén, hibázhat, mely eredhet a térképes adatok forrásaként használt OpenStreetMap adataiból, a

tervrajzzá alakításkor alkalmazott heurisztikák hibáiból, vagy magának a táblázó programnak a korlátozásaiból.

A jelzőtábla-kompozíciókban elhelyezett jelzőtáblák a tervrajz részét képezik, a szerver mellé fejlesztett alkalmazás képes prezentálni a táblázott tervrajzot. A megjelenítéshez az egyes jelzőtáblák jelképeit a szerver prezentációs komponense (3.2.6. fejezet) elkészíti, testre szabható táblák esetén a tábla adatai alapján szerkesztve a jelkép grafikáját. Az alkalmazásban a tervrajz táblázása szerkeszthető, a jelzőtábla-kompozíciók törölhetők, módosíthatók, létrehozhatók a felhasználó által kívánt táblákkal.

5. Irodalomjegyzék

- [1] 83/2004. (VI. 4.) GKM rendelet a közúti jelzőtáblák megtervezésének, alkalmazásának és elhelyezésének követelményeiről (jogsabály) (<https://net.jogtar.hu/jogszabaly?docid=a0400083.gkm>) (elérés dátuma: 2026. 04. 28.)
- [2] Google Térkép: utcai felvétel, helyszín: Esztergom, Kiss János vezérezredes út-Baross Gábor út kereszteződése, felvétel dátuma: 2022. október (utcakép) (<https://maps.app.goo.gl/J1uerzon4whkdzCy6>) (elérés dátuma: 2026. 04. 28.)
- [3] Oracle JDK 21 Certified System Configurations (weboldal) (<https://www.oracle.com/java/technologies/javase/products-doc-jdk21certconfig.html>) (elérés dátuma: 2026. 04. 28.)
- [4] GitHub: Electron README fájlja (weboldal) (<https://github.com/electron/electron>) (elérés dátuma: 2026. 04. 28.)
- [5] Quarkus: Introduction to Contexts and Dependency Injection (CDI) (weboldal) (<https://quarkus.io/guides/cdi#applicationscoped-and-singleton-look-very-similar-which-one-should-i-choose-for-my-quarkus-application>) (elérés dátuma: 2026. 04. 28.)
- [6] IBM Technology: Spring Boot versus Quarkus (YouTube) (videó) (<https://www.youtube.com/watch?v=mJJpZ70q9M0>) (elérés dátuma: 2026. 04. 28.)
- [7] Quarkus.io: Extensions (weboldal) (<https://quarkus.io/extensions/>) (elérés dátuma: 2026. 04. 28.)
- [8] Red Hat: 10 reasons to build your next Java application with Quarkus (weboldal) (<https://www.redhat.com/en/resources/10-reasons-for-quarkus-ebook>) (elérés dátuma: 2026. 04. 28.)
- [9] Simple Logging Facade for Java (SLF4J) (weboldal) (<https://www.slf4j.org/>) (elérés dátuma: 2026. 04. 28.)
- [10] GitHub: Quarkus Batik README fájlja (weboldal) (<https://github.com/quarkiverse/quarkus-batik>) (elérés dátuma: 2026. 04. 28.)
- [11] MDN HTTP Reference: GET Method (weboldal) (<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods/GET>) (elérés dátuma: 2026. 04. 29.)

- [12] Oliver Drotbohm: Whoops! Where did my architecture go (weboldal) (<https://odrotbohm.de/2013/01/whoops-where-did-my-architecture-go/>) (elérés dátuma: 2026. 04. 28.)
- [13] OpenStreetMap Wiki: Overpass API/Overpass QL (weboldal) (https://wiki.openstreetmap.org/wiki/Overpass_API/Overpass_QL) (elérés dátuma: 2026. 04. 29.)
- [14] JOSM honlapja (weboldal) (<https://josm.openstreetmap.de/>) (elérés dátuma: 2026. 04. 28.)
- [15] John P. Snyder: Map projections: A working manual, U.S. Government Printing Office, 1987 (publikáció) (<https://pubs.usgs.gov/pp/1395/report.pdf>) (elérés dátuma: 2026. 04. 29.)
- [16] OSM Wiki: Key:urban (sic) (weboldal) (<https://wiki.openstreetmap.org/wiki/Key:urban>) (elérés dátuma: 2026. 04. 29.)
- [17] e-UT 03.04.13:2019/M2:2025 - Kerékpározható közutak tervezése (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1751>) (elérés dátuma: 2026. 04. 29.)
- [18] e-UT 04.02.11 - Közúti jelzőtáblák (T): A jelzőtáblák megtervezése, alkalmazása és elhelyezése (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/838>) (elérés dátuma: 2026. 04. 29.)
- [19] e-UT 04.02.12 - Közúti jelzőtáblák (Y): A feliratok betűi, számjegyei és írásjelei (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1317>) (elérés dátuma: 2026. 04. 29.)
- [20] e-UT 04.02.21 - Közúti jelzőtáblák (A): Veszélyre figyelmeztető jelzőtáblák és jelképeik (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1318>) (elérés dátuma: 2026. 04. 29.)
- [21] e-UT 04.02.22 - Közúti jelzőtáblák (B): Elsőbbségi jelzőtáblák és jelképeik (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1319>) (elérés dátuma: 2026. 04. 29.)
- [22] e-UT 04.02.23 - Közúti jelzőtáblák (C): Tiltó vagy korlátozó jelzőtáblák és jelképeik (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1320>) (elérés dátuma: 2026. 04. 29.)
- [23] e-UT 04.02.24 - Közúti jelzőtáblák (D): Kötelező jelzőtáblák és jelképeik (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1321>) (elérés dátuma: 2026. 04. 29.)

- [24] e-UT 04.02.31- Közúti jelzőtáblák (E): Különleges szabályokat jelző táblák és jelképeik (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1379>) (elérés dátuma: 2026. 04. 29.)
- [25] e-UT 04.02.32 - Közúti jelzőtáblák (G): Útbaigazító és utaló jelzőtáblák és jelképeik (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/864>) (elérés dátuma: 2026. 04. 29.)
- [26] e-UT 03.03.32:2022/M1:2023 - A jelzőlámpás forgalomirányítás tervezése, telepítése és üzemeltetése (útügyi műszaki előírás) (<https://ume.kozut.hu/dokumentum/1300>) (elérés dátuma: 2026. 04. 30.)
- [27] MDN Web Docs: Document Object Model (DOM) (weboldal) (https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model) (elérés dátuma: 2026. 04. 29.)
- [28] Electron – Introduction (weboldal) (<https://www.electronjs.org/docs/latest>) (elérés dátuma: 2026. 04. 29.)
- [29] TypeScript – The Basics (weboldal) (<https://www.typescriptlang.org/docs/handbook/2/basic-types.html>) (elérés dátuma: 2026. 04. 29.)
- [30] GoJS – Introduction to GoJS Diagramming Components (weboldal) (<https://gojs.net/latest/intro/>) (elérés dátuma: 2026. 04. 29.)
- [31] Quasar – Why Quasar? (weboldal) (<https://quasar.dev/introduction-to-quasar>) (elérés dátuma: 2026. 04. 30.)
- [32] Vue.js – Introduction (weboldal) (<https://vuejs.org/guide/introduction>) (elérés dátuma: 2026. 04. 30.)
- [33] Verdana Typeface: Best Choice for Technical Writing, Computer Display, and Industrial Component Labeling (weboldal) (<https://www.linkedin.com/pulse/verdana-typeface-best-choice-technical-writing-computer-eason>) (elérés dátuma: 2026. 04. 30.)
- [34] Leaflet API reference (weboldal) (<https://leafletjs.com/reference.html>) (elérés dátuma: 2026. 04. 30.)
- [35] Martin Fowler: Given When Then (weboldal) (<https://martinfowler.com/bliki/GivenWhenThen.html>) (elérés dátuma: 2026. 04. 28.)

- [36] 41/2003. (VI. 20.) GKM rendelet a forgalomirányító jelzőlámpák követelményeiről, tervezési, telepítési és üzemeltetési előírásairól (jogsabály) (<https://net.jogtar.hu/jogszabaly?docid=a0300041.gkm>) (elérés dátuma: 2026. 04. 28.)